China Daheng Group, Inc. Beijing Image Vision Technology Branch

DotNET SDK Programmer's Guide

Version：V1.4.3

Date：2026-05-06

DAHENG IMAGING 大恒图像

All rights reserved. No parts of this manual may be used or reproduced, in any forms or by any means, without prior written permission of China Daheng Group, Inc. Beijing Image Vision Technology Branch.

The right is also reserved to modify or change any parts of this manual in the future without prior notification.

All other trademarks are the properties of their respective owners.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Web: www.daheng-imaging.com/en

Sales Email: isales@daheng-imaging.com

Sales Tel: +86 10 8282 8878-8081

Support Email: isupport@daheng-imaging.com

Contents

1. Introduction 1
1.1. Module Introduction 1
1.2. Supported Framework 1
2. Programming Guide 2
2.1. Build Programming Environment 2
2.1.1. Using the Wizard to Create a Console Application 2
2.1.2. Add the GxIAPINET.dll to the Project Manually 2
2.1.3. How to Call the GxIAPINET Library 4
2.2. Precautions for Debugging the GigE Vision Cameras 4
2.3. Initialization and Uninitialization 5
2.4. Error Handling 5
2.5. Enumerating Devices 6
2.6. Get Interface Information 6
2.7. Get Interface Object 7
2.8. Configure Camera IP Address 7
2.9. Open and Close Devices 8
2.10. Features Control 9
2.10.1. Controller Type 9
2.10.2. Data Node Type 11
2.10.3. Access Type 11
2.10.4. How to Get the Camera's Parameters 11
2.10.5. How to Get the Sample Code for Feature Read & Write 12
2.10.6. Different Types of Feature Sample Code 13
2.10.7. Precautions 15
2.11. Acquisition Control and Image Processing 15
2.11.1. DQBuf 16
2.11.2. GetImage 17
2.11.3. Acquisition with Callback Functions 18
2.11.4. Set the Number of Acquisition Buffer 19
2.11.5. Image Processing 20
2.11.6. Stream Object Features Control 27
2.11.7. Frame Information 28
2.11.8. Precautions 32
2.12. Get Devices Events 32
2.12.1. Selection Events 32
2.12.2. Enabled Events 32
2.12.3. Registered Events Message Callback Function 33
2.12.4. Get Events Data Information 33
2.12.5. Sample Code 33
2.13. Reconnection 35

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.13.1. Procedure 35
2.13.2. Device Offline Event 35
2.13.3. Device Disconnection Event and Device Reconnection Event 36
2.14. Import and Export Cameras' Configuration Parameters 40
2.15. Properties Setting UI 40
2.15.1. Feature Description 40
2.15.2. Sample Code 42
2.15.3. Notes 43
2.16. Saving Images and Videos 43
2.16.1. Saving Images 43
2.16.2. Saving Videos 44
3. Data Type Definition 46
3.1. GX_STATUS_LIST 46
3.2. GX_LOG_TYPE_LIST 46
3.3. GX_DEVICE_CLASS_LIST 47
3.4. GX_ACCESS_STATUS 47
3.5. GX_ACCESS_MODE 47
3.6. GX_PIXEL_FORMAT_ENTRY 47
3.7. GX_FRAME_STATUS_LIST 50
3.8. GX_FEATURE_TYPE 50
3.9. GX_BAYER_CONVERT_TYPE_LIST 51
3.10. GX_VALLID_BIT_LIST 51
3.11. GX_IP_CONFIGURE_MODE 51
3.12. GX_RESET_DEVICE_MODE 51
3.13. COLOR_TRANSFORM_FACTOR 51
3.14. GX_TL_TYPE_LIST 52
3.15. GX_ACTUAL_BITS 52
3.16. GX_PIXEL_COLOR_FILTER 52
3.17. GX_STATIC_DEFECT_CORRECTION 52
3.18. GX_IMAGE_INFO 53
3.19. GX_REGISTER_STACK_ENTRY 53
3.20. GX_GIGE_ACTION_COMMAND_RESULT 53
3.21. GX_FLAT_FIELD_CORRECTION_PARAMETER 53
3.22. GX_ENUM_ITEM_VALUE 54
3.23. GX_WINDOWS_ID 54
3.24. GX_WINDOWS_SHOW_MODE 54
3.25. GX_CFA_METHOD 54
3.26. GX_IMAGE_FORMAT_TYPE 54
3.27. GX_VIDEO_FORMAT_TYPE 55
3.28. GX_SAVE_IMAGE_INFO 55

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3.29. GX_RECORD_PARAM 55
4. Handle Type Definition 56
5. Delegate Function Type Definition 57
6. Module Interface Definition 58
6.1. IGXFactory 58
6.1.1. GetInstance 58
6.1.2. Init 59
6.1.3. Uninit 59
6.1.4. SetLogType 59
6.1.5. GetLogType 59
6.1.6. UpdateDeviceList 59
6.1.7. UpdateAllDeviceList 60
6.1.8. UpdateAllDeviceListEx 60
6.1.9. OpenDeviceByIP/MAC/SN/UserID 60
6.1.10. GigEIPConfiguration 60
6.1.11. GigEForceIP 61
6.1.12. GigEResetDevice 61
6.1.13. CreateImageFormatConvert 62
6.1.14. CreateImageSpecialFunction 62
6.1.15. GetAllInterfaceInfo 62
6.1.16. GetInterface 62
6.1.17. CreateImageProcess 62
6.1.18. GigEIssueActionCommand 63
6.1.19. GigEIssueScheduledActionCommand 63
6.1.20. CreateDecompressor 64
6.1.21. SaveImage 64
6.1.22. CreateVideoSaver 65
6.2. IGXInterfaceInfo 65
6.2.1. GetType 65
6.2.2. GetModelName 65
6.2.3. GetDisplayName 65
6.2.4. GetSerialNumber 66
6.2.5. GetInterfaceID 66
6.2.6. GetInitFlag 66
6.2.7. GetVendorName 66
6.3. IGXInterface 66
6.3.1. GetInterfaceInfo 66
6.3.2. GetAllDeviceInfo 67
6.3.3. GetFeatureControl 67
6.4. IGXDeviceInfo 67
6.4.1. GetVendorName 68
6.4.2. GetModelName 68
6.4.3. GetSN 68

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.4.4. GetDisplayName 68
6.4.5. GetDeviceID 68
6.4.6. GetUserID 68
6.4.7. GetAccessStatus 69
6.4.8. GetDeviceClass 69
6.4.9. GetMAC 69
6.4.10. GetIP 69
6.4.11. GetSubnetMask 69
6.4.12. GetGateway 69
6.4.13. GetNICMAC 70
6.4.14. GetNICIP 70
6.4.15. GetNICSubnetMask 70
6.4.16. GetNICGateway 70
6.4.17. GetNICDescription 70
6.5. IGXDevice 70
6.5.1. GetDeviceInfo 71
6.5.2. GetStreamCount 71
6.5.3. OpenStream 72
6.5.4. GetFeatureControl 72
6.5.5. GetRemoteFeatureControl 72
6.5.6. GetEventNumInQueue 72
6.5.7. FlushEvent 72
6.5.8. RegisterDeviceOfflineCallback 72
6.5.9. UnregisterDeviceOfflineCallback 73
6.5.10. RegisterDeviceReconnectCallback 73
6.5.11. UnregisterDeviceReconnectCallback 73
6.5.12. RegisterDeviceDisconnectCallback 73
6.5.13. UnregisterDeviceDisconnectCallback 73
6.5.14. CreateImageProcessConfig 73
6.5.15. ExportConfigFile 74
6.5.16. ImportConfigFile 74
6.5.17. ExportConfigFileW 74
6.5.18. ImportConfigFileW 74
6.5.19. Close 74
6.5.20. GetParentInterface 74
6.5.21. GXCreateWnd 75
6.5.22. GXDestroyWnd 75
6.5.23. GXSetShowPosition 75
6.5.24. GXSetShowMode 75
6.5.25. GXShowWnd 75
6.5.26. GXSetWndTitle 75
6.6. IGXFeatureControl 76
6.6.1. GetFeatureNameList 76
6.6.2. GetFeatureType 77

6.6.3. IsImplemented....77
6.6.4. IsReadable....77
6.6.5. IsWritable....77
6.6.6. GetIntFeature....77
6.6.7. GetFloatFeature....77
6.6.8. GetEnumFeature....78
6.6.9. GetBoolFeature....78
6.6.10. GetStringFeature....78
6.6.11. GetCommandFeature....78
6.6.12. GetRegisterFeature....78
6.6.13. RegisterFeatureCallback....78
6.6.14. UnregisterFeatureCallback....79
6.6.15. ReadPort....79
6.6.16. WritePort....79
6.6.17. ReadPortStacked....79
6.6.18. WritePortStacked....80
6.6.19. FeatureSave....80
6.6.20. FeatureSaveW....80
6.6.21. FeatureLoad....81
6.6.22. FeatureLoadW....81
6.7. IIntFeature....81
6.7.1. GetMin....81
6.7.2. GetMax....81
6.7.3. GetInc....81
6.7.4. GetValue....82
6.7.5. SetValue....82
6.8. IFloatFeature....82
6.8.1. GetMin....82
6.8.2. GetMax....82
6.8.3. HasInc....83
6.8.4. GetInc....83
6.8.5. GetUnit....83
6.8.6. GetValue....83
6.8.7. SetValue....83
6.9. IEnumFeature....83
6.9.1. GetEnumEntryList....84
6.9.2. GetValue....84
6.9.3. SetValue....84
6.9.4. SetEnumValue....84
6.9.5. GetEnumValue....84
6.9.6. GetEnumValue....84
6.10. IBoolFeatur....85
6.10.1. GetValue....85
6.10.2. SetValue....85

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.11. IStringFeature 85
6.11.1. GetValue 85
6.11.2. SetValue 86
6.11.3. GetStringMaxLength 86
6.12. ICommandFeature 86
6.12.1. Execute 86
6.13. IRegisterFeature 86
6.13.1. GetLength 86
6.13.2. GetBuffer 87
6.13.3. SetBuffer 87
6.14. IGXStream 87
6.14.1. StartGrab 88
6.14.2. StopGrab 88
6.14.3. RegisterCaptureCallback 88
6.14.4. UnregisterCaptureCallback 88
6.14.5. GetImage 89
6.14.6. GetImageNoThrow 89
6.14.7. DQBuf 89
6.14.8. QBuf 89
6.14.9. GetFeatureControl 90
6.14.10. FlushQueue 90
6.14.11. SetAcquisitionBufferNumber 90
6.14.12. Close 90
6.14.13. GetOptimalPacketSize 91
6.14.14. GetPayloadSize 91
6.14.15. RegisterBuffer 91
6.14.16. UnRegisterBuffer 92
6.15. IFrameData 93
6.15.1. GetStatus 93
6.15.2. GetPayloadSize 93
6.15.3. GetWidth 94
6.15.4. GetHeight 94
6.15.5. GetPixelFormat 94
6.15.6. GetFrameID 94
6.15.7. GetTimeStamp 94
6.15.8. GetBuffer 94
6.15.9. ConvertToRaw8 95
6.15.10. ConvertToRGB24 95
6.15.11. ImageProcess 95
6.15.12. GetUserParam 95
6.15.13. GetChunkDataFeatureControl() 96
6.15.14. GetOffsetX() 96
6.15.15. GetOffsetY() 96
6.16. The vertical offset of the image.IImageData 96

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.17. IImagProcessConfig 96
6.17.1. SetValidBit 98
6.17.2. GetValidBit 98
6.17.3. EnableDefectivePixelCorrect 98
6.17.4. IsDefectivePixelCorrect 98
6.17.5. EnableSharpen 98
6.17.6. IsSharpen 98
6.17.7. EnableAccelerate 99
6.17.8. IsAccelerate 99
6.17.9. SetSharpenParam 99
6.17.10. GetSharpenParam 99
6.17.11. SetContrastParam 99
6.17.12. GetContrastParam 100
6.17.13. SetGammaParam 100
6.17.14. GetGammaParam 100
6.17.15. SetLightnessParam 100
6.17.16. GetLightnessParam 100
6.17.17. EnableDenoise 100
6.17.18. IsDenoise 101
6.17.19. SetSaturationParam 101
6.17.20. GetSaturationParam 101
6.17.21. SetConvertType 101
6.17.22. GetConvertType 101
6.17.23. EnableConvertFlip 101
6.17.24. IsConvertFlip 102
6.17.25. EnableColorCorrection 102
6.17.26. IsColorCorrection 102
6.17.27. Reset 102
6.17.28. Destroy 102
6.17.29. IsUserSetCCParam 102
6.17.30. EnableUserSetCCParam 103
6.17.31. SetUserCCParam 103
6.17.32. GetUserCCParam 103
6.18. IGXImageFormatConvert 103
6.18.1. Support conversion types 104
6.18.2. SetDstFormat 105
6.18.3. GetDstFormat 106
6.18.4. SetInterpolationType 106
6.18.5. GetInterpolationType 106
6.18.6. SetAlphaValue 106
6.18.7. GetAlphaValue 106
6.18.8. GetBufferSizeForConversion 107
6.18.9. GetBufferSizeForConversion 107
6.18.10. Convert 107

6.18.11. Convert 108
6.18.12. SetValidBits 108
6.18.13. GetValidBits 108
6.18.14. Set3DCalibParam 109
6.18.15. Get3DCalibParam 109
6.18.16. SetYStep 109
6.18.17. GetYStep 109
6.19. IGXImageProcess 109
6.19.1. StaticDefectCorrection 110
6.19.2. ImageImprovment 110
6.19.3. ImageImprovment 111
6.19.4. CalculaLut 111
6.19.5. ReadLutFile 112
6.20. IGXFlatFieldCorrection 112
6.20.1. SetFrameCount 112
6.20.2. GetCoefficientsSize 113
6.20.3. Calculate 113
6.20.4. FlatFieldCorrection 113
6.21. IGXDecompressor 114
6.21.1. Decompression 114
6.22. IGxVideoSaver 115
6.22.1. AddFrame 115
6.22.2. Close 115
6.23. CgalaxyException 115
6.23.1. GetErrorCode 115
6.24. IGXImageSpecialFunction (No longer maintained) 116
6.24.1. CalculaLut 116
6.24.2. ReadLutFile 116
7. Revision History 118

1. Introduction

1.1. Module Introduction

The GxIAPINET.dll library is an encapsulated general and unified application programming interface which based on the object-oriented method; it is suit for MERCURY, MARS series of DAHENG IMAGING camera.

The main modules and functions as follow:

Main modules	Functions
IGXFactory	Initialize the interface library, enumerate devices, open devices.
IGXInterface	Interface interface, through the interface layer object instance, can obtain interface layer information, obtain the IGXFeatureControl object instance corresponding to the interface layer, obtain the device list enumerated by the interface instance and other functions
IGXDevice	Device object, using this object as an entry point for attribute control, image acquisition, camera event retrieval, etc
IGXStream	Stream object, obtained from IGXDevice, specifically responsible for image acquisition related functions
IGXFeatureControl	Attribute control objects, which can be obtained from IGXInterface, IGXDevice, and IGXStream respectively, are responsible for various attribute controls
IImageData	The image structure of callback acquisition and single frame acquisition, including acquisition output results such as image buffer and image information
IGXImageProcess	Image enhancement, Lut processing and other functions

Table 1-1 Main modules and functions

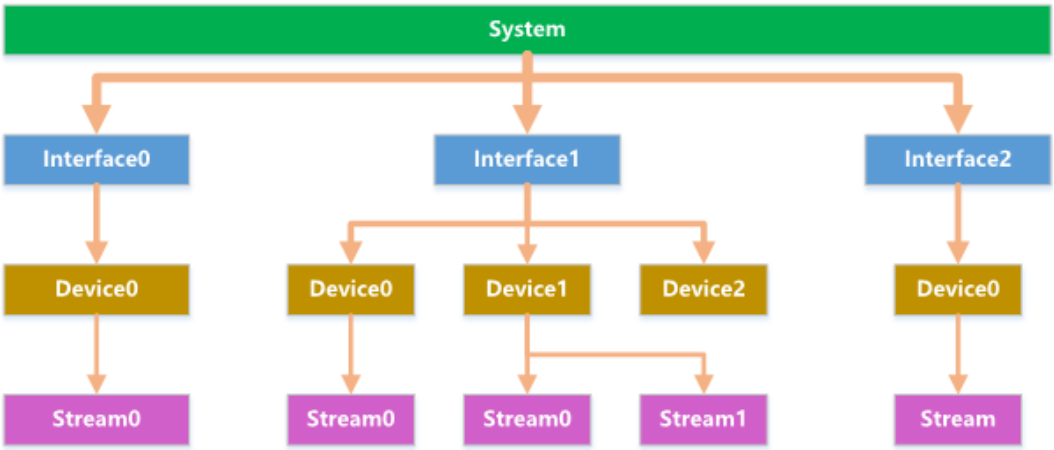


Figure 1-1 Module hierarchy diagram

1.2. Supported Framework

The supported versions of the .NET Framework are .net 3.5 /.net 4.0, using the GxIAPINET.dll file in the corresponding directory.

The supported versions of .NET Core are .net 6.0 and above, using the GxIAPINET.dll file located in the .net 6.0 directory.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2. Programming Guide

2.1. Build Programming Environment

2.1.1. Using the Wizard to Create a Console Application

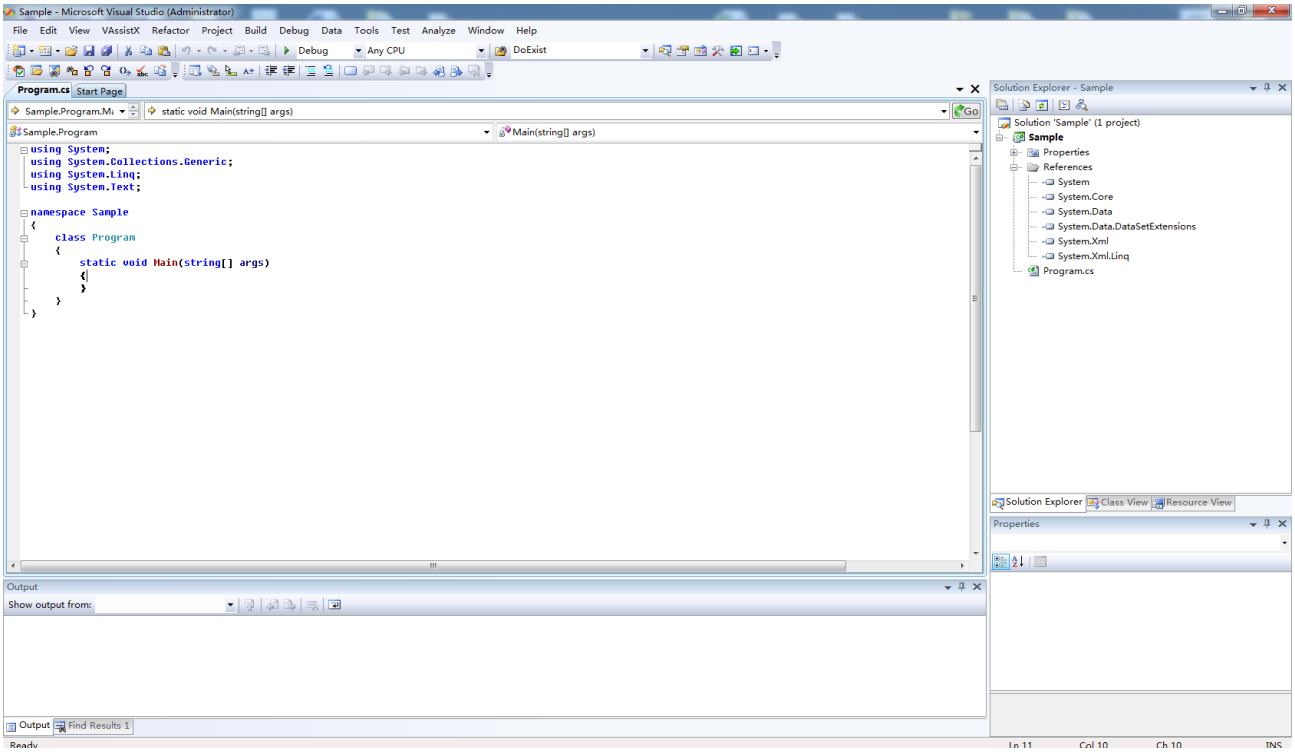


Figure 2-1

2.1.2. Add the GxIAPINET.dll to the Project Manually

Right click the **References** in the solution explorer, select **AddReference**, as shown in the following figure:

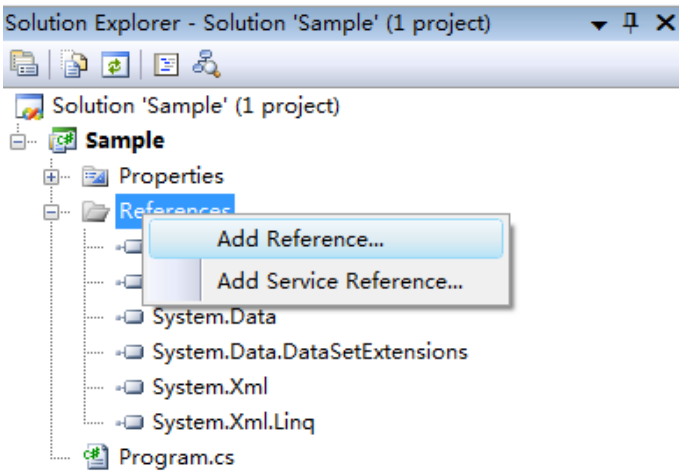


Figure 2-2

In the pop-up **Add Reference** dialog box, select **Browse** tab and select the **GxIAPINET.dll** file, then click **OK**, see the following figure.

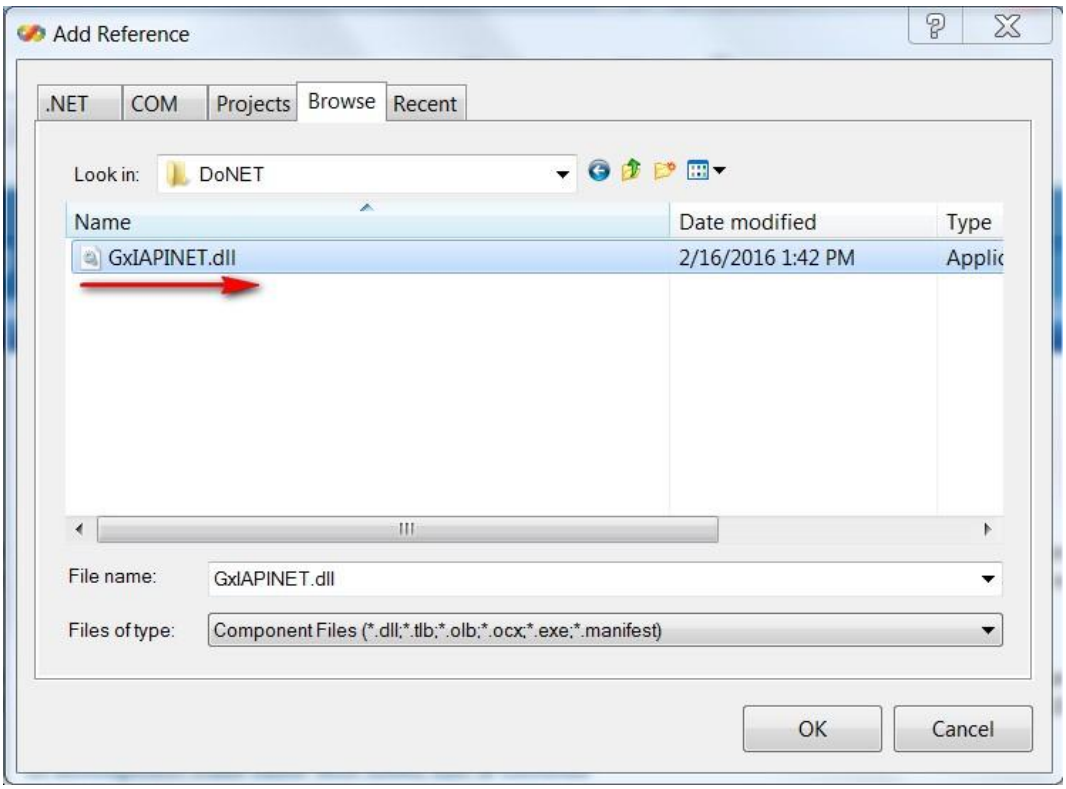


Figure 2-3

Click **OK** and we can see the library which we have selected in the **Reference**, see the following figure:

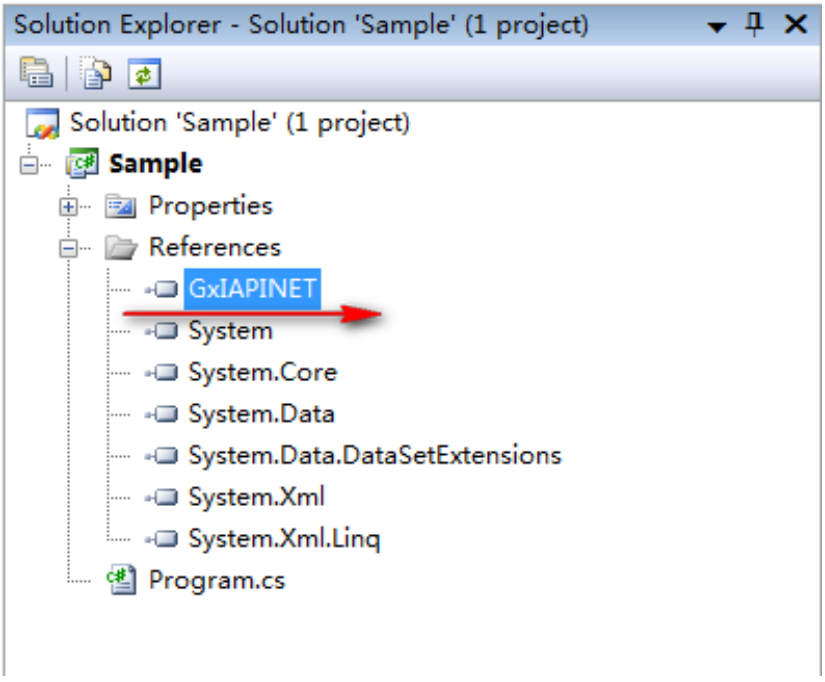


Figure 2-4

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.1.3. How to Call the GxIAPINET Library

Before calling the library, you should add a using namespace **GXIAPINET** in the Code Editor area, see the following figure:

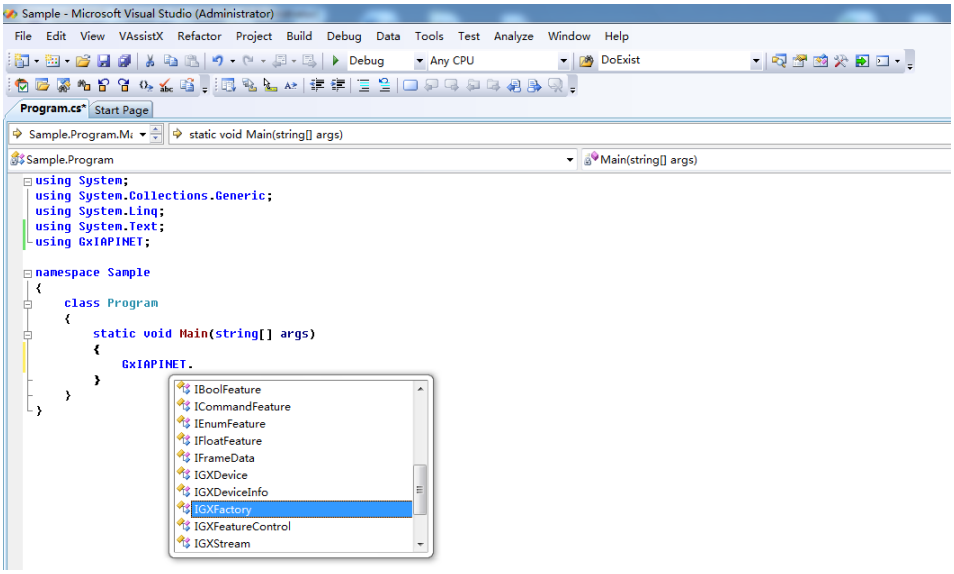


Figure 2-5

2.2. Precautions for Debugging the GigE Vision Cameras

When the Windows users use Visual Studio development platform debugging the GigE Vision cameras, they may encounter device offline, which is caused by the heartbeat timeout. The application must send heartbeat packet to the camera in defined intervals. If the camera doesn't receive this heartbeat, it will consider the current connection as broken and won't accept any commands from the application.

When the user running the application, it will send heartbeat packet normally and keep connect with the camera, but in debug mode, when the application runs to the breakpoint, the debugger will suspend all the thread, including the sending heartbeat packet thread, so in debug mode, the application will not send heartbeat packet to the camera.

Users can add heartbeat timeout to resolve it.

Users can change the heartbeat timeout by two ways:

The first one is: open the device in the program and add the following code:

Only in debugging mode can use this code snippet, don't use it in the release program.

```
//Open device, here, open the device through Serial Number (SN), the SN will
//be subject to the actual device. Users can also open the device by other ways.
#if DEBUG

IGXDevice objIGXDevice = objIGXFactory.OpenDeviceBySN("RN0001007012",
                                                      GX_ACCESS_EXCLUSIVE);

//Get remote device feature controller.
IGXFeatureControl objFeatureControl = objIGXDevice.GetRemoteFeatureControl();

//Set heratbeat timeout for 5 minutes.
objFeatureControl.GetIntFeature("GevHeartbeatTimeout").SetValue(300000);
```

```
#endif
```

The other way is: add environment variable 'MER_HEARTBEAT_TIMEOUT' in the system, and assign a value greater than zero, then open the device with application program, the device's heartbeat timeout will automatically become the environment variable's value. Only to add the environment variable in the development system and they will work in the application no matter in debug mode or release mode.

Note: If the user set the heartbeat timeout very long, when close the program it will not call the close device method to close the device, this will lead to the device cannot to be reset in the heartbeat time, thus when the user try again to open the device, it will fail. This problem can be solved by resetting the device.

There are two methods to reset the device:

1) Select “Reset Device” button directly in the IP Configurator.

2) Reset the device by the interface 'GXGigEResetDevice'.

Reset device: It is equivalent to powering off and powering up the device, and the programs in the camera are completely reloaded.

2.3. Initialization and Uninitialization

The **GxIAPINET** library must be initialized before use. Before using any other methods, the **IGXFactory.GetInstance ().Init ()** must be called first.

```
//Call this function to initialize before using any other methods.
IGXFactory.GetInstance().Init();
```

The **IGXFactory.GetInstance().Uninit()** must be called to release all the resources of the **GxIAPINET** application before the process exits.

```
//After closing the device, can't call any other interface library.
IGXFactory.GetInstance().Uninit();
```

2.4. Error Handling

All the **GxIAPINET** API functions throw exceptions when errors are detected. The exception type is **CGalaxyException**, which inherited from the Exception. Users can call the function **CGalaxyException::GetErrorCode ()** to get the error codes, and the error code type can be referred to GX_STATUS_LIST. Users can also access the exception class properties by **CGalaxyException.Message** to get detailed error description information string.

A typical error handling code might look like:

```
try
{
    IGXFactory.GetInstance().Init();
}
catch (CGalaxyException objError)
{
    String strErrorInfo = "error code:" + objError.GetErrorCode().ToString() +
                                          "error message:" + objError.Message;
    Console.WriteLine(strErrorInfo);
}
```

2.5. Enumerating Devices

Users can call function **IGXFactory.GetInstance().UpdateAllDeviceList()** and **GXFactory::GetInstance().UpdateAllDeviceListEx()** to enumerate all the current available devices, and get a device information list, the list type is **List<IGXDeviceInfo>**, usage of pseudocode snippet as follows:

```
List<IGXDeviceInfo> lstGXDeviceInfo = new(nothrow) List<IGXDeviceInfo>();

//Enumeration timeout assumptions for 200ms.
IGXFactory.GetInstance().UpdateAllDeviceList(200, listGXDeviceInfo);
foreach (IGXDeviceInfo objDeviceInfo in lstGXDeviceInfo)
{
    Console.WriteLine(objDeviceInfo.GetModelName());
    Console.WriteLine(objDeviceInfo.GetVendorName());
    //More devices information can refer IGXDeviceInfo method.
}
List<IGXDeviceInfo> lstGXDeviceInfo = new(nothrow) List<IGXDeviceInfo>();

//Enumeration timeout assumptions for 200ms.
IGXFactory.GetInstance().UpdateAllDeviceListEx(GX_TL_TYPE_USB|
     GX_TL_TYPE_GEV | GX_TL_TYPE_U3V | GX_TL_TYPE_CXP,200, listGXDeviceInfo);
foreach (IGXDeviceInfo objDeviceInfo in lstGXDeviceInfo)
{
    Console.WriteLine(objDeviceInfo.GetModelName());
    Console.WriteLine(objDeviceInfo.GetVendorName());

    //More devices information can refer IGXDeviceInfo method.
}
```

Note: Except the enumerated methods above, **GxIAPINET** also provides a enumerated method: **IGXFactory.GetInstance().UpdateDeviceList**. For non-GigE Vision cameras, they are the same in the function, but for GigE Vision cameras, they are different in the internal enumeration mechanism.

1) UpdateAllDeviceList(): Enumerate in all subnet.
2) UpdateDeviceList(): Enumerate in the same subnet.
3) UpdateDeviceListEx(): Enumerate in all subnet.

2.6. Get Interface Information

Users can get Interface information by calling **IGXFactory::GetInstance().GetAllInterfaceInfo()**.

The parameter type is **GxIAPICPP::gxinterfaceinfo_vector**.

Sample Code:

```
//Get Interface information
List<IGXInterfaceInfo> lstGXInterfaceInfo =
                                new(nothrow) List<IGXInterfaceInfo>();

IGXFactory.GetInstance().GetAllInterfaceInfo(lstGXInterfaceInfo);
for each (IGXInterfaceInfo objInterfaceIofo in lstGXInterfaceInfo)
{
    Console.WriteLine(lstGXInterfaceInfo.GetModelName());
    Console.WriteLine(lstGXInterfaceInfo.GetVendorName());

    //See more details in IGXDeviceInfo interface
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.7. Get Interface Object

Users can get Interface Instance objects by calling **IGXFactory::GetInstance().GetInterface()**

The parameter type is integer, and subscripts start counting from 1.

Sample Code:

```
//Get Interface pointer
IGXInterface objInterface;
objInterface = IGXFactory.GetInstance().GetInterface(1);
```

2.8. Configure Camera IP Address

Users can call the following two different modes to set the IP address of Camera.

IGXFactory::GetInstance().GigEIpConfiguration()

IGXFactory::GetInstance().GigEForceIp()

Use the function **GigEIpConfiguration()** can set the camera static (permanent) IP address, and provide four setting modes: direct write static IP address, use DHCP server to assign IP addresses, use LLA (Link-Local Address) to configure camera IP and use default mode to set IP. When you choose the default mode, the internal camera will enable the other three configure modes, but the static IP as a preferred way to configure camera IP address.

Use the function **GigEForceIp()** can perform ForceIP operations on the camera. ForceIP means that setting the IP address is valid only for this use. When the camera power off and power on will restore the original IP address.

Two ways to configure the IP address of the camera sample code is as follows.

```
//Sample MAC address, actual camera MAC address can be obtained by enumeration.
String strMAC = "00-21-49-00-00-00";
String strIPAddress = "192.168.10.10";
String strSubnetMask = "255.255.255.0";
String strDefaultGateway = "192.168.10.2";
String strUserID = "DAHENG Imaging";
GX_IP_CONFIGURE_MODE  emIpConfigureMode  = IP_CONFIGURE_STATIC_IP;

IGXFactory.GetInstance().GigEIpConfiguration(strMAC, emIpConfigureMode,
                                             strIpAddress, strSubnetMask,
                                             strDefaultGateway, strUserID);
```

```
//Sample MAC address, actual camera MAC address can be obtained by enumeration.
String strMAC = "00-21-49-00-00-00";
String strIPAddress = "192.168.10.10";
String strSubnetMask = "255.255.255.0";
String strDefaultGateway = "192.168.10.2";

//ForceIp
IGXFactory.GetInstance().GigEForceIp (strMAC, strIpAddress, strSubnetMask,
                                      strDefaultGateway);
```

Note:

1) Before calling these two interfaces, you must first enumeration, and when you do this, the camera can't be opened.
2) The maximum length of the user-defined name (UserID) allowed is 16 characters.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.9. Open and Close Devices

Users can call the following four different modes to open the device:

IGXFactory.GetInstance().OpenDeviceBySN()

IGXFactory.GetInstance().OpenDeviceByUserID()

IGXFactory.GetInstance().OpenDeviceByMAC()

IGXFactory.GetInstance().OpenDeviceByIP()

SN: device serial number.

UserID: user defined name (if the device does not support **UserID**, assign this item as a null string).

MAC: the MAC address of the device (for non-GigE Vision cameras, assign this item as a null string).

IP: the device's IP address (for non-GigE Vision cameras, assign this item as a null string).

It is strongly recommended the user to call the enumeration device method and update the **GxIAPINET** library internal device list before open the device, otherwise may open the device failure. The sample code snippet as follows:

```
List<IGXDeviceInfo> lstGXDeviceInfo = new(nothrow) List<IGXDeviceInfo>();
IGXFactory.GetInstance().UpdateAllDeviceList(200, lstGXDeviceInfo);

if (lstGXDeviceInfo.Count> 0)
{
    //Open the first device in the list.
    IGXDevice objDevice = null;
    String strSN     = listGXDeviceInfo[0].GetSN();
    String strUserID = listGXDeviceInfo[0].GetUserID();
    String strMAC    = listGXDeviceInfo[0].GetMAC();
    String strIP     = listGXDeviceInfo[0].GetIP();
    //Users can also assign the device information of the opened device
    //directly. The information shown in the following code is indicative only,
    //the actual information may differ.
    //String strSN     = "GA0140100002";
    //String strUserID = "MyUserName";
    //String strMAC    = "A1-0B-32-7C-6F-81";
    //String strIP     = "192.168.0.100";
    objDevice = IGXFactory.GetInstance().OpenDeviceBySN(strSN,
                                                        GX_ACCESS_EXCLUSIVE);
    //objDevice = IGXFactory.GetInstance().OpenDeviceByUserID(strUserID,
                                                        GX_ACCESS_EXCLUSIVE);
    //objDevice = IGXFactory.GetInstance().OpenDeviceByMAC(strMAC,
                                                        GX_ACCESS_EXCLUSIVE);
    //objDevice = IGXFactory.GetInstance().OpenDeviceByIP(strIP,
                                                        GX_ACCESS_EXCLUSIVE);
}
```

Note: For GigE Vision devices, users can open the device directly without enumeration, because the network mechanism allows to establishment connection directly by IP address.

Users can call the **IGXDevice::Close** to close device, release all the device resources. Code snippet as follows:

```
//After closing the device, user can not to call the function 'IDevice' and
//all the 'IFeatureControl&IStream' functions of the device.
objDevice.Close();
```

Note: After closing the device, all the resources about the device will be released, including the **IGXFeatureControl** object and **IGXStream** object which get from the device object, so when the device is closed, these objects' interface are not allowed to be called again.

2.10. Features Control

2.10.1. Controller Type

GenTL (Generic Transport Layer) is a part of the GenICam (Generic Interface for Cameras) standard, which defines a communication abstraction layer between cameras and applications, supporting different interfaces such as GigE Vision, USB3 Vision, etc. The module division of GenTL aims to simplify hardware access and provide a unified operating method. GenTL mainly abstracts and divides the following modules:

1) System is the top-level layer used for enumerating and managing interfaces. A System layer can enumerate and open multiple Interface layer instances.
2) Interface is an abstraction layer for specific physical interfaces, responsible for enumerating and opening devices connected under the interface layer.
3) Device is an abstraction layer for communication with remote physical devices (cameras) under the interface, which has two concepts: remote devices and local devices.
    a) Remote device: This module is responsible for communicating with the device, enumerating and controlling camera functions. Camera functions such as gain and exposure.
    b) The local device is responsible for enumerating and instantiating the DataStream module, and providing module configuration functions to users (such as the TL corresponding to the POLS series frame grabber, which implements the BayerConvert function on the frame grabber side. Whether to enable the BayerConvert function for specific cameras under the frame grabber can be configured in the local device layer).
4) DataStream manages the data stream output by devices (such as image transmission), and a device can have one or more DataStream instances.

GxIAPCPPEx is implemented based on the GenTL standard encapsulation. To achieve the corresponding module configuration, GxIAPICPPEx abstracts IGXFeatureControl to achieve completion control of visual components. In GxIAPICPPEx, the above concepts are abstracted into the following four instances of attribute controllers:

1) IGXFeatureControl instance for Device (remote):

    Obtain by IGXDevice::GetRemoteFeatureControl(). The remote device attribute control corresponds to the remote device in GenTL, which realizes camera functions such as width, height, exposure, gain, enumeration control, etc. Generally, users mainly use this attribute controller.
2) IGXFeature Control instance for Device (Local):
    Obtain by IGXDevice::GetFeatureControl().This instance corresponds to the local device in GenTL and implements a functional configuration similar to the local device module in GenTL. When users use the POLS series frame grabber, they can use this attribute controller if they use functions similar to Bayercovert.
3) IGXFeatureControl instance of Datastream layer:
    Obtain by IGXStream::GetFeatureControl().This instance corresponds to the configuration and retrieval of the Datastream layer in GenTL, such as acquisition data statistics. Users can use this

attribute controller to monitor information such as the number of acquisition frames and whether frames have been lost.

4) IGXFeatureControl instance of Interface layer:

Obtain by IGXInterface::GetFeatureControl(). Contains main interface information, and different types of interfaces have different functions. For example, the POLS series frame grabber provides rich I/O configuration functions, and users can control it through this instance.

Running the **GalaxyView** demo to open the device, you can see the parameters in the right properties controller tree, there are three parts: the upper one is the properties which return from **IGXDevice::GetRemoteFeatureControl()**, and the middle one is the properties which return from **IGXDevice::GetFeatureControl()**. The lower one is the return properties from **IGXStream::GetFeatureControl()**.

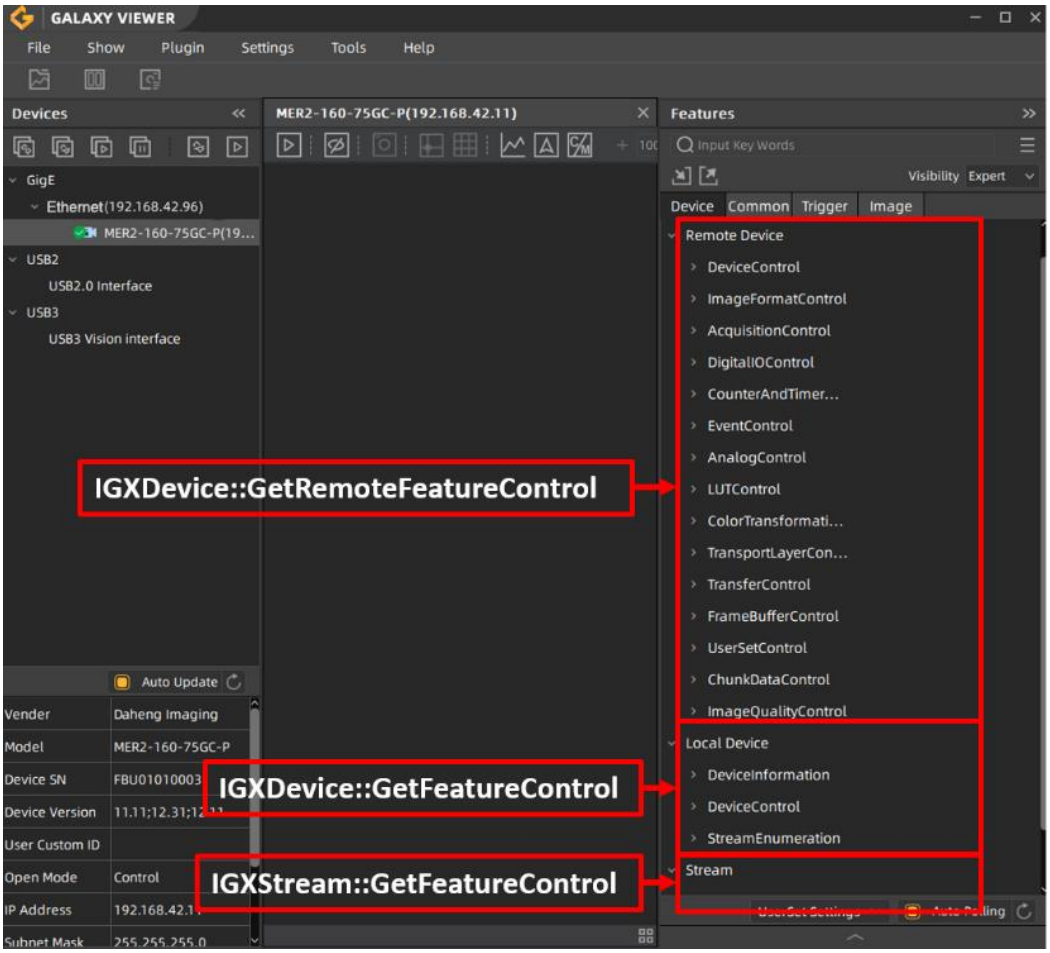


Figure 2-6

Click interface node in GalaxyView and you can see the property tree on the right side, the corresponding functions are properties returned by IGXInterface::GetFeatureControl(). See Figure 2-7

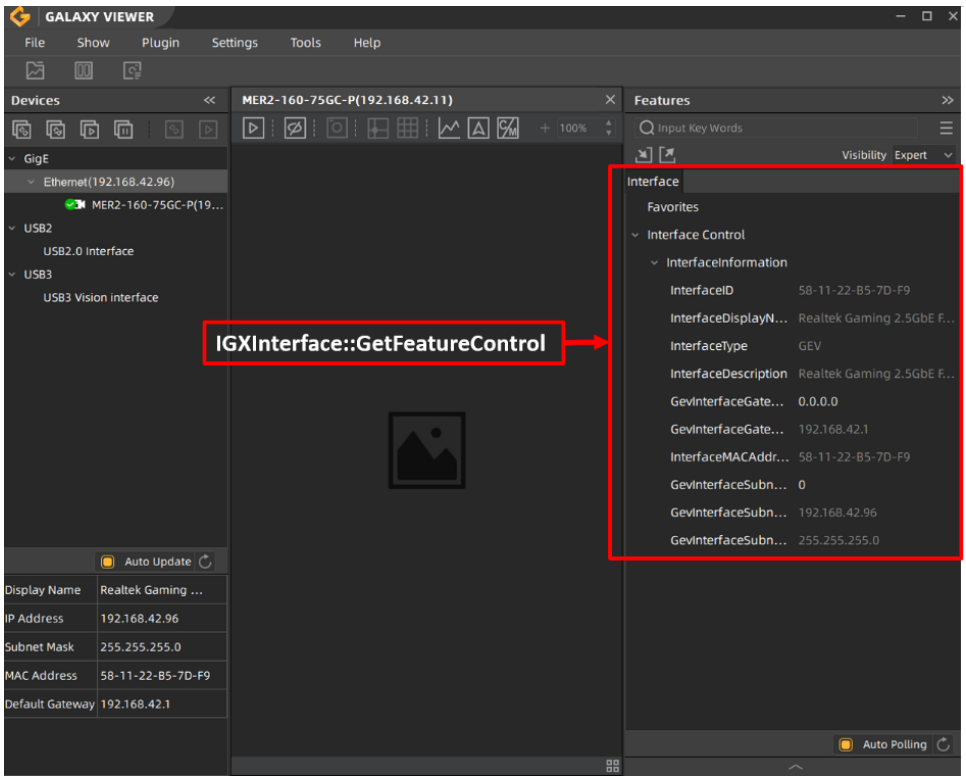


Figure 2-7 Interface properties

2.10.2. Data Node Type

There are seven data types, see as follows:

1) IIntFeatureIGXFeatureControl::GetIntFeature() //integer data node
2) IFloatFeatureIGXFeatureControl::GetFloatFeature() //float data node
3) IBoolFeatureIGXFeatureControl::GetBoolFeature() //Boolean data node
4) IEnumFeatureIGXFeatureControl::GetEnumFeature() //enumerated data node
5) IStringFeatureIGXFeatureControl::GetStringFeature() //string data node
6) ICommandFeatureIGXFeatureControl::GetCommandFeature() //command data node
7) IRegisterFeatureIGXFeatureControl::GetRegisterFeature() //register data node

2.10.3. Access Type

There are three access types, see as follows:

1) bool IGXFeatureControl.IsImplemented() //Is the data node implemented?
2) bool IGXFeatureControl.IsReadable() //Is the data node readable?
3) bool IGXFeatureControl.IsWritable() //Is the data node writable?

2.10.4. How to Get the Camera's Parameters

Get from the function: IGXFeatureControl.GetFeatureNameList(), returns the function name string which the current features control supported.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Get from the **GalaxyView** demo: open the device by **GalaxyView** demo, look over the right side—properties list, the English name of every control node, as shown in the following figure:

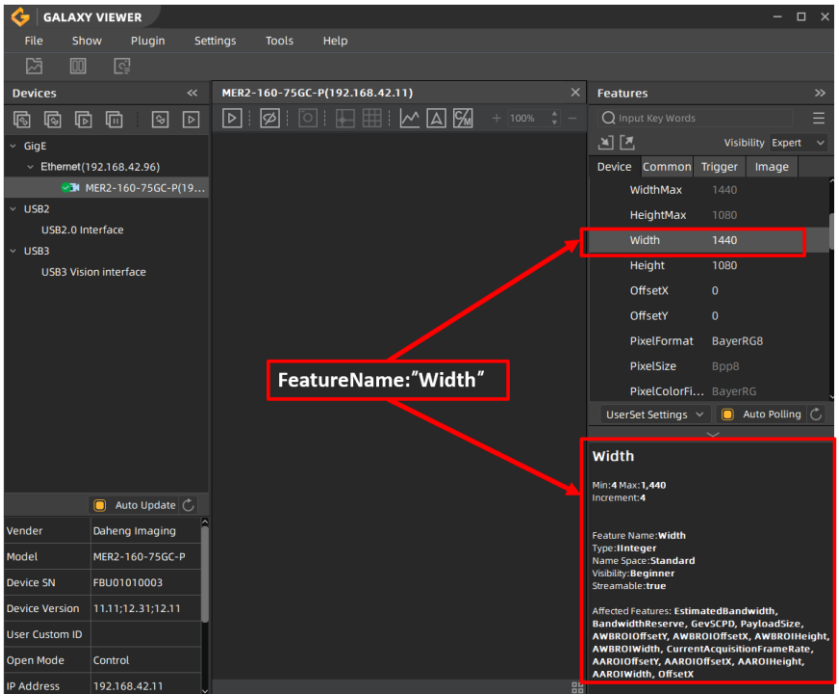


Figure 2-8

As shown above, when the mouse pitch on **image width**, the function name, maximum, minimum, step etc. will be shown in the **Function Properties Description Area**.

Note: The function name string is case sensitive.

2.10.5. How to Get the Sample Code for Feature Read & Write

The GalaxyView.exe provides sample codes for feature read & write. In the demo program menu bar: View -> Feature Document, you can open the window, as shown in the following:

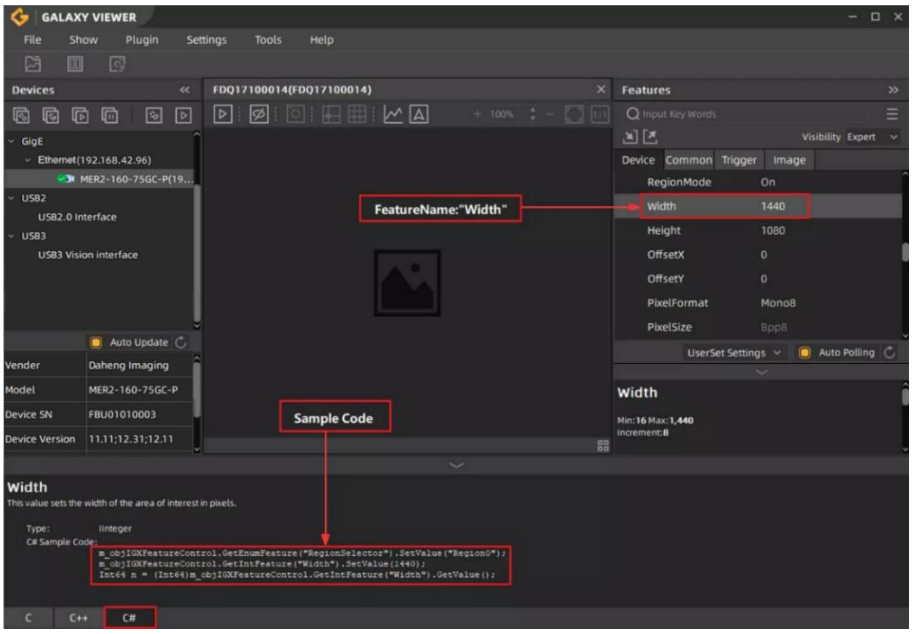


Figure 2-9

In the view of feature document, three development languages are available. Switch to C# Tab to get the code for reading and writing user-specified feature in C# language. The code can be directly copied to the user's development project.

2.10.6. Different Types of Feature Sample Code

2.10.6.1. Read & Write Access Control

All functions are controlled by function string. Check to see if the feature is currently available, and then to read or write.

```
//Obtain remote device attribute controller
IGXFeature Control objIGXFeatureControl =
                                        objIGXDevice.GetRemoteFeatureControl();
//Retrieve the read and write properties of the current data node
Boolean bIsImplemented = objIGXFeatureControl.IsImplemented("Width");

if (bIsImplemented)
{
    //Is readable?
    Boolean bIsReadable = objIGXFeatureControl.IsReadable("Width");
    //Is writable?
    Boolean bIsWritable = objIGXFeatureControl.IsWritable("Width");
}
```

2.10.6.2. Integer Data Node

According to the data type, the camera's function is divided into seven categories, integer is one of them, including width, height and other integer control parameters, users can use the integer controller **IntFeature** to read the current value, inquire the maximum, minimum and step.

```
//Obtain remote device attribute controller
IGXFeatureControl objIGXFeatureControl =
                                        objIGXDevice.GetRemoteFeatureControl();
//Get integer data nodes
IIntFeature objInt = objIGXFeatureControl.GetIntFeature("Width");

//Get the maximum value.
Int64 i64Max = objInt.GetMax();

//Get the minimum value.
Int64 i64Min = objInt.GetMin();

//Get the increment value.
Int64 i64Inc = objInt.GetInc();

//Get the current value.
Int64 i64Value = objInt.GetValue();

//Set the current value.
objInt.SetValue(nValue);
```

2.10.6.3. Float Data Node

To access the float data by **FloatFeature**, you can read and write the current value, check the Maximum, Minimum, whether to support increment, increment value and unit.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Obtain remote device attribute controller
IGXFeatureControl objIGXFeatureControl =
                                        objIGXDevice.GetRemoteFeatureControl();
//Get float data nodes
IFloatFeature objFloat = objIGXFeatureControl.GetFloatFeature("Gain");

//Get the maximum value.
Double dMax = objFloat.GetMax();

//Get the minimum value.
Double dMin = objFloat.GetMin();

Boolean bHasInc = objFloat.HasInc();
if (bHasInc)
{
    //Get the increment value.
    Double dInc = objFloat.GetInc();
}

//Get the unit.
String strUnit = objFloat.GetUnit();
//Get the current value.
Double dValue = objFloat.GetValue();
//Set the current value.
objFloat.SetValue(dValue);
```

2.10.6.4. Boolean Data Node

Boolean is relatively simple, only **True** and **False**.

```
//Obtain remote device attribute controller
IGXFeatureControl objIGXFeatureControl = objIGXDevice.GetRemoteFeatureControl();

//Get bool data nodes
IBoolFeature objBool = objIGXFeatureControl.GetBoolFeature("ChunkModeActive");

//Get the current value.
Boolean bValue = objBool.GetValue();

//Set the current value.
objBool.SetValue(bValue);
```

2.10.6.5. Enumeration Data Node

Enumerated can check the options, and read/write interfaces which the enumerated function supported.

```
//Obtain remote device attribute controller
IGXFeatureControl objIGXFeatureControl = objIGXDevice.GetRemoteFeatureControl();

//Get enumerate data nodes
IEnumFeature objEnum = objIGXFeatureControl.GetEnumFeature("AcquisitionMode");
List<String> lstEnumEntry = new(nothrow) List<String>();

//Get the enum items list.
lstEnumEntry = objEnum.GetEnumEntryList();
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
foreach (String strStr in lstEnumEntry)
{
    //Print all the enum items.
    Console.WriteLine(strStr);
}
//Get the current item.
String strValue = objEnum.GetValue();

//Set the current item.
objEnum.SetValue(strValue);
```

2.10.6.6. String Data Node

String provides the function to read/write the current value, check the maximum string length supported before write.

```
//Obtain remote device attribute controller
IGXFeatureControl objIGXFeatureControl = objIGXDevice.GetRemoteFeatureControl();

//Get string data nodes
IStringFeature objString = objIGXFeatureControl.GetStringFeature("DeviceUserID");

//Get the current value.
String strValue = objString.GetValue();

//Get the maximum length can be written to.
Int64 i64MaxLength = objString.GetStringMaxLength();

//Write the current value.
objString.SetValue(strValue);
```

2.10.6.7. Command Data Node

The command type is simplest, only one executes method.

```
//Obtain remote device attribute controller
IGXFeatureControl objIGXFeatureControl = objIGXDevice.GetRemoteFeatureControl();

//Get command data nodes
ICommandFeature objCommand = objIGXFeatureControl.GetCommandFeature(
                                                        "AcquisitionStart");

//Execute the command.
objCommand.Execute();
```

2.10.7. Precautions

After getting from the device object, the features controller **IGXFeatureControl** is valid until the device closed. Once the device is closed, the features controller is invalid; you cannot call any features of the features controller.

2.11. Acquisition Control and Image Processing

All the methods and control related to Acquisition are in the **IGXStream** object. How to get and open the stream object please see the following code:

```
UInt32 ui32StreamNum = objIGXDevice.GetStreamCount();
if (ui32StreamNum > 0)
{
    IGXStream objIGXStream = objIGXDevice.OpenStream(0);

    //Stream object control or acquisition.
    //Closing it when no longer to use the stream object.
    objIGXStream.Close();
}
```

There are two acquisition ways: callback acquisition and get a single image.

2.11.1. DQBuf

After enabling stream object acquisition and sending start acquisition commands to the device, the user can call DQBuf() interface to acquire single frames, please see the following code:

```
CGXStreamPointer objStreamPtr = objDevicePtr->OpenStream(0);

//To improve the collection performance of GigE cameras, please refer
//to the following code for the setting method (currently GigE series
//cameras support setting the optimal packet length)
GX_DEVICE_CLASS_LIST objDeviceClass =
                                    m_objDevicePtr->GetDeviceInfo().GetDeviceClass();
if (GX_DEVICE_CLASS_GEV == objDeviceClass)
{
   //Determine whether the device supports streaming channel packet
   //function
    if (true == m_objFeatureControlPtr->IsImplemented("GevSCPSPacketSize"))
    {
       //Obtain the optimal packet length value for the current network
       //environment
       int nPacketSize = m_objStreamPtr->GetOptimalPacketSize();

       // Set the optimal packet length value to the current device's
       //flow channel packet length value
      m_objFeatureControlPtr->GetIntFeature("GevSCPSPacketSize")->SetValue(
                                                                    nPacketSize);
   }
}
//Open flow channel acquisiton
objStreamPtr->StartGrab();

//Send acquisition commands to the device
CGXFeatureControlPointer objFeatureControlPtr =
                                      objDevicePtr->GetRemoteFeatureControl();
objFeatureControlPtr->GetCommandFeature("AcquisitionStart")->Execute();

//Zero copy single frame acquisition
CImageDataPointer objImageDataPtr;

//The timeout period is 500ms, and users can set it themselves
objImageDataPtr = objStreamPtr->DQBUF(500);
if (GX_FRAME_STATUS_SUCCESS == objImageDataPtr->GetStatus())
{
```

```
    //The image acquisition is successful and the frame is complete,
    //allowing for image processing...
}

// After using the image, QBuf needs to be called, otherwise it may
//result in the inability to loop to obtain the image normally
objStreamPtr->QBuf(objImageDataPtr);

//Stop acquisiton
objFeatureControlPtr->GetCommandFeature("AcquisitionStop")->Execute();
objStreamPtr->StopGrab();

//Close stream channel
objStreamPtr->Close();
```

Note: StartGrab() must be called first to enable the acquisition of the streaming channel, and then the acquisition command must be sent to the device, otherwise the acquisition command is invalid.

2.11.2. GetImage

After opening the stream object acquisition and sending start acquisition command to the device, you can call the **GetImage()** to get a single image, please see the following code:

```
IGXStream objIGXStream = objIGXDevice.OpenStream(0);

//Improve the acquisition performance of the network camera, the setting
//method refers to the following code (currently only the Gigabit network
//camera supports the optimal packet length).
GX_DEVICE_CLASS_LIST objDeviceClass =
                              m_objIGXDevice.GetDeviceInfo().GetDeviceClass();
if (GX_DEVICE_CLASS_LIST.GX_DEVICE_CLASS_GEV == objDeviceClass)
{
    //Determine whether the device supports the stream channel packet function.
    if (true == m_objIGXFeatureControl.IsImplemented("GevSCPSPacketSize"))
    {
        //Get the optimal packet length value of the current network environment.
        UInt32 ui32PacketSize = objIGXStream.GetOptimalPacketSize();

        //Set the optimal packet length value to the stream channel packet
        //length of the current device.
        m_objIGXFeatureControl.GetIntFeature("GevSCPSPacketSize").SetValue(
                                                                    nPacketSize);
    }
}
//Open stream channel acquisition.
objIGXStream.StartGrab();

//Send start acquisition command to the device.
IGXFeatureControl objIGXFeatureControl = objIGXDevice.GetRemoteFeatureControl();
objIGXFeatureControl.GetCommandFeature("AcquisitionStart").Execute();

//Get a single image.
IImageData objImageData = null;

//Set 'timeout'=500ms, users can set the value by themselves.
objImageData = objIGXStream.GetImage(500);
if (objImageData.GetStatus() == GX_FRAME_STATUS_SUCCESS)
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
{
    //If the acquisition is success and the image is a full frame, then the
    //user can process the image.
}
//Destroy the 'objImageData' object.
objImageData.Destroy();

//Stop acquisition.
objIGXFeatureControl.GetCommandFeature("AcquisitionStop").Execute();
objIGXStream.StopGrab();

//Close stream channel.
objIGXStream.Close();
```

Note: Users must call the **StartGrab()** to start the stream channel acquisition first, and then send start acquisition command to the device, otherwise the start acquisition command will invalid. Users use the function **GetImage()** to get the **IImageData()** object, if you do not use this function anymore, please destroy it by calling the **Destroy** method. **When using a high-resolution camera for high-speed data acquisition, recommend you to use callback acquisition mode, because the buffer copy inside the GetImage() will affect the transmission performance.**

2.11.3. Acquisition with Callback Functions

Users can register callback function to grab images, see the following code:

```
IGXStream objIGXStream = objIGXDevice.OpenStream(0);
//Improve the acquisition performance of the network camera, the setting
//method refers to the following code (currently only the Gigabit network
//camera supports the optimal packet length).
GX_DEVICE_CLASS_LIST objDeviceClass =
                              m_objIGXDevice.GetDeviceInfo().GetDeviceClass();
if (GX_DEVICE_CLASS_LIST.GX_DEVICE_CLASS_GEV == objDeviceClass)
{
    //Determine whether the device supports the stream channel packet function.
    if (true == m_objIGXFeatureControl.IsImplemented("GevSCPSPacketSize"))
    {
        //Get the optimal packet length value of the current network environment.
        UInt32 ui32PacketSize = m_objIGXStream.GetOptimalPacketSize();

        //Set the optimal packet length value to the stream channel packet
        //length of the current device.
        m_objIGXFeatureControl.GetIntFeature("GevSCPSPacketSize").SetValue(
                                                                    nPacketSize);
    }
}

//Register the acquisition callback function. Note: the first parameter is a user's
//private parameter, user can set any type of object, and also can set it null.
//The user's private parameter used within the callback function for restore,
//if not use the private parameter, you can set it null.
objIGXStream.RegisterCaptureCallback(objIGXDevice, OnFrameCallbackFun);

//Start stream channel acquisition.
objIGXStream.StartGrab();

//Send start acquisition command to the device.
IGXFeatureControl objIGXFeatureControl = objIGXDevice.GetRemoteFeatureControl();
```

```
objIGXFeatureControl.GetCommandFeature("AcquisitionStart").Execute();

//Callback acquisition process, please refer the callback function.
//Stop acquisition, unregister acquisition callback function.
objIGXFeatureControl.GetCommandFeature("AcquisitionStop").Execute();
objIGXStream.StopGrab();
objIGXStream.UnregisterCaptureCallback();

//Close stream channel.
objIGXStream.Close();
```

User's callback function, see the following code:

```
public void OnFrameCallbackFun(object objPara, IFrameData objIFrameData)
{
//User's private parameter obj, the device object was set when the user
//register the callback function, you can restore the parameter as a user
//private parameter within the callback function.
IGXDevice objIGXDevice = objPara as IGXDevice;
if (GX_FRAME_STATUS_SUCCESS == objIFrameData.GetStatus())
  {
    //If the image acquisition is a full frame, then you can read the image
    //width, height,data format and etc.
    UInt64 ui64Width = objIFrameData.GetWidth();
    UInt64 ui64Height = objIFrameData.GetHeight();
    GX_PIXEL_FORMAT_ENTRY emPixelFormat = objIFrameData.GetPixelFormat();
    //More images information please refer the IFrameData interfacedefine.
  }
}
```

Note: Users must call the function **StartGrab()** to start stream channel acquisition, and then send start acquisition command to the device; otherwise the start acquisition command will invalid.

2.11.4. Set the Number of Acquisition Buffer

After opening the stream, users can call SetAcqusitionBufferNumber to set the number of acquisition buffer. Industrial camera data acquisition is a complete production consumer model. When the number of buffers is insufficient, such as only one, the acquisition driver completes a single acquisition (buffer is in the consumer queue). If there is no buffer available within the acquisition driver, it may result in frame loss (depending on the output method of the camera). Therefore, it is recommended that the customer configure at least 3 buffers and complete the image usage as soon as possible. See the following code:

```
IGXStream objStream= objDevice.OpenStream(0);

//Improve the acquisition performance of the network camera, the setting
//method refers to the following code (currently only the Gigabit network
//camera supports the optimal packet length).
GX_DEVICE_CLASS_LIST objDeviceClass =
                          m_objIGXDevice.GetDeviceInfo().GetDeviceClass();
if (GX_DEVICE_CLASS_LIST.GX_DEVICE_CLASS_GEV == objDeviceClass)
{
    //Determine whether the device supports the stream channel packet function.
    if (true == m_objIGXFeatureControl.IsImplemented("GevSCPSPacketSize"))
    {
      //Get the optimal packet length value of the current network environment.
       UInt32 ui32PacketSize = m_objIGXStream.GetOptimalPacketSize();

       //Set the optimal packet length value to the stream channel packet
```

```
        //length of the current device.
        m_objIGXFeatureControl.GetIntFeature("GevSCPSPacketSize").SetValue(
                                                                nPacketSize);
    }
}
//Set the number of acquisition buffer.
objStream.SetAcqusitionBufferNumber(10);

//Start stream channel acquisition.
objStream.StartGrab();

//start acquisition command to the device.
IGXFeatureControl objIGXFeatureControl = objIGXDevice.GetRemoteFeatureControl();
objIGXFeatureControl.GetCommandFeature("AcquisitionStart").Execute();

// ……

//Stop acquisition.
objIGXFeatureControl.GetCommandFeature("AcquisitionStop").Execute();
objStream.StopGrab();

//Close stream channel.
objStream.Close();
```

Note:

1) This interface is an optional interface. When multiple cameras are acquisiting at the same time, if the frame rate of individual cameras is 0, you can call this interface to adjust the number of acquisition buffers of all cameras to ensure that all cameras have buffers for acquisition.
2) The buffer number must be set before calling StartGrab() to start the stream channel acquisition, otherwise the setting is invalid.

2.11.5. Image Processing

2.11.5.1. Image Format Conversion

Function description: Specified for an 8 bit valid data. Assume that the original data is non 8-bit, no matter mono or color, you can call the function **Covert()** to specify for an 8-bit data, and the return value type is **IntPtr**. The specified memory size is **Width** * **Height**.

Sample Code:

```
//ObjImageConvPtr can be either passed in by the acquisition callback function
//or obtained as CImageDataPointer by GetImage() Or obtained from DQBuf

//Create a conversion image format instantiation object
CGXImageFormatConvertPointer objImageConvPtr =
IGXFactory::GetInstance().CreateImageFormatConvert();

void* pRaw8Buffer = NULL;
uint64_t i64RawBufferLen = 0;

//Set conversion algorithm
objImageConvPtr->SetInterpolationType( GX_RAW2RGB_NEIGHBOUR);

//Assuming the original data is Mono10 images
//Set the effective number of digits
objImageConvPtr->SetValidBits( GX_BIT_2_9);
```

```
//Set target conversion format
objImageConvPtr->SetDstFormat( GX_PIXEL_FORMAT_MONO8);

//Assuming the original data is Mono12 images
//Set the effective number of digits
objImageConvPtr->SetValidBits( GX_BIT_4_11);
//Set target conversion format
objImageConvPtr->SetDstFormat( GX_PIXEL_FORMAT_MONO8);

//Assuming the original data is a BayerRG10 image
//Set the effective number of digits
objImageConvPtr->SetValidBits( GX_BIT_2_9);
//Set target conversion format
objImageConvPtr->SetDstFormat( GX_PIXEL_FORMAT_BAYER_RG8);

//Assuming the original data is a BayerRG12 image
//Set the effective number of digits
objImageConvPtr->SetValidBits( GX_BIT_4_11);
//Set target conversion format
objImageConvPtr->SetDstFormat( GX_PIXEL_FORMAT_BAYER_RG8);

//Obtain the length of the converted buffer
i64RawBufferLen = objImageConvPtr->GetBufferSizeForConversion(
                                                    objImageDataPtr);
//Request Buffer Length
pRaw8Buffer = new byte[i64RawBufferLen];
memset( pRaw8Buffer, 0, i64RawBufferLen);
//Convert pixel format
objImageConvPtr->Convert(objImageDataPtr, pRaw8Buffer,
                         i64RawBufferLen);
if( NULL != pRaw8Buffer)
{
   delete[] pRaw8Buffer;
   pRaw8Buffer = NULL;
}
```

Function description: Bayer format convert to RGB24 format, no matter the current image output is mono or color, 8-bit or non 8-bit, you can call the function **Convert()** to complete data interpolation processing, and the return **IntPtr** is RGB24 format. The memory size is **Width*Height*3**.

Sample Code:

```
//ObjImageConvPtr can be either passed in by the acquisition callback function
//or obtained as CImageDataPointer by GetImage() Or obtained from DQBuf

//Create a conversion image format instantiation object
CGXImageFormatConvertPointer objImageConvPtr =
IGXFactory::GetInstance().CreateImageFormatConvert();

void* pRGB24Buffer = NULL;
uint64_t i64RGB24BufferLen = 0;

//Set conversion algorithm
objImageConvPtr->SetInterpolationType( GX_RAW2RGB_NEIGHBOUR);
```

```
//Assuming the original data is Mono8 images
//Set the effective number of digits
objImageConvPtr->SetValidBits( GX_BIT_2_9);
//Set target conversion format
objImageConvPtr->SetDstFormat( GX_PIXEL_FORMAT_RGB8);

//Assuming the original data is Mono12 images
//Set the effective number of digits
objImageConvPtr->SetValidBits( GX_BIT_4_11);
//Set target conversion format
objImageConvPtr->SetDstFormat( GX_PIXEL_FORMAT_RGB8);

//Assuming the original data is BayerRG8 images
//Set the effective number of digits
objImageConvPtr->SetValidBits( GX_BIT_2_9);
//Set target conversion forma
objImageConvPtr->SetDstFormat( GX_PIXEL_FORMAT_RGB8);

//Assuming the original data is BayerRG12 images
//Set the effective number of digits
objImageConvPtr->SetValidBits( GX_BIT_4_11);
//Set target conversion format
objImageConvPtr->SetDstFormat( GX_PIXEL_FORMAT_RGB8);

//Obtain the length of the converted buffer
i64RGB24BufferLen = objImageConvPtr->GetBufferSizeForConversion(
                                                    objImageDataPtr);
//Request Buffer Length
pRGB24Buffer = new byte[i64RGB24BufferLe];
memset( pRGB24Buffer, 0, i64RGB24BufferLen);
//Convert pixel format
objImageConvPtr->Convert(objImageDataPtr, pRGB24Buffer,
                         i64RGB24BufferLen);
if( NULL != pRGB24Buffer)
{
   delete[] pRGB24Buffer;
   pRGB24Buffer = NULL;
}
```

2.11.5.2. Image Effect Enhanced

The interface library also provides the image effect enhanced in software side, users can do some image effect enhanced operate selectively, such as: defective-pixel correction, sharpening, contrast, brightness and etc. The image effect enhanced involves the functions **IImageProcessConfig**, **IFrameData,** **IImageData** and **IGXImageProcess**.

Sample Code:

```
// ObjIBaseData can be IFrameData passed in by the callback function, ImageData
//obtained by GetImage, or IframeData obtained by DQBuf

//Obtain image processing objects
IGXImageProcess objImageProcess =
                        IGXFactory::GetInstance().CreateImageProcess();
```

```
//Constructing image processing configuration objects through device
//objects
IImageProcessConfig objCfg = objIGXDevice.CreateImageProcessConfig();

//Apply for Buffer
IntPtr pRGB24Processed = IntPtr.Zero;
pRGB24Processed = Marshal.AllocCoTaskMem(objIBaseData.GetWidth() *
                                         objIBaseData.GetHeight() * 3);

//The return result is the RGB24 format data that has been enhanced
//with image effects
m_objImageProcess.ImageImprovment(objIBaseData, pRGB24Processe,
                                  objCfg);
//When the user is not using objcfg, it needs to be released
objCfg.Destroy();
if( IntPtr.Zero != pRGB24Processed)
{
   Marshal.FreeHGlobal( pRGB24Processed);
   pRGB24Processed = IntPtr.Zero;
}
```

For different data type cameras, the support of effect enhanced from interface library is also different. Details as follows:

function	Corresponding interface	RawData			
		Mono8	Mono non8	Bayer8	Bayer non8
Set valid bit	SetValidBit	Yes	Yes	Yes	Yes
Defective Pixel correction switch	EnableDefectivePixelCorrect	Yes	Yes	Yes	Yes
Sharpen switch	EnableSharpen	Yes	Yes	Yes	Yes
Sharpen factor	SetSharpenParam	Yes	Yes	Yes	Yes
Contrast	SetContrastParam	Yes	Yes	Yes	Yes
Gamma	SetGammaParam	Yes	Yes	Yes	Yes
Brightness	SetLightnessParam	Yes	Yes	Yes	Yes
Noise reduction switch	EnableDenoise	No	No	Yes	Yes
Saturation	SetSaturationParam	No	No	Yes	Yes
RGB24 interpolation type	SetConvertType	No	No	Yes	Yes
RGB24 interpolation flip switch	EnableConvertFlip	No	No	Yes	Yes
Color correction switch	EnableColorCorrection	No	No	Yes	Yes

Table 2-1 Image effect enhanced comparison table

The parameter range of Image effect enhance and the detail explanation as follows:

Functions	Default value	Range	Remarks
Set valid bit	GX_BIT_0_7	Refer GX_VALID_BIT_LIST	● For 8bit data , only one to select : GX_BIT_0_7

			● For 10bit data, you can select GX_BIT_0_7, GX_BIT_1_8, GX_BIT_2_9, recommend GX_BIT_2_9 ● For 12bit data, you can select GX_BIT_0_7, GX_BIT_1_8, GX_BIT_2_9, GX_BIT_3_10, GX_BIT_4_11, recommend GX_BIT_4_11
Defective Pixel correction switch	false	true, false	true: switch on defective Pixel correction false: switch off defective Pixel correction
Sharpen switch	false	true, false	true: switch on sharpen false: switch off sharpen
Sharpen factor	0.1	[0.1, 5]	Increase sharpening effect gradually
Contrast	0	[-50, 100]	0: the contrast ratio does not change > 0: increasing contrast ratio <0: decrease contrast ratio
Gamma	0	[0.1, 10]	-
Brightness	0	[-150, 150]	0 : the brightness does not change > 0: increasing brightness < 0: decrease brightness
Noise reduction switch	false	true, false	true: switch on noise reduction false: switch off noise reduction
Saturation	64	[0, 128]	64: saturation does not change >64: increasing saturation < 64: decrease saturation 128: double the current saturation 0: mono image
RGB24 interpolation type	GX_RAW2RGB_NEIGHBOUR	Refer GX_BAYER_CONVERT_TYPE_LIST	-
RGB24 interpolation flip switch	false	true, false	true: flip false: non-flip

Table 2-2 Image effect enhanced parameters range and description

The advanced user can fine-tune the parameter after getting **IImageProcessConfig** object, see as follows:

```
// ObjIBaseData can be IFrameData passed in by the callback function, ImageData
//obtained by GetImage, or IframeData obtained by DQBuf
//Obtain image processing objects
IImageProcessConfig objCfg = objIGXDevice.CreateImageProcessConfig();
//Initialization the default configuration parameter when constructing the
//'objcfg' object, the user can choose to fine-tune the configuration parameter.
//Select significance bit 0-7.
objCfg.SetValidBit(GX_BIT_0_7);
//Enable defective Pixel correction.
objCfg.EnableDefectivePixelCorrect(true);
```

```
//Enable sharpen.
objCfg.EnableSharpen(true);
//Set sharpen factor is 1.
objCfg.SetSharpenParam(1);
//Set contrast control parameter.
objCfg.SetContrastParam(0);
//Set Gamma factor.
objCfg.SetGammaParam(1);
//Set brightness control parameter.
objCfg.SetLightnessParam(0);
//Enable noise reduction switch, not support mono camera.
objCfg.EnableDenoise(true);
//Set saturation control factor, not support mono camera.
objCfg.SetSaturationParam(0);
//Set interpolation algorithm, not support mono camera.
objCfg.SetConvertType(GX_RAW2RGB_NEIGHBOUR);
//Enable interpolation flip, not support mono camera.
objCfg.EnableConvertFlip(true);
//User can also choose to restore the optimum default parameter configuration.
objCfg.Reset();
//Apply for Buffer
IntPtr pRGB24Processed = IntPtr.Zero;
pRGB24Processed = Marshal.AllocCoTaskMem(objIBaseData.GetWidth() *
    objIBaseData.GetHeight() * 3);
//The return result is the RGB24 format data that has been enhanced
//with image effects
m_objImageProcess.ImageImprovment(objIBaseData, pRGB24Processe,
    objCfg);
//When the user is not using objcfg, it needs to be released
objCfg.Destroy();
if( IntPtr.Zero != pRGB24Processed)
{
   Marshal.FreeHGlobal( pRGB24Processed);
   pRGB24Processed = IntPtr.Zero;
}
```

2.11.5.3. The Result of Image Effect Enhanced

According on the cameras' type and usage scenario, the image effect enhanced configuration parameters are also different, but there are still a few recommend functions can meet the needs of most users. Take MER-200-20GC GigE Vision camera for example, the image which the camera output after image format conversion to RGB24 bit, the result as follows without any effect enhance processing:



Figure 2-10 Neighborhood interpolation RGB image (color correction: off; saturation: 64; Gamma: 1; contrast: 0)

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

On the basis of Figure 2-10, switch on the color correction function, see the result as follows:



Figure 2-11 Switch on color correction based on figure2-10 (color correction: on; saturation: 64; Gamma: 1; Contrast: 0)

On the basis of Figure 2-11, increase the saturation to 80, see the result as follows:



Figure 2-12 Set saturation to 80(color correction: on; saturation: 80; Gamma: 1; Contrast: 0)

On the basis of Figure 2-12, adjust the Gamma to 1.98, make the image more close to what the users see in the actual scene, the value 1.98 is just for an example, for different cameras the Gamma value will differ. For the USB2.0 and USB3.0 cameras, users can read the gamma by the read/write interface. But for the GigE Vision camera, the user should adjust the gamma by himself, there is not an interface to be called directly.



Figure 2-13 Set the Gamma is 1.98(color correction: on; saturation: 80; Gamma: 1.98; Contrast: 0)

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

On the basis of Figure 2-13, set the contrast to 40, see the result as follows:



Figure 2-14 Set the contrast is 40(color correction: on; saturation: 80; Gamma: 1.98; Contrast: 40)

2.11.6. Stream Object Features Control

We have mentioned that the stream layer has its own features control in the chapter **FeaturesControl**, users can get it by calling the interface **IGXStream::GetFeatureControl**. The control features of the stream layer include the acquisition control and statistical information. Take the Gigabit Ethernet stream layer statistical for example, such as to check the current acquisition and statistical information in the process of acquisition.

```
//'objGXStream' is an 'IGXStream' object which get from the interface
//'IGXDevice::OpenStream'.
IGXFeatureControl objStreamFeatureControl = objGXStream.GetFeatureControl();

//Check the acquisition and statistical information.
//The lost frame count which caused by the buffer lack.
objStreamFeatureControl.GetIntFeature("StreamLostFrameCount").GetValue();

//The count of incomplete frame.
objStreamFeatureControl.GetIntFeature("StreamIncompleteFrameCount").GetValue();

//The delivered packet count.
objStreamFeatureControl.GetIntFeature("StreamDeliveredPacketCount").GetValue();

//The resend packet count.
objStreamFeatureControl.GetIntFeature("StreamResendPacketCount").GetValue();

//Set acquisition configuration parameter.
//Set the block timeout is 200ms.
objStreamFeatureControl.GetIntFeature("BlockTimeout").SetValue(200);
```

The "StreamBufferHandlingMode" in the stream object features can set the processing mode of the Buffer. Three Buffer processing modes are available:

1) OldestFirst: The default value. The image buffer follows the first-in-first-out principle. After all the buffers are filled, the new image data will be discarded until the user completes the processing of the buffer that has filled the image data. The typical application is to receive each frame of images acquired by the camera without losing frames. In order to achieve no frame loss, the speed of image data transmission and processing need to be as fast as possible (at least less than the frame period).

2) OldestFirstOverwrite: Follow the first-in-first-out principle. The difference from the OldestFirst mode is that when all the buffers are filled, the SDK will automatically discard one frame of image buffer with the oldest timestamp to receive new image data. The typical application is that it does not require receiving each frame of images acquired by the camera, and the image data transmission and processing speed is slow.

3) NewestOnly: In this mode, the user always receives the latest image received by the SDK. Each time the SDK receives a new frame of image data, it will automatically discard the image with the old timestamp. Therefore, when the user's image processing is not timely or the speed is slow, frame loss will occur. In the main applications, the real-time requirements of image acquisition and display are high, and it is not required to receive each frame of images acquired by the camera. However, depending on the camera's frame rate, memory cache, transmission speed, and user applications, there may be a delay between the latest image received by the SDK and the latest image exposed by the camera.

Note: The above code snippet just shown the stream layer control function of the GigE Vision cameras, for the other cameras, please use the **GalaxyView** demo to open the device, or call the function **IGXFeatureControl.GetFeatureNameList** to get the function list.

2.11.7. Frame Information

2.11.7.1. Principle

When the frame information is activated, the camera will append a chunk immediately after the returned capturing image cache, which contains the ID, length, and frame information content that the user wishes to obtain.

The captured image is placed at the beginning as the first data chunk, and the additional chunks are added sequentially as chunks 2 and 3.

The sample of a collection of data chunks containing preprocessed image data chunks and additional frame information chunks is shown as the following schematic diagram.



Figure 2-15 Image Frame Information Structure Schematic Diagram

When manually parsing, parse backwards from the end to the beginning. First read a 32-bit length, then read a 32-bit ID, and immediately after that read the frame information content which length corresponding to the length read in the first step.

Note: When the frame information is activated, the size returned by GXGetPayLoadSize() will be greater than the value of width multiplied by height multiplied by pixel depth.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.11.7.2. Enabling and Obtaining the Supported Types of Frame Information

- For specific available frame information, please refer to the chunk data control section in the property tree of GalaxyView, as shown in the following:

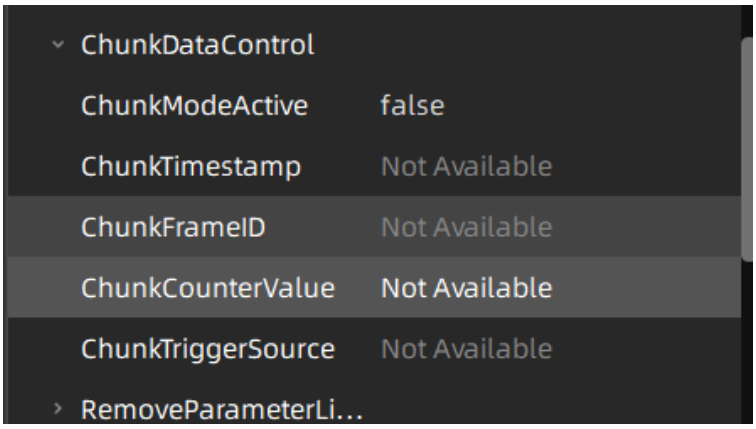


Figure 2-16 Network Frame Information Nodes

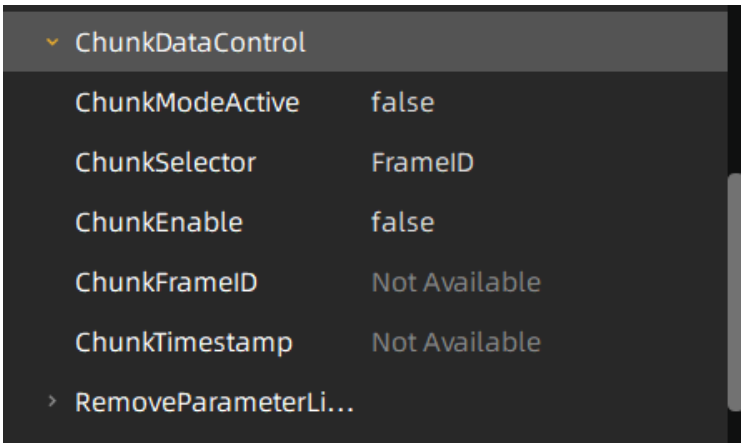


Figure 2-17 U3 Frame Information Nodes

- Enable frame information

1. Set the ChunkModeActive to true

Code Sample:

```
// Get the remote device feature controller.
IGXFeatureControl objIGXFeatureControl =
          objIGXDevice.GetRemoteFeatureControl();


// Frame information has been activated before capturing.
if(objIGXFeatureControl.IsImplemented("ChunkModeActive")&&
      objIGXFeatureControl.IsWritable("ChunkModeActive"))
{
    objIGXFeatureControl.GetBoolFeature("ChunkModeActive").SetValue(true);
}
```

2. If the chunkselector node is supported by the camera, set it to the following optional feature items, i.e. the feature you want to enable:

    1) FrameID

    2) Timestamp

3) CounterValue

3. Enable the selected chunk type by setting the ChunkEnable to true.

4. Repeat steps 2 and 3 for each frame information that needs to be enabled.

Note: Steps 2 and 3 above can only be set if supported by the camera.

Code Sample:

```
// Get the remote device feature controller.
IGXFeatureControl objIGXFeatureControl =
          objIGXDevice.GetRemoteFeatureControl();


//Turn on the frame ID.
if(objIGXFeatureControl.IsImplemented("ChunkSelector")&&
     objIGXFeatureControl.IsWritable("ChunkSelector"))
{
    objIGXFeatureControl.GetEnumFeature("ChunkSelector").SetValue(
                         "FrameID");
}

if(objIGXFeatureControl.IsImplemented("ChunkEnable")&&
     objIGXFeatureControl.IsWritable("ChunkEnable"))
{
    objIGXFeatureControl.GetBoolFeature("ChunkEnable").SetValue(true);
}
```

2.11.7.3. Get Frame Information through the Frame Information Feature Control

1) Get the frame information feature control object through the IFrameData object obtained by the acquisition with callback. It is used to obtain the frame information content.

Code Sample:

```
//Private acquisition with callback processing interface.
private void __OnFrameCallbackFun(object objUserParam, IFrameData
objIFrameData)
{
    try
    {
        // Get frame information feature controller.

        IGXFeatureControl objChunkDataFC =
              objIFrameData.GetChunkDataFeatureControl();


        // Determines whether Frame ID is supported and readable.
        bool objValue = objChunkDataFC.IsImplemented("ChunkFrameID") &&
objChunkDataFC.IsReadable("ChunkFrameID");
        if (objValue )
        {
            //Get the frame ID of current image.
            Int64 nChunkFrameID =
objChunkDataFC.GetIntFeature("ChunkFrameID").GetValue();
        }
    }
    catch (CGalaxyException ex)
    {
        //If the frame information dose not exist, it will raise an exception
    }
```

```
    catch (Exception)
    {
        //Other exceptions.
    }
}
```

2) Get the frame information feature control object through the IFrameData object obtained by the DQBuf(). It is used to obtain the frame information content.

Code Sample:

```
//Open stream.
IGXStream objStream = objDevice.OpenStream(0);
//Set the camera parameters.
//…
// Zero-copy single frame capture.
IFrameData objIFrameData = objStream.DQBuf(1000);
try
{
    //Get the frame information controller.
    IGXFeatureControl objChunkDataFC =
              objIFrameData.GetChunkDataFeatureControl();

    //Determines whether Frame ID is supported and readable.
    bool objValue = objChunkDataFC.IsImplemented("ChunkFrameID")
                && objChunkDataFC.IsReadable("ChunkFrameID");
     if (objValue )
     {
         //Get the frame ID of current image.
         Int64 nChunkFrameID = objChunkDataFC.GetIntFeature(
                         "ChunkFrameID").GetValue();
     }
}
catch (CGalaxyException ex)
{
    // If the frame information dose not exist, it will raise an exception
}
catch (Exception)
{
    // Other exceptions.
}


//Call the QBuf after using the image, otherwise it may result in the
//inability to obtain images normally in a loop.
objStream.QBuf(objIFrameData);
```

3) Obtain the frame information feature control object through the IImageData object obtained by the GetImage(). It is used to obtain the frame information content.

Code Sample:

```
//Open stream.
IGXStream objStream = objDevice.OpenStream(0);
//Set the camera parameters.
//…
//Single frame capture.
IImageData objImageData = objIGXStream.GetImage(500);
try
{
```

```
    //Get the frame information controller.
    IGXFeatureControl objChunkDataFC =
              objIFrameData.GetChunkDataFeatureControl();

     //Determines whether Frame ID is supported and readable.
         bool objValue = objChunkDataFC.IsImplemented("ChunkFrameID")
                  && objChunkDataFC.IsReadable("ChunkFrameID");
      if (objValue )
      {
          //Get the frame ID of current image.
          Int64 nChunkFrameID = objChunkDataFC.GetIntFeature(
                          "ChunkFrameID").GetValue();
      }
}
catch (CGalaxyException ex)
{
    //If the frame information dose not exist, it will raise an exception
}
catch (Exception)
{
    //Other exceptions.
}

//ImageData needs to release object manually.
objImageData.Destroy();
```

2.11.8. Precautions

After getting from the device object, the features controller **IGXStream** is valid, until the device is closed. Once the device is closed, the stream object will invalid, you cannot call any interface of the stream object.

2.12. Get Devices Events

DAHENG IMAGING GigE Vision cameras can send event messages. For example: when a sensor exposure has finished, the camera can send an end-of-exposure event to the PC. The event can be received by the PC before the image data for the finished exposure has been completely transferred. In this section will illustrate how to obtain events and event data.

2.12.1. Selection Events

Feature name string	Function type	Description	Options (subject to the current device)
EventSelector	enumeration	Event source select	Maybe include the following options: ExposureEnd [Exposure end.] BlockDiscard [Block discard.] EventOverrun [Event queue overflow.] FrameStartOvertrigger [Trigger signal overflow.] BlockNotEmpty [Image buffer is not empty.] InternalError [Internal error]

2.12.2. Enabled Events

Feature name string	Function type	Description	Options (subject to the current device)
EventNotification	enumeration	Event enable	include the following options: Off [off] On [on]

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.12.3. Registered Events Message Callback Function

The function which the register event used is **IGXFeatureControl.RegisterFeatureCallback()**, the first parameter is the function code of the event which to be registered, the event function code to be selected is as follows:

Feature name string	Function type	Description
EventExposureEnd	Integer	ID exposure end event ID
EventBlockDiscard	Integer	data block discard event ID
EventOverrun	Integer	event queue overflow ID
EventFrameStartOvertrigger	Integer	trigger signal is shield event ID
EventBlockNotEmpty	Integer	the frame buffer is not empty event ID
EventInternalError	Integer	internal error event ID

2.12.4. Get Events Data Information

User can call the function **CGXFeatureControlPointer** to read/write the current channel event information within the event callback function, the section 2.11.4 declared 5 event types, the event information of the 5 types of events can be getting by the following control codes:

string for Register event	related event data string	Function type	Description
EventExposureEnd	EventExposureEndTimestamp	integer	exposure end event timestamp
	EventExposureEndFrameID	integer	exposure end event ID
EventBlockDiscard	EventBlockDiscardTimestamp	integer	data block discard event timestamp
EventOverrun	EventOverrunTimestamp	integer	event queue overflow timestamp
EventFrameStartOvertrigger	EventFrameStartOvertriggerTimestamp	integer	trigger signal is shield event timestamp
EventBlockNotEmpty	EventBlockNotEmptyTimestamp	integer	the frame buffer is not empty event timestamp
EventInternalError	EventInternalErrorTimestamp	integer	internal error event time stamp

2.12.5. Sample Code

Take the getting exposure end event for example, the code snippet as follows:

```
//The 'objIGXDevice' is the 'IGXDevice' device object, the device is open.
//If the device event's features is on the far-end device feature controller,
//get the far-end device feature controller first.
IGXFeatureControl objRemoteFeatureControl = objIGXDevice.GetRemoteFeatureControl();

//Select the event source.
objRemoteFeatureControl.GetEnumFeature("EventSelector").SetValue("ExposureEnd");

//Enable the event.
objRemoteFeatureControl.GetEnumFeature("EventNotification").SetValue("On");
```

```
//Register event callback function, note that the second parameter is a user
//private parameter, the user can set any 'object' type object.
//The private parameter can be restored for the user's use within the callback
//function, if the user does not use the private parameter, the parameter can
//be set to null.
//Here we set the 'objRemoteFeatureControl' object.
GX_FEATURE_CALLBACK_HANDLE hEventHandle = null;
hEventHandle = objRemoteFeatureControl.RegisterFeatureCallback(
                                                    "EventExposureEnd",
                                                    objRemoteFeatureControl,
                                                    OnFeatureCallback);

//Start stream channel acquisition.
objIGXStream.StartGrab();

//Send start acquisition command to the device.
IGXFeatureControl objIGXFeatureControl = objIGXDevice.GetRemoteFeatureControl();
objIGXFeatureControl.GetCommandFeature("AcquisitionStart").Execute();

//Send start acquisition command and the camera start exposuring and
//outputting images, when the exposure is end, an exposure end event will
//generate, then will activate the callback 'OnFeatureCallback' interface.
//Receive the exposure end event, please refer to the
//'OnFeatureCallback'function.
//Send stop acquisition command to the device.
objIGXFeatureControl.GetCommandFeature("AcquisitionStop").Execute();
objIGXStream.StopGrab();

//Unregister event.
objRemoteFeatureControl.UnregisterFeatureCallback(hEventHandle);
```

Event delegate function as follows:

```
void OnFeatureCallback(object objUserParam, string strFeatureName)
{
    //Use 'IGXFeatureControl' object as a user private parameter when registering
    //the events.
    //Resotre it within the delegate callback function, to get the data of
    //exposure end event.
    IGXFeatureControl objFeatureControl = objUserParam as IGXFeatureControl;

    //Get exposure end event timestamp.
    objFeatureControl.GetIntFeature("EventExposureEndTimestamp").GetValue();

    //Get exposure end event ID.
    objFeatureControl.GetIntFeature("EventExposureEndFrameID").GetValue();
}
```

Note: Users must send the start acquisition command first, then the camera can output exposure end event and send start acquisition command. So the user must call the **IGXStream::StartGrab()** first, and then get the **AcquisitionStart** command node by the features controller and to execute the command **Execute**.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.13. Reconnection

2.13.1. Procedure

GigE Vision camera and USB3 Vision camera provides local device event, including: device offline event, device disconnection event and device reconnection event. The procedure for obtaining events is as follows:

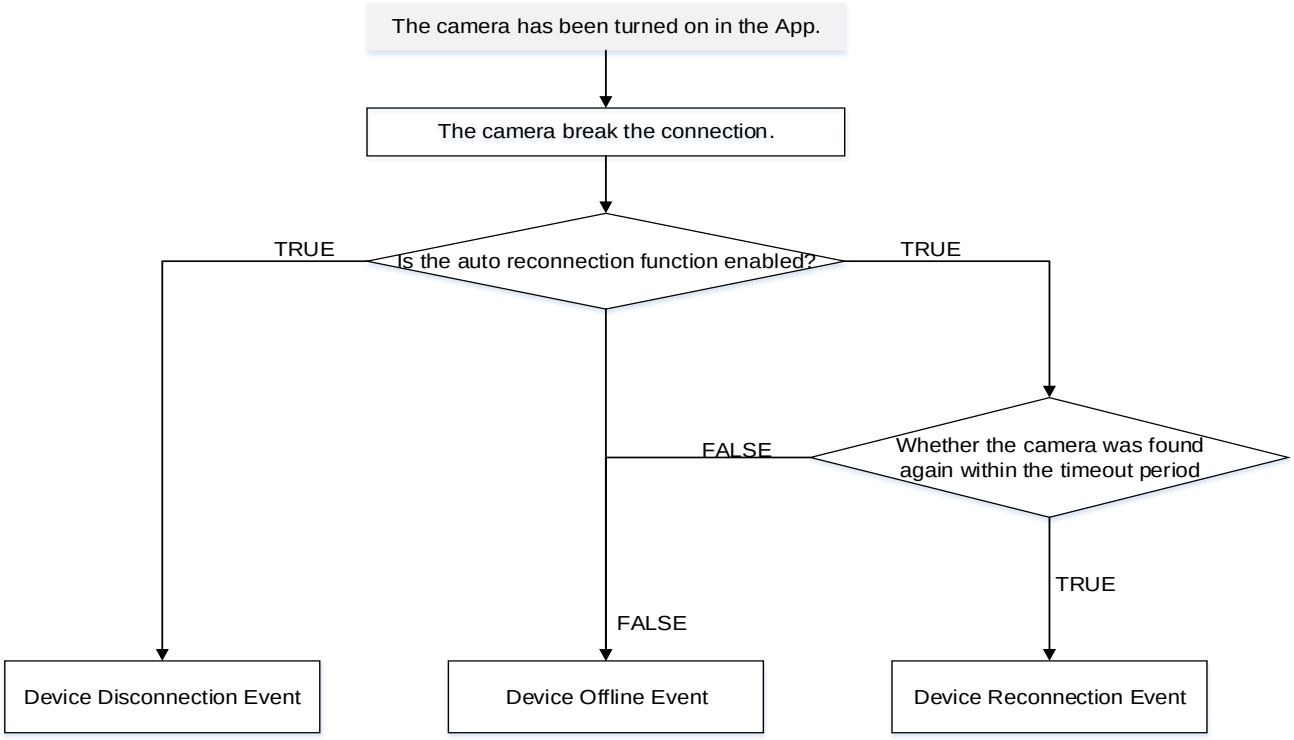


Figure 2-18 Offline and Reconnection Procedure

If a device offline event is registered, the device offline event will be received directly when the automatic reconnection function is not enabled.

If a device offline event or a device disconnection event is registered, an event in the following situations will be received when the automatic reconnection function is enabled:

1. As soon as the device breaks the connection, the API will call the device offline event callback function.
2. If the camera can be found within the timeout period, the API will call the device reconnection event callback function.
3. If the camera can not be found within the timeout period, the API will call the device disconnection event callback function.

Note: If IP address of the camera is changed, the auto reconnection function will not work in this case.

2.13.2. Device Offline Event

DAHENG IMAGING GigE Vision camera and USB3.0 camera provides device offline event mechanism; users can get the device offline event by callback function, and register the event after open the device.

Steps of get the offline events:

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

1) Register the offline events.

2) Motivate the callback function after the device offline.

3) Unregister the offline events.

Register offline events code snippet as follows:

```
GX_DEVICE_OFFLINE_CALLBACK_HANDLE hDeviceOffline = null;
//ObjIGXDevice is the IGXDevice device object, the device is open.
//The user can set a string to the first parameter, such as device name, the
//parameter is user private, user can restore and print it within the callback function.
//If not use it, set it null.
hDeviceOffline = objIGXDevice.RegisterDeviceOfflineCallback(null,
                                                    OnDeviceOfflineEvent);
```

Unregister events before close the device:

```
objIGXDevice.UnregisterDeviceOfflineCallback(hDeviceOffline);
```

The callback function as follows:

```
public void OnDeviceOfflineEvent(object objUserParam)
{
    //Obj is a user private parameter, users can set it when registering the
    //callback function and restore it here.
    //Users are not allowed to execute close device inner the offline callback
    //function, otherwise an illegal call exception will throw out.
}
```

Note: Not allowed to execute close the device operation within the offline callback function.

2.13.3. Device Disconnection Event and Device Reconnection Event

2.13.3.1. Supported Range

	Auto Reconnection		Restore Image Capture	Restore Parameter	
	ON	OFF		Thought UserSet	Auto
GigE Vision Transport Layer Socket	✓	✓	✓	✓	×
USB3 Vision Transport Layer	✓	✓	✓	✓	×
GxIAPI C	✓	✓	✓	✓	×
GxIAPI C++	✓	✓	✓	✓	×
GxIAPI C#	✓	✓	✓	✓	×
GxIAPI Python	✓	✓	✓	✓	×

If the user wish to enable the automatic reconnection function, so that the API automatically helps the user to restore the capture status after the camera has successfully reconnected, the user must manually save the current parameters as UserSet0 before starting the capture and set the parameter group to UserSet0. Otherwise, if the width, height, pixel format, and other properties of the camera's image differ before and after reconnection, the capture may encounter exceptions, such as image distortion or program crashes.

Take a network camera as an example. When the device is offline but not powered off, the parameters will not change. In this scenario, users can choose not to set UserSet0 as the startup parameter group.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.13.3.2. Property Node

Users can control the auto reconnection function and set the reconnection timeout period by using the relevant property nodes in the local device layer.

Model	Property Node	Explanation
LocalDevice/DeviceAutoConnection	EnableAutoConnection	Enable auto reconnection.
	DeviceConnectStatus	Device connection status.
	AutoConnectCounts	Auto reconnection count.
	AutoConnectTimeout	Auto reconnection timeout period, units ms.

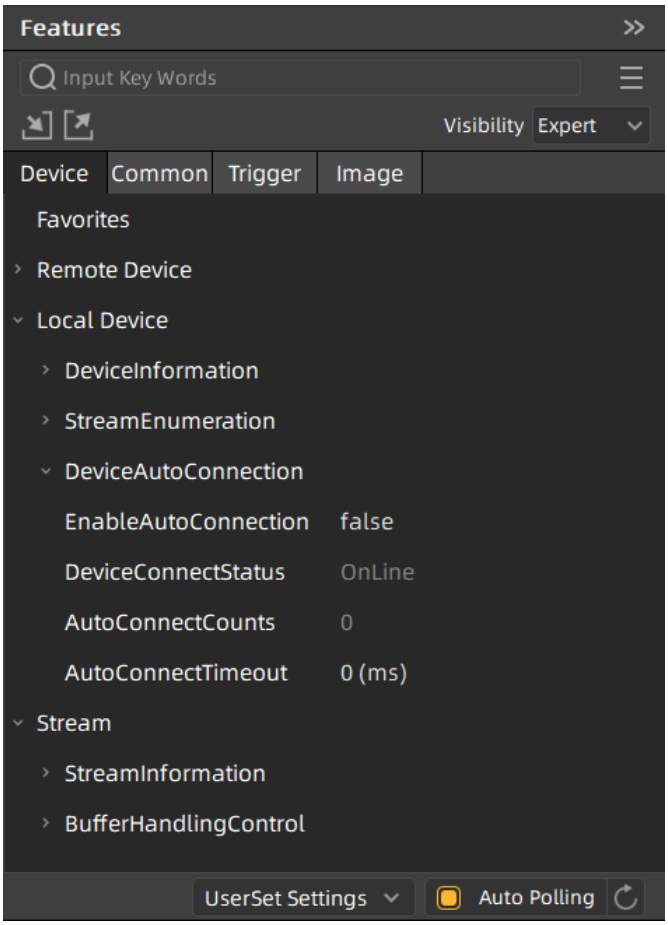


Figure 2-19 Auto Reconnection Attribute Nodes

2.13.3.3. Code Sample

GigE Vision camera and USB3 Vision camera provides device reconnection event and device disconnection event mechanism; users can get the device reconnection event and device disconnection event by calling function, and register the event after open the device.

Obtaining reconnection events involves the following steps:

1) Enable the reconnection function.

2) Register reconnection events.

3) Activate the callback function after the device reconnects.

4) Unregister reconnection events.

Obtaining disconnection events involves the following steps:

1) Enable the reconnection function.

2) Register disconnection events.

3) Activate the callback function after the device disconnects.

4) Unregister disconnection events.

Note: Be sure to unregister events before closing the device.

Closing the device is not allowed within the reconnection callback function, otherwise an illegal calling exception will be raised.

Closing the device is not allowed within the disconnection callback function, otherwise an illegal calling exception will be raised.

```
using GxIAPINET;

namespace GxAutoConnect
{
    class Sample
    {
        // Callback Function
        private static void __OnDeviceReconnectCallbackFun(object pUserParam)
        {
           // Upon receiving a reconnection notification, users perform
            // the necessary operations…

        }

        //Device disconnection callback function.
        private static void __OnDeviceDisconnectCallbackFun(object pUserParam)
        {
           // Upon receiving a disconnection notification, users perform the
           // necessary operations…
        }

        static void Main(string[] args)
        {
            try
            {
                // Initialise library.
                IGXFactory.GetInstance().Init();

                // Enumerate cameras.
                List<IGXDeviceInfo> lstDevInfo = new List<IGXDeviceInfo>();
                IGXFactory.GetInstance().UpdateAllDeviceList(300, lstDevInfo);
                if (lstDevInfo.Count < 1)
                {
                    IGXFactory.GetInstance().Uninit();
                    return;
                }

                // Turn on  the camera.
                IGXDevice objDevice = IGXFactory.GetInstance().
                    OpenDeviceBySN(lstDevInfo[0].GetSN(),
                    GX_ACCESS_MODE.GX_ACCESS_CONTROL);

                // Auto reconnection is activated.
                IBoolFeature pAutoConnect =
```

```
                    objDevice.GetFeatureControl()
                    .GetBoolFeature("EnableAutoConnection");
                if (pAutoConnect == null)
                {
                    objDevice.Close();
                    IGXFactory.GetInstance().Uninit();
                    return;
                }
                pAutoConnect.SetValue(true);

                            DeviceReconnectDelegate reconnectDelegate = new
DeviceReconnectDelegate(__OnDeviceReconnectCallbackFun);
                           DeviceDisconnectDelegate disconnectDelegate = new
DeviceDisconnectDelegate(__OnDeviceDisconnectCallbackFun);

                // Register device reconnection callback.
                             objDevice.RegisterDeviceReconnectCallback(null,
reconnectDelegate);
                // Register device disconnection callback.
                            objDevice.RegisterDeviceDisconnectCallback(null,
disconnectDelegate);

                // ......

                // Unregister device reconnection callback.
                objDevice.UnregisterDeviceReconnectCallback();
                // Unregister device disconnection callback.
                objDevice.UnregisterDeviceDisconnectCallback();

                objDevice.Close();
                IGXFactory.GetInstance().Uninit();
            }
            catch (CGalaxyException ex)
            {
                Console.WriteLine("GalaxyException: " + ex.Message);
            }
            catch (Exception ex)
            {
                Console.WriteLine("Exception: " + ex.Message);
            }
        }
    }
}
```

2.13.3.4. Notes

1) If the camera successfully reconnects, residual frames may be generated during the reconnection process.

2) When using the auto-reconnection function, it is strongly recommended to use the default parameter group and avoid any parameter configuration. If a non-default parameter group is used, modifying ROI settings may cause the camera to fail to resume image acquisition after reconnection. If the auto-reconnection enable status is modified during disconnection, the camera may still attempt to reconnect.

3) Upon successful reconnection, the camera's transmission data will be reset to the initial acquisition state, including the delivered frame count and residual frames being cleared.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.14. Import and Export Cameras' Configuration Parameters

Open device by running the **GalaxyView** Demo, expanding the menu **File**, you can see the menu **Import Device Setting** and **Export Device Setting**, see as follows:

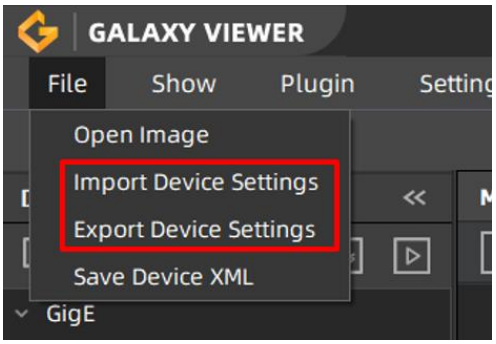


Figure 2-20

Select **Export Device Setting** menu, a new text file will create in the local disk to save the current parameter of the device.

Select **Import Device Setting** menu, select the local file and import it.

There is also having corresponding user interface to be called in C# interface library.

Import/export configuration files, sample code as follows:

```
//Open device, here we open the device by the serial number, the serial number
//is subject to the actual device.Users can open the device by other ways.
IGXDevice objIGXDevice = objIGXFactory.OpenDeviceBySN("RN0001007012",
                                                      GX_ACCESS_EXCLUSIVE);
//Open the first stream object of the device.
IGXStream objIGXStream = objIGXDevice.OpenStream(0);

//Export configuration files, users can free to take the file name, but the
//export catalog must be existed first.
objIGXDevice.ExportConfigFile("D:\\Config.txt");

//Import configuration files.
objIGXDevice.ImportConfigFile("D:\\Config.txt");
```

Note: Before executing the import/export configuration file, you should open the device first, and open the first stream object of the device.

2.15. Properties Setting UI

GxIAPI provides a properties setting window UI based on WxWidgets, supporting four programming languages: C, C++, C#, and Python. It is a pop-up window and is non-blocking by default.

2.15.1. Feature Description

Feature controller	Feature explanation
Search	After entering keywords of the property node in the search bar, all nodes containing those keywords within the camera will be filtered. Clicking the corresponding item will navigate to the settings location for that node.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

ExpandAll / CollapseAll	Click ExpandAll to display all nodes in the properties tree. And click CollapseAll to collapse the nodes in the properties tree.
Auto Polling	Tick Auto Polling to activate the Polling function. When properties within the camera changed, the corresponding UI elements in the properties tree will update accordingly. Automatic Auto Polling are disabled by default.
Visibility	Visibility levels are categorised as Beginner, Expert, and Guru. Selecting the corresponding visibility level will filter the relevant property nodes within the properties tree.
Properties tree	The properties tree displays camera device information, stream information, local configuration information, and frame grabber information in sequence. Corresponding sub-items may be selected for configuration.

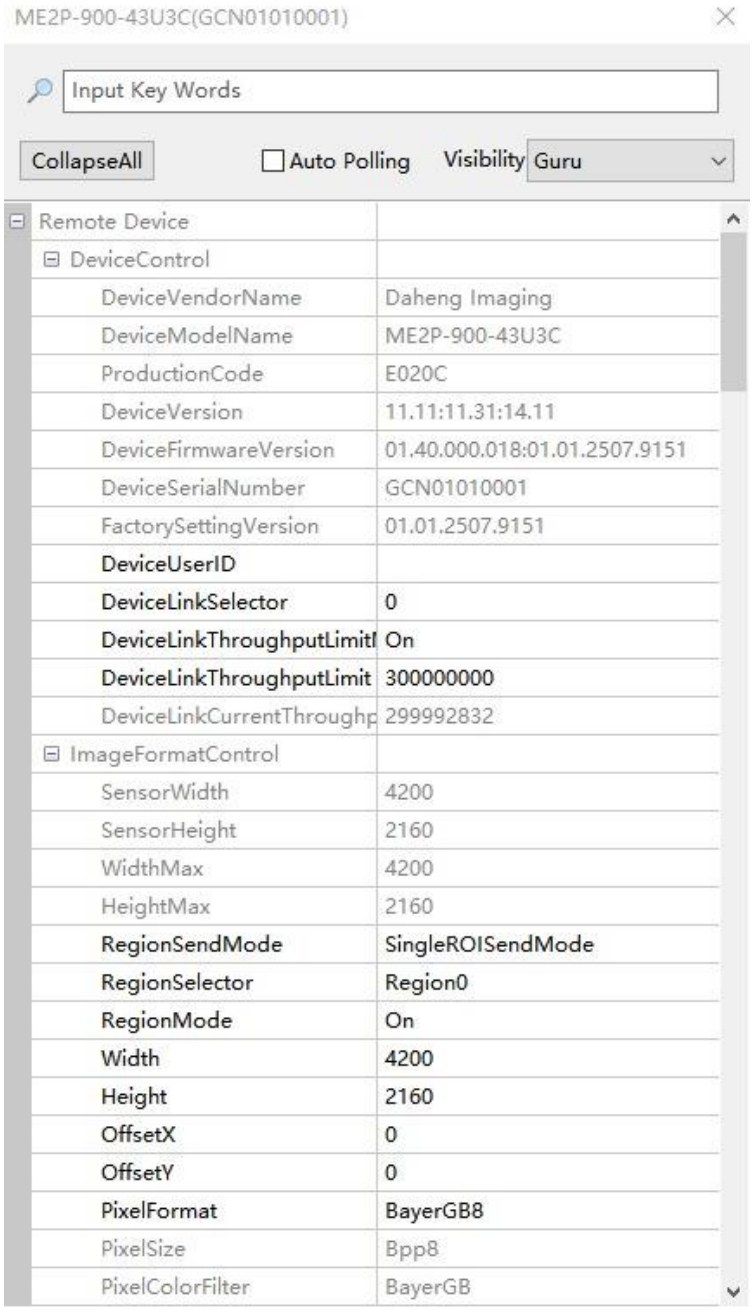


Figure 2-21 Properties Setting UI

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.15.2. Sample Code

Steps to display the properties setting window UI:

(1) Create the handle of properties window.

(2) Set the display position of the Properties window.

(3) Set the shown mode of the Properties window (The BLOCK_SHOW_MODE setting is only configured within the console environment. Do not configure it within the GUI application, as doing so may result in undefined behavior).

(4) Set the display title of the Properties window. If no title is set, the default display will be the custom name setting by the user. Should neither the custom name nor any title be configured, the camera's model number and serial number will be displayed as the title.

(5) Destroy the properties window.

```
    namespace GxAutoConnect
    {
        class Sample
        {

            static void Main(string[] args)
            {
                try
                {
                    // Initialise library.
                    IGXFactory.GetInstance().Init();

                    // Enumerate cameras.
                    List<IGXDeviceInfo> lstDevInfo = new List<IGXDeviceInfo>();
                             IGXFactory.GetInstance().UpdateAllDeviceList(300,
lstDevInfo);
                    if (lstDevInfo.Count < 1)
                    {
                        IGXFactory.GetInstance().Uninit();
                        return;
                    }

                    // Turn on the camera.
                    IGXDevice objDevice = IGXFactory.GetInstance().
                        OpenDeviceBySN(lstDevInfo[0].GetSN(),
                        GX_ACCESS_MODE.GX_ACCESS_CONTROL);

                   //1. Create a window operation handle.
                    IntPtr hWindowHandle = IntPtr.Zero;
                   objDevice.GXCreateWnd(GX_WINDOWS_ID.GX_WND_PROP,
                                         IntPtr.Zero,
                                         ref hWindowHandle);

                   //2. The Properties window is positioned relative to the
                   // top-left corner of the screen, with a width and height
                   // of 700 pixels.
                    objDevice.GXSetShowPosition(hWindowHandle, 0, 0,
                                                700, 700);

                  //3. Set the properties window display mode to block (set
                  // BLOCK_SHOW_MODE when the programme is a console
                  // application. Do not set for GUI programmes).
```

```
                objDevice.GXSetShowMode(hHandle, BLOCK_SHOW_MODE);

                //4. Set the title of the properties window.
                objDevice.GXSetWndTitle(hHandle, "TITLE");

                //5. Display the properties window.
                objDevice.GXShowWnd(hHandle, true);

                //6. Destroy the properties window.
                objDevice.GXDestroyWnd(hHandle);



                objDevice.Close();
                IGXFactory.GetInstance().Uninit();
            }
            catch (CGalaxyException ex)
            {
                Console.WriteLine("GalaxyException: " + ex.Message);
            }
            catch (Exception ex)
            {
                Console.WriteLine("Exception: " + ex.Message);
            }
        }
    }
}
```

2.15.3. Notes

(1) This properties window does not support saving or loading the coefficients of pixel level FFC. Should you require this functionality, please utilise GalaxyView or the FFC plugin.

(2) The blocking display mode (BLOCK_SHOW_MODE) is only applicable when invoked within a console framework. Should your application be a GUI programme (such as Qt, MFC, Winforms, etc.), please refrain from invoking it, as doing so may result in undefined behaviour.

2.16. Saving Images and Videos

2.16.1. Saving Images

The sample code of saving images.

```
String strError = "";
GX_STATUS_LIST emStatus = GX_STATUS_LIST.GX_STATUS_SUCCESS;
try
{
    IGXFactory.GetInstance().Init();

    List<IGXDeviceInfo> listGXDeviceInfoEx = new List<IGXDeviceInfo>();
    ulong nType = (ulong)(GX_TL_TYPE_LIST.GX_TL_TYPE_GEV |
              GX_TL_TYPE_LIST.GX_TL_TYPE_U3V);
    IGXFactory.GetInstance().UpdateAllDeviceListEx(nType, 1000,
              listGXDeviceInfoEx);

    IGXDevice objDevice = IGXFactory.GetInstance().OpenDeviceBySN
    (listGXDeviceInfoEx[0].GetSN(),GX_ACCESS_MODE.GX_ACCESS_EXCLUSIVE);

    IGXFeatureControl objFeatureControl =
              objDevice.GetRemoteFeatureControl();
```

```
    IGXStream objStream = objDevice.OpenStream(0);
    objFeatureControl.GetEnumFeature("AcquisitionMode").SetValue("Continuous
");

    objStream.StartGrab();
    objFeatureControl.GetCommandFeature("AcquisitionStart").Execute();

    IFrameData frameData = objStream.DQBuf(1000);

    GX_SAVE_IMAGE_INFO stSaveImageInfo = new GX_SAVE_IMAGE_INFO();
    stSaveImageInfo.emCfaMethod = GX_CFA_METHOD.GX_CFA_METHOD_BALANCE;
    stSaveImageInfo.emImgFormat =
            GX_IMAGE_FORMAT_TYPE.GX_IMAGE_FORMAT_TIFF;
    stSaveImageInfo.emSrcFormat =
            (GX_PIXEL_FORMAT_ENTRY)frameData.GetPixelFormat();
    stSaveImageInfo.nHeight = (uint)frameData.GetHeight();
    stSaveImageInfo.nWidth = (uint)frameData.GetWidth();
    stSaveImageInfo.nImgQuality = 99;
    stSaveImageInfo.pImageBuffer = frameData.GetBuffer();
    string path = "SaveImageTest.tiff";

    stSaveImageInfo.pImgPath = path;
    IGXFactory.GetInstance().SaveImage(ref stSaveImageInfo);
    objStream.QBuf(frameData);

    objFeatureControl.GetCommandFeature("AcquisitionStop").Execute();
    objStream.StopGrab();

}
catch (CGalaxyException ex)
{
    emStatus = (GX_STATUS_LIST)ex.GetErrorCode();
    strError = ex.ToString();

    IGXFactory.GetInstance().Uninit();
    Assert.AreEqual(GX_STATUS_LIST.GX_STATUS_INVALID_CALL, emStatus);
}
finally
{
    IGXFactory.GetInstance().Uninit();
}
```

2.16.2. Saving Videos

The sample code of saving videos.

```
// To activate the device, use the serial number as an example here. The serial
// number should be verified against the actual device. Users may select alternative
// methods to activate the device.
IGXDevice objIGXDevice = objIGXFactory.OpenDeviceBySN("RN0001007012",
                                                      GX_ACCESS_EXCLUSIVE);
String strError = "";
GX_STATUS_LIST emStatus = GX_STATUS_LIST.GX_STATUS_SUCCESS;
try
{
    IGXFactory.GetInstance().Init();

    List<IGXDeviceInfo> listGXDeviceInfoEx = new List<IGXDeviceInfo>();
    ulong nType = (ulong)(GX_TL_TYPE_LIST.GX_TL_TYPE_GEV |
                          GX_TL_TYPE_LIST.GX_TL_TYPE_U3V);
    IGXFactory.GetInstance().UpdateAllDeviceListEx(nType, 1000,
                          listGXDeviceInfoEx);
```

```
    IGXDevice objDevice = IGXFactory.GetInstance().OpenDeviceBySN
    (listGXDeviceInfoEx[0].GetSN(),GX_ACCESS_MODE.GX_ACCESS_EXCLUSIVE);

    IGXFeatureControl objFeatureControl =
                      objDevice.GetRemoteFeatureControl();

    IGXStream objStream = objDevice.OpenStream(0);

   objFeatureControl.GetEnumFeature("AcquisitionMode").SetValue("Continuous
    ");

    objStream.StartGrab();
    objFeatureControl.GetCommandFeature("AcquisitionStart").Execute();

    long nHeight =
    objFeatureControl.GetIntFeature("Height").GetValue();
    long nWidth = objFeatureControl.GetIntFeature("Width").GetValue();
    UInt64 nPixelFormat = objFeatureControl.GetEnumFeature
                ("PixelFormat").GetEnumValue().nCurValue;

    GX_RECORD_PARAM stRecordParam = new GX_RECORD_PARAM();
    stRecordParam.emVideoFormat =
                GX_VIDEO_FORMAT_TYPE.GX_VIDEO_FORMAT_H264_MP4;
    stRecordParam.emPixelFormat = (GX_PIXEL_FORMAT_ENTRY)nPixelFormat;
    stRecordParam.nFrameRate = 1;
    stRecordParam.nBitRate = 20;
    stRecordParam.nHeight = (uint)nHeight;
    stRecordParam.nWidth = (uint)nWidth;
    string path = "SaveImageTest.avi";

    stRecordParam.pVideoPath = path;
    IGxVideoSaver   saver   =   IGXFactory.GetInstance().CreateVideoSaver(ref
stRecordParam);

    for(int i = 0; i < 30; i++)
    {
        IFrameData pFrame = objStream.DQBuf(1000);
        saver.AddFrame(pFrame.GetBuffer());
        objStream.QBuf(pFrame);
    }

    saver.Close();
    objFeatureControl.GetCommandFeature("AcquisitionStop").Execute();
    objStream.StopGrab();
}
catch (CGalaxyException ex)
{
    emStatus = (GX_STATUS_LIST)ex.GetErrorCode();
    strError = ex.ToString();

    IGXFactory.GetInstance().Uninit();
}
finally
{
    IGXFactory.GetInstance().Uninit();
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3. Data Type Definition

3.1. GX_STATUS_LIST

Definition	Value	Explanation
GX_STATUS_SUCCESS	0	Success.
GX_STATUS_ERROR	-1	Error unexpected.
GX_STATUS_NOT_FOUND_TL	-2	Cannot find TL library.
GX_STATUS_NOT_FOUND_DEVICE	-3	Cannot find devices.
GX_STATUS_OFFLINE	-4	Current device status is offline.
GX_STATUS_INVALID_PARAMETER	-5	Invalid parameter.
GX_STATUS_INVALID_HANDLE	-6	Invalid handle.
GX_STATUS_INVALID_CALL	-7	Invalid interface calling.
GX_STATUS_INVALID_ACCESS	-8	Current function cannot be accessed or the device access mode is incorrect.
GX_STATUS_NEED_MORE_BUFFER	-9	User's buffer is not enough.
GX_STATUS_ERROR_TYPE	-10	Feature type error, such as use IntFeature to handle float function.
GX_STATUS_OUT_OF_RANGE	-11	Input value is out of the range.
GX_STATUS_NOT_IMPLEMENTED	-12	Unsupported function.
GX_STATUS_NOT_INIT_API	-13	Not call initialize interface.
GX_STATUS_TIMEOUT	-14	Timeout error.
GX_STATUS_MEMORY_NOT_ENOUGH	-15	Out of memory

3.2. GX_LOG_TYPE_LIST

Definition	Value	Explanation
GX_LOG_TYPE_OFF	0x00000000	Turn off all types of log output
GX_LOG_TYPE_FATAL	0x00000001	Output FATAL logs
GX_LOG_TYPE_ERROR	0x00000010	Output ERROR logs
GX_LOG_TYPE_WARN	0x00000100	Output WARN logs
GX_LOG_TYPE_INFO	0x00001000	Output INFO logs
GX_LOG_TYPE_DEBUG	0x00010000	Output DEBUG logs
GX_LOG_TYPE_TRACE	0x00100000	Output TRACE logs

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3.3. GX_DEVICE_CLASS_LIST

Definition	Value	Explanation
GX_DEVICE_CLASS_UNKNOWN	0	The class of the Device is unknown.
GX_DEVICE_CLASS_USB2	1	USB2.0 device.
GX_DEVICE_CLASS_GEV	2	GigE device.
GX_DEVICE_CLASS_U3V	3	USB3.0 device.
GX_DEVICE_CLASS_SMART	4	Smart device
GX_DEVICE_CLASS_CXP	5	CXP device

3.4. GX_ACCESS_STATUS

Definition	Value	Explanation
GX_ACCESS_STATUS_UNKNOWN	0	The status of the current device is unknown.
GX_ACCESS_STATUS_READWRITE	1	The status of the current device is readable/writable.
GX_ACCESS_STATUS_READONLY	2	The status of the current device is read-only.
GX_ACCESS_STATUS_NOACCESS	3	The status of the current device is not readable / writable.

3.5. GX_ACCESS_MODE

Definition	Value	Explanation
GX_ACCESS_READONLY	2	The access mode is read-only mode.
GX_ACCESS_CONTROL	3	The access mode is control mode.
GX_ACCESS_EXCLUSIVE	4	The access mode is exclusive mode.

3.6. GX_PIXEL_FORMAT_ENTRY

Definition	Value	Explanation
GX_PIXEL_FORMAT_UNDEFINED	0x00000000	-
GX_PIXEL_FORMAT_MONO8	0x01080001	Monochrome 8-bit
GX_PIXEL_FORMAT_MONO8_SIGNED	0x01080002	Monochrome 8-bit signed
GX_PIXEL_FORMAT_MONO10	0x01100003	Monochrome 10-bit unpacked
GX_PIXEL_FORMAT_MONO10_P	0x010A0046	Monochrome 10-bit packed
GX_PIXEL_FORMAT_MONO12	0x01100005	Monochrome 12-bit unpacked
GX_PIXEL_FORMAT_MONO12_P	0x010C0047	Monochrome 12-bit packed
GX_PIXEL_FORMAT_MONO14	0x01100025	Monochrome 14-bit unpacked
GX_PIXEL_FORMAT_MONO14_P	0x010E0104	Monochrome 14-bit packed
GX_PIXEL_FORMAT_MONO16	0x01100007	Monochrome 16-bit

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_PIXEL_FORMAT_BAYER_GR8	0x01080008	Bayer Green-Red 8-bit
GX_PIXEL_FORMAT_BAYER_RG8	0x01080009	Bayer Red-Green 8-bit
GX_PIXEL_FORMAT_BAYER_GB8	0x0108000A	Bayer Green-Blue 8-bit
GX_PIXEL_FORMAT_BAYER_BG8	0x0108000B	Bayer Blue-Green 8-bit
GX_PIXEL_FORMAT_BAYER_GR10	0x0110000C	Bayer Green-Red 10-bit
GX_PIXEL_FORMAT_BAYER_GR10_P	0x010A0056	Bayer Green-Red 10-bit packed
GX_PIXEL_FORMAT_BAYER_RG10	0x0110000D	Bayer Red-Green 10-bit
GX_PIXEL_FORMAT_BAYER_RG10_P	0x010A0058	Bayer Red-Green 10-bit packed
GX_PIXEL_FORMAT_BAYER_GB10	0x0110000E	Bayer Green-Blue 10-bit
GX_PIXEL_FORMAT_BAYER_GB10_P	0x010A0054	Bayer Green-Blue 10-bit packed
GX_PIXEL_FORMAT_BAYER_BG10	0x0110000F	Bayer Blue-Green 10-bit
GX_PIXEL_FORMAT_BAYER_BG10_P	0x010A0052	Bayer Blue-Green 10-bit packed
GX_PIXEL_FORMAT_BAYER_GR12	0x01100010	Bayer Green-Red 12-bit
GX_PIXEL_FORMAT_BAYER_GR12_P	0x010C0057	Bayer Green-Red 12-bit packed
GX_PIXEL_FORMAT_BAYER_RG12	0x01100011	Bayer Red-Green 12-bit
GX_PIXEL_FORMAT_BAYER_RG12_P	0x010C0059	Bayer Red-Green 12-bit packed
GX_PIXEL_FORMAT_BAYER_GB12	0x01100012	Bayer Green-Blue 12-bit
GX_PIXEL_FORMAT_BAYER_GB12_P	0x010C0055	Bayer Green-Blue 12-bit packed
GX_PIXEL_FORMAT_BAYER_BG12	0x01100013	Bayer Blue-Green 12-bit
GX_PIXEL_FORMAT_BAYER_BG12_P	0x010C0053	Bayer Blue-Green 12-bit packed
GX_PIXEL_FORMAT_BAYER_GR14	0x01100109	Bayer Green-Red 14-bit
GX_PIXEL_FORMAT_BAYER_GR14_P	0x010E0105	Bayer Green-Red 14-bit packed
GX_PIXEL_FORMAT_BAYER_RG14	0x0110010A	Bayer Red-Green 14-bit
GX_PIXEL_FORMAT_BAYER_RG14_P	0x010E0106	Bayer Red-Green 14-bit packed
GX_PIXEL_FORMAT_BAYER_GB14	0x0110010B	Bayer Green-Blue 14-bit
GX_PIXEL_FORMAT_BAYER_GB14_P	0x010E0107	Bayer Green-Blue 14-bit packed
GX_PIXEL_FORMAT_BAYER_BG14	0x0110010C	Bayer Blue-Green 14-bit
GX_PIXEL_FORMAT_BAYER_BG14_P	0x010E0108	Bayer Blue-Green 14-bit packed
GX_PIXEL_FORMAT_BAYER_GR16	0x0110002E	Bayer Green-Red 16-bit
GX_PIXEL_FORMAT_BAYER_RG16	0x0110002F	Bayer Red-Green 16-bit
GX_PIXEL_FORMAT_BAYER_GB16	0x01100030	Bayer Green-Blue 16-bit
GX_PIXEL_FORMAT_BAYER_BG16	0x01100031	Bayer Blue-Green 16-bit
GX_PIXEL_FORMAT_RGB8_PLANAR	0x02180021	Red-Green-Blue 8-bit planar
GX_PIXEL_FORMAT_RGB10_PLANAR	0x02300022	Red-Green-Blue 10-bit unpacked

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_PIXEL_FORMAT_RGB12_PLANAR	0x02300023	Red-Green-Blue 12-bit unpacked
GX_PIXEL_FORMAT_RGB16_PLANAR	0x02300024	Red-Green-Blue 16-bit planar
GX_PIXEL_FORMAT_RGB8	0x2180014	Red-Green-Blue 8-bit
GX_PIXEL_FORMAT_RGB10	0x2300018	Red-Green-Blue 10-bit
GX_PIXEL_FORMAT_RGB12	0x230001A	Red-Green-Blue 12-bit
GX_PIXEL_FORMAT_RGB14	0x230005E	Red-Green-Blue 14-bit
GX_PIXEL_FORMAT_RGB16	0x2300033	Red-Green-Blue 16-bit
GX_PIXEL_FORMAT_BGR8	0x2180015	Blue-Green-Red 8-bit
GX_PIXEL_FORMAT_BGR10	0x2300019	Blue-Green-Red 10-bit
GX_PIXEL_FORMAT_BGR12	0x230001B	Blue-Green-Red 12-bit
GX_PIXEL_FORMAT_BGR14	0x230004A	Blue-Green-Red 14-bit
GX_PIXEL_FORMAT_BGR16	0x230004B	Blue-Green-Red 16-bit
GX_PIXEL_FORMAT_RGBA8	0x2200016	Red-Green-Blue-Alpha 8-bit
GX_PIXEL_FORMAT_BGRA8	0x2200017	Blue-Green-Red-Alpha 8-bit
GX_PIXEL_FORMAT_ARGB8	0x2200018	Alpha-Red-Green-Blue 8-bit
GX_PIXEL_FORMAT_ABGR8	0x2200019	Alpha-Blue-Green-Red 8-bit
GX_PIXEL_FORMAT_R8	0x010800C9	Red 8-bit
GX_PIXEL_FORMAT_G8	0x010800CD	Green 8-bit
GX_PIXEL_FORMAT_B8	0x010800D1	Blue 8-bit
GX_PIXEL_FORMAT_COORD3D_ABC32F	0x026000C0	3D coordinate A-B-C 32-bit floating point
GX_PIXEL_FORMAT_COORD3D_ABC32F_PLANAR	0x026000C1	3D coordinate A-B-C 32-bit floating point planar
GX_PIXEL_FORMAT_COORD3D_C16	0x011000B8	3D coordinate C 16-bit
GX_PIXEL_FORMAT_COORD3D_C16_I16	0x0110FF02	custom pixel format
GX_PIXEL_FORMAT_COORD3D_C16_S16	0x0110FF03	custom pixel format
GX_PIXEL_FORMAT_COORD3D_C16_I16_S16	0x0110FF04	custom pixel format
GX_PIXEL_FORMAT_YUV444_8	0x2180020	YUV444 8-bit
GX_PIXEL_FORMAT_YUV422_8	0x2100032	YUV422 8-bit
GX_PIXEL_FORMAT_YUV411_8	0x20C001E	YUV411 8-bit
GX_PIXEL_FORMAT_YUV420_8_PLANAR	0x20C0040	YUV420 8-bit planar
GX_PIXEL_FORMAT_YCBCR444_8	0x218005B	YCBCR 444 8-bit
GX_PIXEL_FORMAT_YCBCR422_8	0x210003B	YCBCR 422 8-bit
GX_PIXEL_FORMAT_YCBCR411_8	0x20C005A	YCBCR 411 8-bit
GX_PIXEL_FORMAT_YCBCR601_444_8	0x218003D	YCBCR601 444 8-bit

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_PIXEL_FORMAT_YCBCR601_422_8	0x210003E	YCBCR601 422 8-bit
GX_PIXEL_FORMAT_YCBCR601_411_8	0x20C003F	YCBCR601 411 8-bit
GX_PIXEL_FORMAT_YCBCR709_444_8	0x2180040	YCBCR709 444 8-bit
GX_PIXEL_FORMAT_YCBCR709_422_8	0x2100041	YCBCR709 422 8-bit
GX_PIXEL_FORMAT_YCBCR709_411_8	0x20C0042	YCBCR709 411 8-bit
GX_PIXEL_FORMAT_MONO10_PACKED	0x010C0004	GigE Vision specific format, Monochrome 10-bit packed
GX_PIXEL_FORMAT_MONO12_PACKED	0x010C0006	GigE Vision specific format, Monochrome 12-bit packed
GX_PIXEL_FORMAT_BAYER_BG10_PACKED	0x010C0029	GigE Vision specific format, Bayer Blue-Green 10-bit packed
GX_PIXEL_FORMAT_BAYER_BG12_PACKED	0x010C002D	GigE Vision specific format, Bayer Blue-Green 12-bit packed
GX_PIXEL_FORMAT_BAYER_GB10_PACKED	0x010C0028	GigE Vision specific format, Bayer Green-Blue 10-bit packed
GX_PIXEL_FORMAT_BAYER_GB12_PACKED	0x010C002C	GigE Vision specific format, Bayer Green-Blue 12-bit packed
GX_PIXEL_FORMAT_BAYER_GR10_PACKED	0x010C0026	GigE Vision specific format, Bayer Green-Red 10-bit packed
GX_PIXEL_FORMAT_BAYER_GR12_PACKED	0x010C002A	GigE Vision specific format, Bayer Green-Red 12-bit packed
GX_PIXEL_FORMAT_BAYER_RG10_PACKED	0x010C0027	GigE Vision specific format, Bayer Red-Green 10-bit packed
GX_PIXEL_FORMAT_BAYER_RG12_PACKED	0x010C002B	GigE Vision specific format, Bayer Red-Green 12-bit packed

3.7. GX_FRAME_STATUS_LIST

Definition	Value	Explanation
GX_FRAME_STATUS_SUCCESS	0	The frame status is successful.
GX_FRAME_STATUS_INCOMPLETE	-1	The frame status is incomplete.

3.8. GX_FEATURE_TYPE

Definition	Value	Explanation
GX_FEATURE_INT	0x10000000	Integer function.
GX_FEATURE_FLOAT	0x20000000	Float function.
GX_FEATURE_ENUM	0x30000000	Enumerated function.
GX_FEATURE_BOOL	0x40000000	Boolean function.
GX_FEATURE_STRING	0x50000000	String function.
GX_FEATURE_BUFFER	0x60000000	Register function.
GX_FEATURE_COMMAND	0x70000000	Command function.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3.9. GX_BAYER_CONVERT_TYPE_LIST

Definition	Value	Explanation
GX_RAW2RGB_NEIGHBOUR	0	Neighborhood average interpolation algorithm.
GX_RAW2RGB_ADAPTIVE	1	Edge adaptive interpolation algorithm.
GX_RAW2RGB_NEIGHBOUR3	2	Larger neighborhood average algorithm.

3.10. GX_VALLID_BIT_LIST

Definition	Value	Explanation
GX_BIT_0_7	0	bit 0~7
GX_BIT_1_8	1	bit 1~8
GX_BIT_2_9	2	bit 2~9
GX_BIT_3_10	3	bit 3~10
GX_BIT_4_11	4	bit 4~11
GX_BIT_5_12	5	bit 5~12
GX_BIT_6_13	6	bit 6~13
GX_BIT_7_14	7	bit 7~14
GX_BIT_8_15	8	bit 8~15

3.11. GX_IP_CONFIGURE_MODE

Definition	Value	Explanation
IP_CONFIGURE_LLA	4	Start LLA mode to configure IP address.
IP_CONFIGURE_STATIC_IP	5	Use static IP mode to configure IP address.
IP_CONFIGURE_DHCP	6	Use DHCP mode, use DHCP server to configure IP address.
IP_CONFIGURE_DEFAULT	7	Use default mode to configure IP address.

3.12. GX_RESET_DEVICE_MODE

Definition	Value	Explanation
GX_MANUFACTURER_SPECIFIC_RESET	2	Device reset. It is equivalent to powering off and powering up the device.

3.13. COLOR_TRANSFORM_FACTOR

Definition	Default value	Value range	Explanation
float fGain00	1.0	-4.0 ~ 4.0	Red Channel Gain contribution to the red pixel.
float fGain01	0.0	-4.0 ~ 4.0	Green Channel Gain contribution to the red pixel.
float fGain02	0.0	-4.0 ~ 4.0	Blue Channel Gain contribution to the red pixel.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

float fGain10	0.0	-4.0 ~ 4.0	Red Channel Gain contribution to the green pixel.
float fGain11	1.0	-4.0 ~ 4.0	Green Channel Gain contribution to the green pixel.
float fGain12	0.0	-4.0 ~ 4.0	Blue Channel Gain contribution to the green pixel.
float fGain20	0.0	-4.0 ~ 4.0	Red Channel Gain contribution to the blue pixel.
float fGain21	0.0	-4.0 ~ 4.0	Green Channel Gain contribution to the blue pixel.

3.14. GX_TL_TYPE_LIST

Definition	Value	Explanation
GX_TL_TYPE_UNKNOWN	0	Unknown type
GX_TL_TYPE_USB	1	USB2.0 type
GX_TL_TYPE_GEV	2	GigE type
GX_TL_TYPE_U3V	4	USB3.0 type
GX_TL_TYPE_CXP	8	CXP type

3.15. GX_ACTUAL_BITS

Definition	Value	Explanation
GX_ACTUAL_BITS_8	8	8bit
GX_ACTUAL_BITS_10	10	10bit
GX_ACTUAL_BITS_12	12	12bit
GX_ACTUAL_BITS_14	14	14bit
GX_ACTUAL_BITS_16	16	16bit

3.16. GX_PIXEL_COLOR_FILTER

Definition	Value	Explanation
NONE	0	Non Bayer format
BAYERRG	1	BayerRG
BAYERGB	2	BayerGB
BAYERGR	3	BayerGR
BAYERBG	4	BayerBG

3.17. GX_STATIC_DEFECT_CORRECTION

Definition	Explanation
UInt32 nImgWidth	The width of the current image
UInt32 nImgHeight	The height of the current image
UInt32 nImgOffsetX	The horizontal offset of the current image
UInt32 nImgOffsetY	The vertical offset of the current image

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

UInt32 nImgWidthMax	The maximum width supported by the device
GX_PIXEL_COLOR_FILTER nBayerType	Image pixel format
GX_ACTUAL_BITS emActualBits	Actual significant Bit of image pixels

3.18. GX_IMAGE_INFO

Definition	Explanation
IntPtr pInputBuffer	Image buffer pointer
UInt32 nWidth	Image width
UInt32 nHeight	Image height
GX_PIXEL_FORMAT_ENTRY emSrcFormat	Image format

3.19. GX_REGISTER_STACK_ENTRY

Definition	Explanation
UInt64 Address	Register address
IntPtr pBuffer	Register memory address
UInt32 Size	Register memory size

3.20. GX_GIGE_ACTION_COMMAND_RESULT

Definition	Explanation
[MarshalAs(UnmanagedType.ByValArray, SizeConst = 16)] public byte[] DeviceAddress	The current device IP that returns ack (Dotted Decimal Notation IPv4, length 16 bytes)
public int Status	The ACTION status returned by the device, the error originates from the GigaVision protocol **0:** Command sent successfully. **0x8013:** The device is not time synchronized with the master clock. Before executing this interface, “PtpEnable” must be enabled and “PtpStatus” must be set to “Master” or “Slave” (indicating synchronization with the master clock). **0x8015:** The device queue or packet data has overflowed. When the device corresponding to strDeviceAddress is executing the previous GXGigEIssueScheduledActionCommand request and receives a new GXGigEIssueScheduledActionCommand request, this error will be returned. **0x8016:** The Scheduled Action Command issued by GXGigEIssueScheduledActionCommand is outdated.

3.21. GX_FLAT_FIELD_CORRECTION_PARAMETER

Definition	Explanation
IntPtr pBrightBuf	The bright-field images can be collected by calling the GetBuffer interface after obtaining images through the DQBuf interface.
void* pDarkBuf	The dark-field images can be collected by calling the GetBuffer interface after obtaining images through the DQBuf interface. It should be set to IntPtr.Zero when the dark-field images are not required.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_PIXEL_FORMAT_ENTRY emPixelFormat	The pixel format of the current image can be obtained through the GetPixelFormat interface in the image object.
Int32 nImgWid	The width of the current image can be obtained through the GetWidth interface in the image object.
Int32 nImgHei	The height of the current image can be obtained through the GetHeight interface in the image object.
Int32 nFFCBlockSize	The FFC accuracy can be obtained by calling the GetValue interface through enumerating 'FFCBlockSize'.
Int32 nFFCExpectedGray	The FFC expected gray value, range -1~255. When set to -1, it indicates that the maximum value of the gray values in the current image block is used.

3.22. GX_ENUM_ITEM_VALUE

Definition	Explanation
nCurValue	Current value.
strCurSymbolic	Sign name of the current value.
reserved	Reserved.

3.23. GX_WINDOWS_ID

Definition	Value	Explanation
GX_WND_PROP	0	The display window type as properties tree.

3.24. GX_WINDOWS_SHOW_MODE

Definition	Value	Explanation
NON_BLOCK_SHOW_MODE	0	Displaying the properties window in non-blocking mode is the default behaviour. No configuration is required in GUI applications.
BLOCK_SHOW_MODE	1	Displaying the properties window in blocking mode is not modal behaviour. Upon setting this mode, the programme blocks at the GXShowWnd interface. This mode is only applicable when configured in console mode.

3.25. GX_CFA_METHOD

Definition	Value	Explanation
GX_CFA_METHOD_QUICK	0	Quick.
GX_CFA_METHOD_BALANCE	1	Balanced.
GX_CFA_METHOD_OPTIMAL	2	Optimal.

3.26. GX_IMAGE_FORMAT_TYPE

Definition	Value	Explanation
GX_IMAGE_FORMAT_JPEG	0	JPEG format.
GX_IMAGE_FORMAT_PNG	1	PNG format.
GX_IMAGE_FORMAT_TIFF	2	TIFF format.
GX_IMAGE_FORMAT_RAW	3	RAW format.
GX_IMAGE_FORMAT_BMP	4	BMP format.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3.27. GX_VIDEO_FORMAT_TYPE

Definition	Value	Explanation
GX_VIDEO_FORMAT_H264_AVI	0	H264_AVI format.
GX_VIDEO_FORMAT_H264_MP4	1	H264_MP4 format.
GX_VIDEO_FORMAT_ORIGINAL_AVI	2	RAW format.

3.28. GX_SAVE_IMAGE_INFO

Definition	Explanation
pImageBuffer	The buffer of captured images. It can be collected by calling GetBuffer interface after obtaining images through the DQBuf interface.
nWidth	The width of the saved image buffer can be obtained through the GetWidth interface in the image object.
nHeight	The height of the saved image buffer can be obtained through the GetHeight interface in the image object.
emSrcFormat	The pixel format of the saved image buffer can be obtained through the GetPixelFormat interface in the image object.
emCfaMethod	The interpolation method of the saved image.
emImgFormat	The format of saved image [bmp/jpg/png/raw/tiff]
pImgPath	The path of saved image.
nImgQuality	Encoding quality, (0-100], applicable to JPEG only.
nReserved	Reserved.

3.29. GX_RECORD_PARAM

Definition	Explanation
emPixelFormat	The pixel format of the saved image buffer can be obtained through the GetPixelFormat interface in the image object.
emVideoFormat	Video format [bmp/jpg/png/raw/tiff]
nWidth	The width of the saved image buffer can be obtained through the GetWidth interface in the image object.
nHeight	The height of the saved image buffer can be obtained through the GetHeight interface in the image object.
nFrameRate	Frame rate (FPS).
nBitRate	Bit rate (kbps).
pPathName	The name of the saving path.
nReserved	Reserved.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

4. Handle Type Definition

Definition	Explanation
GX_DEVICE_OFFLINE_CALLBACK_HANDLE	The handle of Offline event, which can be get by calling the interface: 'IGXDevice::RegisterDeviceOfflineCallback'.
GX_FEATURE_CALLBACK_HANDLE	The handle of feature update event, which can be get by calling the interface: 'IGXFeatureControl::RegisterFeatureCallback'.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

5. Delegate Function Type Definition

Type	Denifition
Acquisition callback delegate function	public delegate void CaptureDelegate(object obj, GxIAPINET.IFrameData objIFrameData)
Device offline event delegate function	public delegate void DeviceOfflineDelegate(object obj)
Feature update event delegate function	public delegate void FeatureDelegate(object objUserParam, String strFeatureName)
Device reconnection event delegate function	public delegate void DeviceReconnectDelegate(object obj)
Device disconnection event delegate function	public delegate void DeviceDisconnectDelegate(object obj)

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6. Module Interface Definition

6.1. IGXFactory

The 'IGXFactory' is responsible for initializing global resources, enumerating devices, and opening devices.

Interface List:

GetInstance	Static function, which can be called globally. Return to IGXFactory object instance.
Init	Initialize the library resources.
Uninit	Release library resources.
SetLogType	Set type of the output log
GetLogType	Get the output log type
UpdateDeviceList	Enumerate the device (subnet enumeration for GigE Vision device).
UpdateAllDeviceList	Enumerate the device (all subnet enumeration for GigE Vision device)
UpdateAllDeviceListEx	Enumerate the device (all subnet enumeration for GigE Vision device)
OpenDeviceByIP	Open the device by IP address.
OpenDeviceByMAC	Open the device by MAC address.
OpenDeviceBySN	Open the device by SN.
OpenDeviceByUserID	Open the device by user ID.
GigEIPConfiguration	Configure the camera static state (perpetual) IP address.
GigEForceIP	Execute Force IP operation.
GigEResetDevice	Execute reset operation.
CreateImageFormatConvert	Get image format conversion pointer
CreateImageSpecialFunction	Get special image interface
GetAllInterfaceInfo	Get all information of frame grabbers
GetInterface	Get the frame grabber pointer object corresponding to the number of specify and count the subscripts starting from 1
CreateImageProcess	Instantiate IGXImageProcess object
GigEIssueActionCommand	All cameras on the network execute “action” simultaneously by sending ActionCommond
GigEIssueScheduaIdActionCommand	By sending ActionCommond at an absolute point in time, all cameras on the network can simultaneously execute “action”

6.1.1. GetInstance

Interface definition:

static IGXFactory GetInstance ()

Function description:

Static function, and can be called globally. Return to the IGXFactory object instance.

6.1.2. Init

Interface definition:

void Init(void)

Function description:

Initialize library resources.

6.1.3. Uninit

Interface definition:

void Uninit(void)

Function description:

Release library resources.

6.1.4. SetLogType

Interface definition:

void SetLogType(UInt32 emLogType)

Function description:

Set type of the output log.

Parameter:

emLogType	[in]	Output log, refer to GX_LOG_TYPE_LIST eg: GX_LOG_TYPE_FATAL | GX_LOG_TYPE_ERROR | GX_LOG_TYPE_INFO

6.1.5. GetLogType

Interface definition:

void GetLogType(ref UInt32 emLogType)

Function description:

Get the output log type.

Parameter1: Return the current available log types for output.

6.1.6. UpdateDeviceList

Interface definition:

void UpdateDeviceList(UInt32 nTimeout, List<IGXDeviceInfo> listDeviceInfo)

Function description:

Enumerate devices (subnet enumerate for GigE Vision device).

Parameter:

nTimeout	[in]	Timeout time, unit: ms
listDeviceInfo	[in]	Returned device information list

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.1.7. UpdateAllDeviceList

Interface definition:

void UpdateAllDeviceList(UInt32 nTimeout, List<IGXDeviceInfo> listDeviceInfo);

Function description:

Enumerate devices (all subnet enumerate GigE Vision device).

Parameter:

nTimeout	[in]	Timeout time, unit: ms
listDeviceInfo	[in]	Returned device information list

6.1.8. UpdateAllDeviceListEx

Interface definition:

void UpdateAllDeviceListEx(UInt64 nTLType, UInt32 nTimeout,
List<IGXDeviceInfo> listDeviceInfo);

Function description:

Enumerate specified devices (all subnet enumerate GigE Vision device).

Parameter:

nTLType	[in]	Enumerate specified devices, please refer to GX_TL_TYPE_LIST
nTimeout	[in]	Timeout time, unit: ms
listDeviceInfo	[in]	Returned device information list

6.1.9. OpenDeviceByIP/MAC/SN/UserID

Interface definition:

IGXDevice OpenDeviceByIP(String strIP, GX_ACCESS_MODE emAccessMode)

Function description:

Open devices by IP address, return IGXDevice object instance.

Parameter:

strIP	[in]	The device's IP address
emAccessMode	[in]	The mode of open device

Return value: IGXDevice object instance.

6.1.10. GigEIPConfiguration

Interface definition:

void GigEIpConfiguration(String strDeviceMacAddress,
GX_IP_CONFIGURE_MODE emIpConfigFlag,
String strIpAddress,
String strSubnetMask,

String strDefaultGateway,
String strUserID)

Function description:

Configure the camera static state (perpetual) IP address.

Parameter:

trDeviceMacAddress	[in]	The MAC address of the device
emIpConfigFlag	[in]	The IP configuration mode
strIpAddress	[in]	The IP address to be configured
strSubnetMask	[in]	The subnet mask to be configured
strDefaultGateway	[in]	The default gateway to be configured
strUserID	[in]	The user-defined name to be configured

6.1.11. GigEForceIP

Interface definition:

void GigEForceIp(String strDeviceMacAddress,
String strIpAddress,
String strSubnetMask,
String strDefaultGateway)

Function description:

Execute Force IP operation.

Parameter:

trDeviceMacAddress	[in]	The MAC address of the device
strIpAddress	[in]	The IP address to be configured
strSubnetMask	[in]	The subnet mask to be configured
strDefaultGateway	[in]	The default gateway to be configured

6.1.12. GigEResetDevice

Interface definition:

void GigEResetDevice (String strDeviceMacAddress, GX_RESET_DEVICE_MODE ui32FeatureInfo)

Function description:

Execute reset operation. See section 2.2 for details.

Device reset is usually used when debugging GigE cameras. The device has been opened, the program is abnormal, and then the device is reopened immediately, and an error is reported (because the heartbeat time is 5 minutes, the device is still open). In this case, you can use the device reset function to make the device unopened. Then open the device again and it will succeed.

Device reset is usually used when the camera status is abnormal. In this case, the device reset function does not work, and there is no condition to repower the device. Try to use the device reset function to power off and power up the device. After the device is reset, you need to enumerate device and open device again.

Note：

1) Device reset takes about 1s, so you need to ensure that the enumeration interface is called after 1s.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2) If the device is acquiring normally, you cannot use the device reset function, otherwise the device will offline.

Parameter:

strDeviceMacAddress	[in]	The MAC address of the device
ui32FeatureInfo	[in]	Reset device mode

6.1.13. CreateImageFormatConvert

Interface definition:

IGXImageFormatConvert CreateImageFormatConvert()

Function description:

Create and return an instance of the image conversion object.

6.1.14. CreateImageSpecialFunction

Interface definition:

IGXImageSpecialFunction CreateImageSpecialFunction()

Function description:

Create and return an instance of the image special object.

6.1.15. GetAllInterfaceInfo

Interface definition:

void GetAllInterfaceInfo(List<IGXInterfaceInfo>, vectorInterfaceInfo);

Function description:

Get all information of the frame grabbers.

Parameter:

vectorInterfaceInfo	[out]	The returned frame grabber information list

6.1.16. GetInterface

Interface definition:

IGXInterface GetInterface(UInt32 nIndex);

Function description:

Get the user specified frame grabber pointer object.

Parameter:

nIndex	[in]	Frame grabber index, with index counting starting from 1

6.1.17. CreateImageProcess

Interface definition:

IGXImageProcess CreateImageProcess();

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Function description:

Create and return an image processing object instance.

6.1.18. GigEIssueActionCommand

Interface definition:

```
void GigEIssueActionCommand(UInt32 deviceKey,
                            UInt32    groupKey,
                            UInt32    groupMask,
                            string    pBroadcastAddress,
                            string    pSpecialAddress,
                            UInt32    timeoutMs,
                            ref UInt32    pNumResults,
                            GX_GIGE_ACTION_COMMAND_RESULT[]    pResults);
```

Function description:

All cameras on the network execute “action” simultaneously by sending ActionCommond.

Parameter1: Device secret key, corresponding to the “ActionDeviceKey” attribute of the device.

Parameter2: Group secret key, corresponding to the “ActionGroupKey” attribute of the device.

Parameter3: Group mask, corresponding to the “ActionGroupMask” attribute of the device, can not be 0.

Parameter4: The destination IP for sending the “action” command can be broadcast, subnet broadcast, or unicast.

Parameter5: The source IP address for sending the “action” command is used to clearly indicate which network adapter is sending the command from. If not specified, each IP of all network adapters will send the current “action” command. Please fill in an empty string without specifying the network port.

Parameter6: 0 means there is no need to wait for ack to return; Non zero indicates the maximum waiting time for ack return, and after the time is up, the parameter pNumResults returns the actual number of ack received.

Parameter7: Indicate the expected number of ack returns. This value represents the expected number of ACKs that the user expects to receive in feedback from the device executing the “action” command. If nTimeoutMs is 0, ignore this parameter. Therefore, if nTimeoutMs is 0, this parameter can be NULL.

Parameter8: An array containing *pNumResults elements, used to store the state of operation command results. The buffer is filled from scratch. If the received results are less than the available result items, the remaining results will not be changed. If nTimeoutMs is 0, ignore this parameter. Therefore, if nTimeoutMs is 0, this parameter can be NULL. Please refer to GX_GIGE_ACTION_COMMAND_RESULT.

Return: None.

6.1.19. GigEIssueScheduledActionCommand

Interface definition:

```
void GigEIssueScheduledActionCommand(UInt32 deviceKey,
                                     UInt32    groupKey,
                                     UInt32    groupMask,
                                     UInt64    actiontimeNs,
                                     string    pBroadcastAddress,
                                     string    pSpecialAddress,
```

```
UInt32    timeoutMs,
ref UInt32    pNumResults,
GX_GIGE_ACTION_COMMAND_RESULT[]    pResults);
```

Function description:

By sending ActionCommond at an absolute point in time, all cameras on the network can simultaneously execute “action”.

Parameter1: Device secret key, corresponding to the “ActionDeviceKey” attribute of the device.

Parameter2: Group secret key, corresponding to the “ActionGroupKey” attribute of the device.

Parameter3: Group mask, corresponding to the “ActionGroupMask” attribute of the device, can not be 0.

Parameter4: Execution time (in nanoseconds). The actual value can be the master clock value used plus the expected delay time. The master clock value of a set of synchronized camera devices can be obtained by locking the timestamp value GXSetCommandValue (hDevice, "TimestampMatch") from a set of camera devices and reading the timestamp value GXGetIntValue (hDevice, "TimestampLatchValue", &pstIntValue).

Parameter5: The destination IP for sending the “action” command can be broadcast, subnet broadcast, or unicast.

Parameter6: The source IP address for sending the “action” command is used to clearly indicate which network adapter is sending the command from. If not specified, each IP of all network adapters will send the current “action” command. Please fill in an empty string without specifying the network port.

Parameter7: 0 means there is no need to wait for ack to return; Non zero indicates the maximum waiting time for ack return, and after the time is up, the parameter pNumResults returns the actual number of ack received.

Parameter8: Indicate the expected number of ack returns. This value represents the expected number of ACKs that the user expects to receive in feedback from the device executing the “action” command. If nTimeoutMs is 0, ignore this parameter. Therefore, if nTimeoutMs is 0, this parameter can be NULL.

Parameter9: An array containing *pNumResults elements, used to store the state of operation command results. The buffer is filled from scratch. If the received results are less than the available result items, the remaining results will not be changed. If nTimeoutMs is 0, ignore this parameter. Therefore, if nTimeoutMs is 0, this parameter can be NULL. Please refer to GX_GIGE_ACTION_COMMAND_RESULT.

Return: None.

6.1.20. CreateDecompressor

Interface definition:

IGXDecompressor CreateDecompressor()

Function description:

Create and return an image decompression object instance.

6.1.21. SaveImage

Interface definition:

void SaveImage(ref GX_SAVE_IMAGE_INFO stSaveImageInfo)

Function description:

Save images.

6.1.22. CreateVideoSaver

Interface definition:

IGxVideoSaver CreateVideoSaver(ref GX_RECORD_PARAM stRecordParam);

Function description:

Create and return an instance of an image recording object

6.2. IGXInterfaceInfo

Interface information storage unit, storing all interface information, users can get from calling GetAllInterfaceInfo() after enumeration.

Interface list:

GetType	Get Interface type, like USB2.0, USB3.0, GigE, CXP
GetModelName	Get Interface name
GetDisplayName	Get Interface display name
GetSerialNumber	Get Interface serial number
GetInterfaceID	Get Interface ID, the value is 0 when is non CXP type
GetInitFlag	Gets the Interface initialization flag, the value is 0 when is non CXP type
GetVendorName	Get Interface vendor name

6.2.1. GetType

Interface definition:

GX_TL_TYPE_LIST GetType()

Function description:

Get the type of the frame grabber.

6.2.2. GetModelName

Interface definition:

String GetModelName()

Function description:

Get frame grabber name.

6.2.3. GetDisplayName

Interface definition:

String GetDisplayName()

Function description:

Get the display name of the frame grabber.

6.2.4. GetSerialNumber

Interface definition:

String GetSerialNumber()

Function description:

Get the serial number of the frame grabber.

6.2.5. GetInterfaceID

Interface definition:

String GetInterfaceID()

Function description:

Get Interface ID, the value is 0 when is non CXP type.

6.2.6. GetInitFlag

Interface definition:

UInt32 GetInitFlag()

Function description:

Gets the Interface initialization flag, the value is 0 when is non CXP type.

6.2.7. GetVendorName

Interface definition:

String GetVendorName()

Function description:

Get Interface vendor name.

6.3. IGXInterface

Interface function pointer, which can be enumerated and obtained through GetInterface().

Interface list:

GetInterfaceInfo	Get Interface layer information pointer
GetAllDeviceInfo	Get all linked device list under the Interface layer
GetFeatureControl	Get Interface property controller pointer

6.3.1. GetInterfaceInfo

Interface definition:

IGXInterfaceInfo GetInterfaceInfo()

Function description:

Get Interface layer information.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.3.2. GetAllDeviceInfo

Interface definition:

void GetAllDeviceInfo(List<IGXDeviceInfo> listInterfaceInfo)

Function description:

Get all linked device list under the Interface layer.

Parameter1: Return device list.

6.3.3. GetFeatureControl

Interface definition:

IGXFeatureControl GetFeatureControl()

Function description:

Get Interface property controller.

6.4. IGXDeviceInfo

Device information storage unit, stores all the information of the device. You can get the information by the enumeration interface: IGXFactory.UpdateDeviceList(), IGXFactory.UpdateAllDeviceList() or IGXFactory.UpdateAllDeviceListEx()

Interface list:

GetVendorName	Get the name of the manufacturer.
GetModelName	Get the name of the device model.
GetSN	Get the Series Number of the device.
GetDisplayName	Get the display name of the device.
GetDeviceID	Get the device ID, and the ID can be used to identify the device uniquely.
GetUserID	Get the user-defined name.
GetAccessStatus	Get the current addressable status of the device.
GetDeviceClass	Get the device class, such as USB2.0, USB3.0, GigE and CXP.
GetMAC	Get the MAC address of the device, set the value to NULL for non-GigE device.
GetIP	Get the IP address of the device, set the value to NULL for non-GigE device.
GetSubnetMask	Get the subnet mask of the device, set the value to NULL for non-GigE device.
GetGateway	Get the default gateway of the device, set the value to NULL for non-GigE device.
GetNICMAC	Get the corresponding NIC MAC address of the device, set the value to NULL for non-GigE device.
GetNICIP	Get the corresponding NIC IP address of the device, set the value to NULL for non-GigE device.
GetNICSubnetMask	Get the corresponding NIC subnet mask of the device, set the value to NULL for non-GigE device.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GetNICGateway	Get the corresponding NIC default gateway of the device, set the value to NULL for non-GigE device.
GetNICDescription	Get the corresponding NIC description information of the device, set the value to NULL for non-GigE device.

6.4.1. GetVendorName

Interface definition:

String GetVendorName()

Function description:

Get the name of the manufacturer.

6.4.2. GetModelName

Interface definition:

String GetModelName()

Function description:

Get the model name of the device.

6.4.3. GetSN

Interface definition:

String GetSN()

Function description:

Get the Series Number of the device.

6.4.4. GetDisplayName

Interface definition:

String GetDisplayName()

Function description:

Get the display name of the device.

6.4.5. GetDeviceID

Interface definition:

String GetDeviceID()

Function description:

Get the DeviceID, and the ID can be used to identify the device uniquely.

6.4.6. GetUserID

Interface definition:

String GetUserID()

Function description:

Get the user-defined name.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.4.7. GetAccessStatus

Interface definition:

GX_ACCESS_STATUS GetAccessStatus()

Function description:

Get the current addressable status of the device.

6.4.8. GetDeviceClass

Interface definition:

GX_DEVICE_CLASS_LIST GetDeviceClass()

Function description:

Get the device class, such as USB2.0, USB3.0, and GigE.

6.4.9. GetMAC

Interface definition:

String GetMAC()

Function description:

Get the MAC address of the device, set the value to NULL for non-GigE device.

6.4.10. GetIP

Interface definition:

String GetIP()

Function description:

Get the IP address of the device, set the value to NULL for non-GigE device.

6.4.11. GetSubnetMask

Interface definition:

String GetSubnetMask()

Function description:

Get the IP address of the device, set the value to NULL for non-GigE device.

6.4.12. GetGateway

Interface definition:

String GetGateway()

Function description:

Get the default gateway of the device, set the value to NULL for non-GigE device.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.4.13. GetNICMAC

Interface definition:

String GetNICMAC()

Function description:

Get the NIC MAC address of the device, set the value to NULL for non-GigE device.

6.4.14. GetNICIP

Interface definition:

String GetNICIP()

Function description:

Get the NIC IP address of the device, set the value to NULL for non-GigE device.

6.4.15. GetNICSubnetMask

Interface definition:

String GetNICSubnetMask()

Function description:

Get the NIC subnet mask of the device, set the value to NULL for non-GigE device.

6.4.16. GetNICGateway

Interface definition:

String GetNICGateway()

Function description:

Get the NIC default gateway of the device, set the value to NULL for non-GigE device.

6.4.17. GetNICDescription

Interface definition:

String GetNICDescription()

Function description:

Get the NIC description information of the device, set the value to NULL for non-GigE device.

6.5. IGXDevice

Device object, is responsible for getting device information, feature controller (control channel), stream object (acquisition channel), and device offline events. You can get the information by calling the interface: IGXFactory.OpenDeviceByIP/MAC/SN/UserID().

Interface list:

GetDeviceInfo	Get the information object of the device.
GetStreamCount	Get the number of stream channel supported by the device.

OpenStream	The user can open the stream by the stream channel number which is specified, and get the stream channel object.
GetFeatureControl	Get the local device layer feature controller, and control all the functions of the local device via this controller.
GetRemoteFeatureControl	Get the remote device layer feature controller, and control all the functions of the remote device via this controller.
GetEventNumInQueue	Get the event queue length of the device, represents how much cache data is in the current event queue.
FlushEvent	Flush the device event queue.
RegisterDeviceOfflineCallback	Register offline callback function.
UnregisterDeviceOfflineCallback	Unregister offline callback function.
RegisterDeviceReconnectCallback	Register the reconnection callback function.
UnregisterDeviceReconnectCallback	Unregister the reconnection callback function.
RegisterDeviceDisconnectCallback	Register the disconnection callback function.
UnregisterDeviceDisconnectCallback	Unregister the disconnection callback function.
CreateImageProcessConfig	Creating the image processing configure parameter object.
ExportConfigFile	Export the current configuration parameter to the text file.
ImportConfigFile	Import the parameters in the configuration file to the camera.
ExportConfigFileW	Export the current camera configuration parameters to a text file (UNICODE version)
ImportConfigFileW	Import parameters from the configuration file into the camera (UNICODE version)
Close	Close the device.
GetParentInterface	Specify the device handle to get the interface handle it belongs to.

6.5.1. GetDeviceInfo

Interface definition:

IGXDeviceInfo GetDeviceInfo()

Function description:

Get the information object of the device.

6.5.2. GetStreamCount

Interface definition:

UInt32 GetStreamCount()

Function description:

Get the number of stream channel.

6.5.3. OpenStream

Interface definition:

IGXStream OpenStream(UInt32 nStreamID)

Parameter:

nStreamID	[in]	Stream channel number, starts from 0. The user can open the stream by the stream channel number which is specified, and get the stream channel object

6.5.4. GetFeatureControl

Interface definition:

IGXFeatureControl GetFeatureControl()

Function description:

Get the local device layer feature controller, and control all the functions of the local device via this controller.

6.5.5. GetRemoteFeatureControl

Interface definition:

IGXFeatureControl GetRemoteFeatureControl()

Function description:

Get the remote device layer feature controller, and control all the functions of the remote device via this controller.

6.5.6. GetEventNumInQueue

Interface definition:

UInt32 GetEventNumInQueue()

Function description:

Get the event queue length of the device, represents how much cache data is in the current event queue.

6.5.7. FlushEvent

Interface definition:

void FlushEvent()

Function description:

Flush the device event queue.

6.5.8. RegisterDeviceOfflineCallback

Interface definition:

GX_DEVICE_OFFLINE_CALLBACK_HANDLE RegisterDeviceOfflineCallback(
Object objUserParam,DeviceOfflineHandler delDeviceNotify)

Function description:

Register offline callback function.

6.5.9. UnregisterDeviceOfflineCallback

Interface definition:

void UnregisterDeviceOfflineCallback(GX_DEVICE_OFFLINE_CALLBACK_HANDLE hCallBack)

Function description:

Unregister offline callback function.

6.5.10. RegisterDeviceReconnectCallback

Interface definition:

void RegisterDeviceReconnectCallback(
Object objUserParam , DeviceReconnectDelegate delDeviceNotify)

Function description:

Register the reconnection callback function.

6.5.11. UnregisterDeviceReconnectCallback

Interface definition:

void UnregisterDeviceReconnectCallback()

Function description:

Unregister the reconnection callback function.

6.5.12. RegisterDeviceDisconnectCallback

Interface definition:

void RegisterDeviceDisconnectCallback(
Object objUserParam , DeviceDisconnectDelegate delDeviceNotify)

Function description:

Register the disconnection callback function.

6.5.13. UnregisterDeviceDisconnectCallback

Interface definition:

void UnregisterDeviceDisconnectCallback()

Function description:

Unregister the disconnection callback function.

6.5.14. CreateImageProcessConfig

Interface definition:

IImageProcessConfig CreateImageProcessConfig()

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Function description:

Creating the image processing configuration parameter object.

6.5.15. ExportConfigFile

Interface definition:

void ExportConfigFile(String strFilePath)

Function description:

Export the current configuration parameter of the camera to text file.

6.5.16. ImportConfigFile

Interface definition:

void ImportConfigFile(String strFilePath)

Function description:

Import the parameters from the configuration file into the camera..

6.5.17. ExportConfigFileW

Interface definition:

void ExportConfigFileW(String strFilePath)

Function description:

Export the current camera configuration parameters to a text file.

6.5.18. ImportConfigFileW

Interface definition:

void ImportConfigFileW(String strFilePath)

Function description:

Import the parameters from the configuration file into the camera.

6.5.19. Close

Interface definition:

void Close()

Function description:

Close the device.

6.5.20. GetParentInterface

Interface definition:

IGXInterface GetParentInterface()

Function description:

Get the parent Interface pointer of the current device.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.5.21. GXCreateWnd

Interface definition:

void GXCreateWnd(GX_WINDOWS_ID emID, IntPtr hParentWnd, ref IntPtr hWnd);

Function description:

Create the handle of window operation.

6.5.22. GXDestroyWnd

Interface definition:

void GXDestroyWnd(IntPtr hWnd)

Function description:

Destroy the window corresponding to the window operation handle.

6.5.23. GXSetShowPosition

Interface definition:

void GXSetShowPosition(IntPtr hWnd, int nPosX, int nPosY, int nWidth, int nHeight)

Function description:

Set the display position and size of the window.

Notes:

The minimum width and minimum height are 450 and 600 respectively.

6.5.24. GXSetShowMode

Interface definition:

void GXSetShowMode(IntPtr hWnd, GX_WINDOWS_SHOW_MODE emMode)

Function description:

Set the shown mode.

6.5.25. GXShowWnd

Interface definition:

void GXShowWnd(IntPtr hWnd, bool bVisible)

Function description:

Set the window to display or hide.

6.5.26. GXSetWndTitle

Interface definition:

void GXSetWndTitle(IntPtr hWnd, string strTitle)

Function description:

Set the title corresponding to the window.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.6. IGXFeatureControl

Feature controller, the device object IGXDevice and stream object IGXStream have their own feature controllers. Users can control all the function features of the camera via the feature controller, such as getting an integer function controller and then controlling read-write function via the integer controller. You can also register a feature update callback function for the feature. Users can get three different types of feature controllers by these four ways: IGXInterface.GetFeatureControl(), IGXDevice.GetFeatureControl(), IGXDevice.GetRemoteFeatureControl and IGXStream.GetFeatureControl.

Interface list:

GetFeatureNameList	Get all the function name string list which the current controller supports.
GetFeatureType	Get the data type of the current string feature: integer, float, enumerated etc., for more details please see the function definition: GX_FEATURE_TYPE.
IsImplemented	Check to see if the current data node control is available.
IsReadable	Check to see if the current data node is readable.
IsWritable	Check to see if the current data node is writable.
GetIntFeature	Get an integer data node
GetFloatFeature	Get a float data node
GetEnumFeature	Get an enumerated data node
GetBoolFeature	Get a Boolean data node
GetStringFeature	Get a string data node
GetCommandFeature	Get a command data node
GetRegisterFeature	Get a register data node
RegisterFeatureCallback	Register data node, update callback function.
UnregisterFeatureCallback	Unregister data node, update callback function.
ReadPort	Read the value of the specified register
WritePort	Set the value of the specified register
ReadPortStacked	Batch reading of camera specified register values
WritePortStacked	Batch writing of camera specified register values
FeatureSave	Save the user parameter group
FeatureSaveW	Save the user parameter group (UNICODE Version)
FeatureLoad	Load the user parameter group
FeatureLoadW	Load the user parameter group (UNICODE Version)

6.6.1. GetFeatureNameList

Interface definition:

```
void GetFeatureNameList(List<String> listFeatureNameList)
```

Function description:

Get all the function name string list which the current controller supports.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.6.2. GetFeatureType

Interface definition:

GX_FEATURE_TYPE GetFeatureType(String strFeatureName)

Function description:

Get the data type of the current string feature: integer, float, enumerated etc. For more details please see the function definition: GX_FEATURE_TYPE.

6.6.3. IsImplemented

Interface definition:

bool IsImplemented(String strFeatureName)

Function description:

Check to see if the current feature control is supported.

6.6.4. IsReadable

Interface definition:

bool IsReadable(String strFeatureName)

Function description:

Check to see if the current feature is readable.

6.6.5. IsWritable

Interface definition:

bool IsWritable(String strFeatureName)

Function description:

Check to see if the current feature is writable.

6.6.6. GetIntFeature

Interface definition:

IIntFeature GetIntFeature(String strFeatureName)

Function description:

Get an integer feature controller.

6.6.7. GetFloatFeature

Interface definition:

IFloatFeature GetFloatFeature(String strFeatureName)

Function description:

Get a float feature controller.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.6.8. GetEnumFeature

Interface definition:

IEnumFeature GetEnumFeature(String strFeatureName)

Function description:

Get an enumerated controller.

6.6.9. GetBoolFeature

Interface definition:

IBoolFeature GetBoolFeature(String strFeatureName)

Function description:

Get a Boolean controller.

6.6.10. GetStringFeature

Interface definition:

IStringFeature GetStringFeature(String strFeatureName)

Function description:

Get a string controller.

6.6.11. GetCommandFeature

Interface definition:

ICommandFeature GetCommandFeature(String strFeatureName)

Function description:

Get a command controller.

6.6.12. GetRegisterFeature

Interface definition:

IRegisterFeature GetRegisterFeature(String strFeatureName)

Function description:

Get a register controller.

6.6.13. RegisterFeatureCallback

Interface definition:

GX_FEATURE_CALLBACK_HANDLE RegisterFeatureCallback(String strFeatureName, Object objUserParam, FeatureDelegate delFeatureNotify)

Function description:

Register the function feature update callback function.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.6.14. UnregisterFeatureCallback

Interface definition:

void UnregisterFeatureCallback(GX_FEATURE_CALLBACK_HANDLE hCallback)

Function description:

Unregister the function feature update callback function.

6.6.15. ReadPort

Interface definition:

virtual void ReadPort(UInt64 uiAddress, byte[] pBuffer, UInt32 piSize)

Function description:

Read the value of the camera specified register.

Parameter:

uiAddress	[in]	Register address
pBuffer	[out]	The address of the pointer to the read memory value, GigE/10GigE devices read the big endian, other devices read the little endian
piSize	[in]	Size of memory to be read

Note: The value of the GigE camera attribute is read as big endian data.

6.6.16. WritePort

Interface definition:

void WritePort(UInt64 uiAddress, byte[] pBuffer, UInt32 piSize)

Function description:

Write the value of the camera specified register.

Parameter:

uiAddress	[in]	Write the value of the camera specified register
pBuffer	[in]	The address of the pointer to the write memory value, GigE/10GigE devices write to the big endian, other devices write to the little endian
piSize	[in]	Size of memory to be read

Note:

1) Attribute values set by the GigE camera need to be converted to the big endian and then set.
2) After calling the current interface, the node values obtained such as GetEnumFeature(), GetIntFeature(), GetBoolFeature(), etc. are still the pre-modification values, and users can use the ReadPort() or ReadPortStacked() interfaces to obtain the latest attribute values.

6.6.17. ReadPortStacked

Interface definition:

void ReadPortStacked(GX_REGISTER_STACK_ENTRY arrEntries, ref UInt32 nNumEntries)

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Function description:

Read camera specified cameras attribute value in batch, only 4 bytes/unit are supported.

Parameter:

arrEntries	[in]	Register structure array of camera attributes (including register address, register value and register value size), not NULL
	[out]	Read register values
nNumEntries	[in]	The number of obtained camera attribute values, not NULL
	[out]	Number of successfully read register values

Note: The acquired attribute value of GigE camera is BigEndian.

6.6.18. WritePortStacked

Interface definition:

void WritePortStacked(GX_REGISTER_STACK_ENTRY arrEntries, ref UInt32 nNumEntries)

Function description:

Set camera specified cameras attribute value in batch, only 4 bytes/unit are supported.

Parameter:

arrEntries	[in]	Register structure array of camera attributes (including register address, register value and register value size), not NULL
nNumEntries	[in]	Number of registers, cannot be NULL
	[out]	Number of successfully written registers

Note:

1. Set GigE camera attribute values under BigEndian mode.
2. After calling the current interface, the node values obtained by using GetEnumFeature(), GetIntFeature(), GetBoolFeature() and other interfaces are still the values before modification. You can get the latest attribute values via ReadPort() or ReadPortStacked().

6.6.19. FeatureSave

Interface definition:

void FeatureSave(String strFilePath)

Function description:

Save the current frame grabber or camera configuration parameters to the text file.

6.6.20. FeatureSaveW

Interface definition:

void FeatureSaveW(String strWFilePath)

Function description:

Save the current frame grabber or camera configuration parameters to the text file.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.6.21. FeatureLoad

Interface definition:

void FeatureLoad(String strFilePath)

Function description:

Load the parameters in the configuration file into the frame grabber or camera.

6.6.22. FeatureLoadW

Interface definition:

void FeatureLoadW(String strWFilePath)

Function description:

Load the parameters in the configuration file into the frame grabber or camera.

6.7. IIntFeature

Integer function node controller, corresponds to one integer node and can be get from IGXFeatureControl.GetIntFeature.

Interface list:

GetMin	Get the minimum value.
GetMax	Get the maximum value.
GetInc	Get the step length.
GetValue	Get the current value.
SetValue	Set the current value.

6.7.1. GetMin

Interface definition:

int64GetMin()

Function description:

Get the minimum value.

6.7.2. GetMax

Interface definition:

int64 GetMax()

Function description:

Get the maximum value.

6.7.3. GetInc

Interface definition:

int64GetInc()

Function description:

Get the step length.

6.7.4. GetValue

Interface definition:

int64 GetValue()

Function description:

Get the current value.

6.7.5. SetValue

Interface definition:

void SetValue(Int64 nValue)

Function description:

Set the current value.

6.8. IFloatFeature

Float data node, corresponds one float function node and can be get from IGXFeatureControl.GetFloatFeature.

Interface list:

GetMin	Get the minimum value.
GetMax	Get the maximum value.
HasInc	Check to see if there is a step length.
GetInc	Get the step length. If HasInc interface returns 'false', then it will return 0 when calling this interface.
GetUnit	Get the unit.
GetValue	Get the current value.
SetValue	Set the current value.

6.8.1. GetMin

Interface definition:

Double GetMin()

Function description:

Get the minimum value.

6.8.2. GetMax

Interface definition:

Double GetMax()

Function description:

Get the maximum value.

6.8.3. HasInc

Interface definition:

bool HasInc()

Function description:

Check to see if there is a step length.

6.8.4. GetInc

Interface definition:

Double GetInc()

Function description:

Get the step length of float type value. If HasInc interface returns 'false', then it will return 0 when calling this interface.

6.8.5. GetUnit

Interface definition:

String GetUnit()

Function description:

Get the unit of the float value.

6.8.6. GetValue

Interface definition:

Double GetValue()

Function description:

Get the current value.

6.8.7. SetValue

Interface definition:

void SetValue(Double dValue)

Function description:

Set the current value.

6.9. IEnumFeature

The enumeration data node is corresponding to one enumeration function node and can be get by the function: IGXFeatureControl.GetEnumFeature.

Interface list:

GetEnumEntryList	Get the supported enumeration item string list.
GetValue	Get the enumeration item string value.
SetValue	Set the enumeration item value by string.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

SetEnumValue	Set the current enumeration item value which is integer variable.
GetEnumValue	Get current enumeration item value.
GetEnumValue	Get the list of current enumeration item value.

6.9.1. GetEnumEntryList

Interface definition:

List<String> GetEnumEntryList()

Function description:

Get the supported enumeration item string list.

6.9.2. GetValue

Interface definition:

String GetValue()

Function description:

Get the current enumeration item string value.

6.9.3. SetValue

Interface definition:

void SetValue(String strValue)

Function description:

Set the current enumeration item value by string.

6.9.4. SetEnumValue

Interface definition:

void SetEnumValue(Int64 i64Value)

Function description:

Set the current enumeration value which is integer variable.

6.9.5. GetEnumValue

Interface definition:

GX_ENUM_ITEM_VALUE void GetEnumValue()

Function description:

Get current enumeration value. See GX_ENUM_ITEM_VALUE for details.

6.9.6. GetEnumValue

Interface definition:

List<GX_ENUM_ITEM_VALUE> GetEnumValue()

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Function description:

Get the list of current enumeration value.

6.10. IBoolFeatur

The boolean data node is corresponding to one Boolean function node and can be get by the function: IGXFeatureControl.GetBoolFeature.

Interface list:

GetValue	Get the current value.
SetValue	Set the current value.

6.10.1. GetValue

Interface definition:

bool GetValue()

Function description:

Get the current value.

6.10.2. SetValue

Interface definition:

void SetValue(bool bValue)

Function description:

Set the current value.

6.11. IStringFeature

The string data node is corresponding to one string function node and can be get by the function: IGXFeatureControl.GetStringFeature.

Interface list:

GetValue	Get the current value.
SetValue	Set the current value.
GetStringMaxLength	Get the maximum length that can be set, not including the terminator.

6.11.1. GetValue

Interface definition:

String GetValue()

Function description:

Get the current value.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.11.2. SetValue

Interface definition:

void SetValue(String strValue)

Function description:

Set the current value.

6.11.3. GetStringMaxLength

Interface definition:

Int64 GetStringMaxLength()

Function description:

Get the maximum length that can be set, not including the terminator.

6.12. ICommandFeature

The command data node is corresponding to one command function node and can be get by the function: IGXFeatureControl.GetCommandFeature.

Interface list:

Execute	Execute command.

6.12.1. Execute

Interface definition:

void Execute()

Function description:

Execute command.

6.13. IRegisterFeature

The register data node controller is corresponding to one command function node and can be get by the function: IGXFeatureControl.GetCommandFeature.

Interface list:

GetLength	Get the length of the register buffer. The user reads this length to read-write buffer.
GetBuffer	Get the value of the current register buffer. The length of input parameter 'ptrBuffer' must equal to the length that the interface 'GetLength' gets.
SetBuffer	Set the value of the current register buffer. The length of input parameter 'ptrBuffer' must equal to the length that the interface 'GetLength' gets.

6.13.1. GetLength

Interface definition:

Int64 GetLength()

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Function description:

Get the length of the register buffer. The user reads this length to read-write buffer.

6.13.2. GetBuffer

Interface definition:

void GetBuffer(array<Byte> ptrBuffer)

Function description:

Get the value of the current register buffer. The length of input parameter 'ptrBuffer' must equal to the length that the interface 'GetLength' gets.

6.13.3. SetBuffer

Interface definition:

void SetBuffer(array<Byte> ptrBuffer)

Function description:

Set the value of the current register buffer. The length of input parameter 'ptrBuffer' must equal to the length that the interface 'GetLength' gets.

6.14. IGXStream

Representing the stream object, and it is responsible for the related operation of grabbing images, such as unregistering callback function and getting a single image. It can be get by the interface function: IGXDevice.OpenStream interface.

Interface list:

StartGrab	Start stream layer acquisition, whether using callback acquisition or a single image acquisition, the user must call the interface function StartGrab first.
StopGrab	Stop stream layer acquisition.
RegisterCaptureCallback	Register the delegation acquisition callback function.
UnregisterCaptureCallback	Unregister the acquisition callback function.
GetImage	Get a single image.
GetImageNoThrow	Acquire a single frame without any abnormalities
DQBuf	Zero copy method for acquiring single frames
QBuf	Return IFrameData to the acquisition system
GetFeatureControl	Get the stream layer feature controller to control the function feature of the stream layer.
FlushQueue	Flush the cache images in the acquisition queue.
SetAcqusitionBufferNumber	Set the count of acquisition buffer.
Close	Close the stream object.
GetOptimalPacketSize	Get the optimal packet length of the device.
GetPayloadSize	Get the size of the device stream data block

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

RegisterBuffer	The buffer applied by the registered user is used for internal acquisition
UnRegisterBuffer	Cancel the buffer registered using the RegisterBuffer() interface

6.14.1. StartGrab

Interface definition:

void StartGrab()

Interface description:

Start stream layer acquisition, whether using callback acquisition or a single image acquisition, the user must call the StartGrab() first and start stream layer acquisition, including start acquisition thread and resource application operation of TLClass layer.

6.14.2. StopGrab

Interface definition:

void StopGrab()

Interface description:

Stop stream layer acquisition, including stop acquisition thread of TLClass layer and release resources operation etc.

6.14.3. RegisterCaptureCallback

Interface definition:

void RegisterCaptureCallback(Object objUserParam,GxIAPINET::CaptureDelegate delCaptureFun)

Interface description:

Register the user delegate acquisition callback function. Parameter 1 is the user private parameter, it is returned to the user by delegation, and parameter 2 is the user delegate callback function object.

Function requirements:

1) After starting acquisition, that is after calling the interface function StartGrab(), you do not allowed to call the register interface function.
2) The user can register the acquisition callback function repeatedly, and using the overwrite mechanism.

6.14.4. UnregisterCaptureCallback

Interface definition:

void UnregisterCaptureCallback()

Interface description:

Unregister acquisition callback function.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.14.5. GetImage

Interface definition:

IImageData GetImage(UInt32 nTimeout)

Interface description:

Get a single image. Parameter 1 is timeout time, unit is ms. If the image is not captured within the time, the timeout error will returned.

Function requirements:

If the user registers the acquisition delegate function, then it is not allowed to call GetImage() interface. If call this interface forcibly, it will return illegal call exception error.

6.14.6. GetImageNoThrow

Interface definition:

IImageData GetImageNoThrow(UInt32 nTimeout)

Interface description:

Get a single image. Parameter 1 is timeout time, unit is ms. If the image is not captured within the time, NULL will be returned.

Function requirements:

If the user registers the acquisition delegate function, then it is not allowed to call GetImage() interface. If call this interface forcibly, it will return illegal call exception error.

6.14.7. DQBuf

Interface definition:

IFrameData DQBuf(UInt32 nTimeout)

Interface description:

Zero copy method for collecting single frames. Parameter 1 is the timeout period, measured in milliseconds. If the image is not obtained after this time, a timeout error will be returned.

Function requirements:

If the user has registered an acquisition delegation function, the DQBuf() interface is not allowed to be used. If this interface is forcefully called, an illegal call exception will be returned.

6.14.8. QBuf

Interface definition:

void QBuf(IFrameData frameData)

Interface description:

Return ImageData to the acquisition system.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Function requirements:

If the user has registered an acquisition delegation function, the QBuf() interface is not allowed to be used. If this interface is forcefully called, an illegal call exception will be returned.

6.14.9. GetFeatureControl

Interface definition:

IGXFeatureControl GetFeatureControl()

Interface description:

Get stream layer feature controller, to control the stream layer function feature.

6.14.10. FlushQueue

Interface definition:

void FlushQueue()

Interface description:

Flush the cache images in the acquisition queue.

6.14.11. SetAcquisitionBufferNumber

Interface definition:

void SetAcqusitionBufferNumber(uint64_t nBufferNum)

Interface description:

The user set the count of acquisition buffer.

Function requirements:

1) This interface is optional and is not necessary in the start acquisition process.
2) The setting value must be greater than 0, if the setting value is too large and exceed the system capacity, it will not to start acquisition successfully.
3) If need to call this interface, it must be called before start acquisition and it is not allowed to call this interface during the acquisition process.
4) Once the user has set up the buffer number and has been successfully captured, the buffer number will remain valid until the stream object is closed.

6.14.12. Close

Interface definition:

void Close()

Interface description:

Close the stream object.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.14.13. GetOptimalPacketSize

Interface definition:

uint32_t GetOptimalPacketSize(void)

Interface description:

Get the optimal packet length of the device.

Function requirements:

1) This interface only supports Gigabit network cameras to obtain optimal packet length.
2) If need to call this interface, it must be called before starting acquisition and it is not allowed to call this interface during the acquisition process.

6.14.14. GetPayloadSize

Interface definition:

uint64_t GetPayloadSize(void)

Function requirements:

Get the size of the device stream data block.

Note: The user image data size needs to be greater than or equal to the return value of the current interface, otherwise an error may occur.

6.14.15. RegisterBuffer

Interface definition:

void RegisterBuffer(IntPtr pBuffer, UInt64 pBufferSize, IntPtr pUserParam)

Function requirements:

The buffer applied by registered users is used for internal collection, and users can independently manage the buffer memory.

Parameter:

pBuffer	[in]	Buffer pointer requested by the user
pBufferSize	[in]	Buffer length requested by the user
pUserParam	[in]	User defined parameters can be accessed through IImageData::GetUserParam()

Note:

1) The buffer size requested by the user cannot be calculated by itself and needs to be obtained from the GetPayloadSize() interface, otherwise it may cause exceptions. Products such as POLS data frame grabber require alignment of memory size and first address for data acquisition based on performance considerations. Therefore, user needs to align the starting address and size according to the alignment byte requirements specified by the StreamBufferAlignment attribute when using this interface.
2) When the number of registered buffers is small, the acquisition driver completes a single acquisition (buffer is in the consumer queue), and no buffer available in the driver may result in frame loss. Therefore, it is recommended to register at least 3 buffers and complete the use of the image as soon as possible. In addition, hardware such as POLS requires a minimum AnnounceBuffer quantity of 3 or more. If the number of buffers registered by the user is insufficient, it will result in acquisition

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

failure.

3) After calling the RegisterBuffer() interface, if the user modifies the parameters of the camera or frame grabber, resulting in a change in the PayloadSize output, the registered buffer needs to be deregistered, and memory needs to be reapplied and registered based on the new length.

4) RegisterBuffer() is an optional interface. If this interface is not called to register the buffer, the API library will automatically apply for and mine the buffer after mining, without affecting the collection process.

6.14.16. UnRegisterBuffer

Interface definition:

void UnRegisterBuffer(IntPtr pBuffer)

Function requirements:

Unregister the buffer that the user registered using the RegisterBuffer() interface.

Parameter:

pBuffer	[in]	Buffer pointer requested by the user

Note:

This interface needs to be called after the acquisition is stopped. Calling it during the acquisition process will affect the normal collection process.

Sample Code:

```
//Get registered buffer information
Uint64 u64BufferSize = objGXStream.GetPayloadSize();
IGXFeatureControl objStreamFeatureControl =
                                        objGXStream.GetFeatureControl();
Uint32 u32BufAddrAlignment = objStreamFeatureControl.GetIntFeature(
                                    "StreamBufferAlignment").GetValue();
//Apply for the buffer length obtained based on the address alignment
value
Intptr pBuffer = _aligned_malloc( u64BufferSize, u32BufAddrAlignment);
if(IntPtr.Zero == pBuffer)
{
   //Exception handling...
}

//User defined data, NULL for empty
IntPtr pUserParam = IntPtr.Zero;

//Register buffer
objGXStream.RegisterBuffer( pBuffer, u64BufferSize, pUserParam);

//Start acquisition...
//Image processing...
//Stop acquisition...

//Anti registration buffer...
objGXStream.UnRegisterBuffer( pBuffer);

//Realese memory...
```

```
if(IntPtr.Zero!= pBuffer)
{
    _aligned_free(pBuffer);
    pBuffer = IntPtr.Zero;
}
```

6.15. IFrameData

Image object, including image databuffer and image information. After the user registering acquisition callback function, when the images come, the image object will return to the user via entrance parameter of the acquisition callback function.

Interface list:

GetStatus	Get the status of the current frame and see if the frame is complete.
GetPayloadSize	Get the current frame load and the image size.
GetWidth	Get the width of the current frame.
GetHeight	Get the height of the current frame.
GetPixelFormat	Get the pixel format of the current frame.
GetFrameID	Get the current frame ID.
GetTimeStamp	Get the timestamp of the current frame.
GetBuffer	Get the image data buffer, return the IntPtr type pointer and point to the unmanaged memory directly.
ConvertToRaw8	The interface is used for non 8-bit. The user can get the specified 8 valid bit in non 8-bit data.
ConvertToRGB24	Convert the raw data to the RGB24 bit data that is interpolated.
ImageProcess	To enhance the current image, and return the image after enhancing effect.
GetUserParam	Get user parameter.
GetChunkDataFeatureControl()	Get the frame information feature controller for the current image.
GetOffsetX	Get the current horizontal offset of the image.
GetOffsetY	Get the current vertical offset of the image.

6.15.1. GetStatus

Interface definition:

GX_FRAME_STATUS_LIST GetStatus()

Function description:

Get the status of the current frame and see if the frame is complete.

6.15.2. GetPayloadSize

Interface definition:

UInt64 GetPayloadSize()

Function description:

Get the current frame load, and the image size. If the frame information is accessible, this value is the sum of image data size and frame information size; if the frame information is not accessible, this value is the image data size.

6.15.3. GetWidth

Interface definition:

UInt64 GetWidth()

Function description:

Get the width of the current frame.

6.15.4. GetHeight

Interface definition:

UInt64 GetHeight()

Function description:

Get the height of the current frame.

6.15.5. GetPixelFormat

Interface definition:

GX_PIXEL_FORMAT_ENTRYGetPixelFormat()

Function description:

Get the pixelformat of the current frame.

6.15.6. GetFrameID

Interface definition:

UInt64 GetFrameID()

Function description:

Get the current frame ID.

6.15.7. GetTimeStamp

Interface definition:

UInt64 GetTimeStamp()

Function description:

Get the timestamp of the current frame.

6.15.8. GetBuffer

Interface definition:

IntPtr GetBuffer()

Function description:

Get the image data buffer, return the IntPtr type pointer and point to the unmanaged memory directly.

6.15.9. ConvertToRaw8

Interface definition:

IntPtr ConvertToRaw8 (GX_VALID_BIT_LIST emValidBits)

Function description:

The interface is used for non 8-bit data. The user can get specified 8 valid bit in non 8-bit data.

6.15.10. ConvertToRGB24

Interface definition:

IntPtr ConvertToRGB24 (GX_VALID_BIT_LIST emValidBits,
GX_BAYER_CONVERT_TYPE_LIST emConvertType,
emConvertType, bool bFlip)

Function description:

Convert the raw data to the RGB24 bit data by interpolation algorithm.

6.15.11. ImageProcess

Interface definition:

IntPtrImageProcess(IImageProcessConfig objCfg)

Function description:

To enhance the current image, and returns the enhanced image data.

Note:

1) If the 'IImageProcessConfig::IsAccelerate' returns 'true', then it will process the images by accelerating mode. At this time the image height must be an integer multiple of four, otherwise an error will occur in the interface.
2) The mono camera void* returns 8bit image data, the image size is image width * image height.
3) The color camera void* returns RGB image data, image size is image width * image height * 3.

6.15.12. GetUserParam

Interface definition:

IntPtr GetUserParam()

Function description:

Retrieve the user parameters set by the user through RegisterBuffer(). If the interface is not called, the return value is empty.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.15.13. GetChunkDataFeatureControl()

Interface definition:

IGXFeatureControl GetChunkDataFeatureControl()

Function description:

Get the frame information feature controller for the current image.

Returns:

The frame information feature controller for the image.

Note:

If the frame information is not supported, it will raise the CGalaxyException type error when call the interface.

6.15.14. GetOffsetX()

Interface definition:

UInt64 GetOffsetX()

Function description:

Get the current horizontal offset of the image.

Returns:

The horizontal offset of the image.

6.15.15. GetOffsetY()

Interface definition:

UInt64 GetOffsetY()

Function description:

Get the current vertical offset of the image.

Returns:

The vertical offset of the image.

6.16. The vertical offset of the image.IImageData

IImageData is an image data obtained from the IGXStream.GetImage interface. The interface of this object is basically consistent with the 'IFrameData' object. Compared with the interface **IFrameData**, the interface IImageData has one interface "void IImageData.Destroy()", which is used for destroying itself and releasing resources.

6.17. IImagProcessConfig

The configurable parameter object of Image enhancement, which has a set of configuration parameter inside, and can be get by the interface: IGXDevice.CreateImageProcessConfig().

Interface List:

SetValidBit	Select the specified 8 bit valid data that to be get, and the interface is used to specify which 8bit is selected.
GetValidBit	Get the 8 valid bits which are currently specified.
EnableDefectivePixelCorrect	Enable defective-pixel correction.
IsDefectivePixelCorrect	Check whether the defective-pixel correction is enabled currently.
EnableSharpen	Enable sharpness function.
IsSharpen	Check whether the sharpness function is enabled.
EnableAccelerate	Enable accelerating image processing.
IsAccelerate	Check whether it works in the accelerating enable status currently.
SetSharpenParam	Set the sharpness intensity factor.
GetSharpenParam	Get the sharpness intensity factor currently used.
SetContrastParam	Set contrast adjustment parameter.
GetContrastParam	Get the contrast adjustment parameter currently used.
SetGammaParam	Set Gamma coefficient.
GetGammaParam	Get Gamma coefficient.
SetLightnessParam	Set brightness adjustment parameter.
GetLightnessParam	Get brightness adjustment parameter.
EnableDenoise	Enable denoise function.
IsDenoise	Check whether the denoise function is enabled currently.
SetSaturationParam	Set saturation coefficient.
GetSaturationParam	Get saturation coefficient.
SetConvertType	Set image format conversion algorithm.
GetConvertType	Get image format conversion algorithm.
EnableConvertFlip	Enable image format conversion flip.
IsConvertFlip	Check whether the image format conversion flip function is enabled currently.
EnableColorCorrection	Enable color correction.
IsColorCorrection	Check whether the color correction function is enabled.
Reset	Restore the default adjustment parameter.
Destroy	Destroy itself and release resources.
IsUserSetCCParam	Check whether the color correction function is user-set mode.
EnableUserSetCCParam	Enable user-set mode for color correction function.
SetUserCCParam	Set color transform factor by user.
GetUserCCParam	Get color transform factor which set by user.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.17.1. SetValidBit

Interface definition:

void SetValidBit(GX_VALID_BIT_LISTemValidBits)

Function description:

Select the specified 8 bit valid data that to be get, and the interface is used for non-8bit raw data.

6.17.2. GetValidBit

Interface definition:

GX_VALID_BIT_LIST GetValidBit()

Function description:

Get specified 8 bit valid data, and the interface is used for non-8bit raw data.

6.17.3. EnableDefectivePixelCorrect

Interface definition:

void EnableDefectivePixelCorrect(bool bEnable)

Function description:

Enable defective-pixel correction function. The defective pixel correction is enabled when the 'bEnable' is 'ture', and the defective pixel correction is disable when the 'bEnable' is 'false'.

6.17.4. IsDefectivePixelCorrect

Interface definition:

bool IsDefectivePixelCorrect()

Function description:

Get the status of defective pixel correction.

6.17.5. EnableSharpen

Interface definition:

void EnableSharpen(bool bEnable)

Function description:

Enable sharpen function. When the 'bEnable' is 'ture', the sharpness function is enabled; when the 'bEnable' is 'false' the sharpness function is disabled.

6.17.6. IsSharpen

Interface definition:

bool IsSharpen()

Function description:

Get the status of sharpness.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.17.7. EnableAccelerate

Interface definition:

void EnableAccelerate(bool bEnable)

Function description:

Image processing accelerating enable. If it is set 'true', the accelerating function is enabled and if it is set 'false', the accelerating function is disabled. If the CPU of the current PC is not allowed to accelerate because of itself limitation, an error will occurs when the user sets 'true'.

Note:

When the acceleration function is enabled, the current image height must be an integer multiple of four, otherwise an error will occurs in the IFrameData::ImageProces interface or IImageData::ImageProcess interface.

6.17.8. IsAccelerate

Interface definition:

bool IsAccelerate()

Function description:

Check whether the device works in the accelerating enable status currently. 'true' means accelerating and 'false' means not accelerating.

6.17.9. SetSharpenParam

Interface definition:

void SetSharpenParam(Double dParam)

Function description:

Set the sharpness intensity factor.

6.17.10. GetSharpenParam

Interface definition:

System::Double GetSharpenParam()

Function description:

Get the sharpness intensity factor currently used.

6.17.11. SetContrastParam

Interface definition:

void SetContrastParam(Int32 nParam)

Function description:

Set the contrast adjustment parameter.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.17.12. GetContrastParam

Interface definition:

Int32 GetContrastParam()

Function description:

Get the contrast adjustment parameter currently used.

6.17.13. SetGammaParam

Interface definition:

void SetGammaParam(Double dParam)

Function description:

Set Gama coefficient.

6.17.14. GetGammaParam

Interface definition:

System::Double GetGammaParam()

Function description:

Get Gamma coefficient.

6.17.15. SetLightnessParam

Interface definition:

void SetLightnessParam(Int32 nParam)

Function description:

Set brightness adjustment parameter.

6.17.16. GetLightnessParam

Interface definition:

Int32 GetLightnessParam()

Function description:

Get brightness adjustment parameter.

6.17.17. EnableDenoise

Interface definition:

void EnableDenoise(bool bEnable)

Function description:

Enable denoise switch (not support for monochrome cameras), when the 'bEnable' is 'true', the denoise function is enabled; when the 'bEnable' is 'false', the denoise function is disabled.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.17.18. IsDenoise

Interface definition:

bool IsDenoise()

Function description:

Get denoise switch (not support for monochrome cameras).

6.17.19. SetSaturationParam

Interface definition:

void SetSaturationParam(Int32 nParam)

Function description:

Set saturation coefficient (not support for monochrome cameras).

6.17.20. GetSaturationParam

Interface definition:

Int32 GetSaturationParam()

Function description:

Get saturation coefficient (not support for monochrome cameras).

6.17.21. SetConvertType

Interface definition:

void SetConvertType(GX_BAYER_CONVERT_TYPE_LIST emConvertType)

Function description:

Set image format conversion algorithm (not support for monochrome cameras).

6.17.22. GetConvertType

Interface definition:

GX_BAYER_CONVERT_TYPE_LIST GetConvertType()

Function description:

Get interpolation algorithm (not support for monochrome cameras).

6.17.23. EnableConvertFlip

Interface definition:

void EnableConvertFlip(bool bFlip)

Function description:

Enable interpolation flip (not support for monochrome cameras).

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.17.24. IsConvertFlip

Interface definition:

bool IsConvertFlip()

Function description:

Get interpolation flit identification (not support for monochrome cameras), 'true' means flipping and 'false' means not flipping.

6.17.25. EnableColorCorrection

Interface definition:

void EnableColorCorrection(bool bEnable)

Function description:

Enable color correction (not support for monochrome cameras), 'true' means enable color correction and 'false' means disabling color correction.

6.17.26. IsColorCorrection

Interface definition:

bool IsColorCorrection()

Function description:

Check whether the color correction is enabled (not support for monochrome cameras).

6.17.27. Reset

Interface definition:

void Reset()

Function description:

The user can select to restore the optimal default parameter configuration.

6.17.28. Destroy

Interface definition:

void Destroy()

Function description:

Destroy itself and release resources.

6.17.29. IsUserSetCCParam

Interface definition:

bool IsUserSetCCParam()

Function description:

Check whether the color correction function is user-set mode (not support for monochrome cameras). Returns true if it is user-set mode, otherwise returns false.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.17.30. EnableUserSetCCParam

Interface definition:

void EnableUserSetCCParam(bool bIsUserMode)

Function description:

Enable user-set mode for color correction function (not support for monochrome cameras). If bIsUserMode is true the user-set mode is enabled; otherwise it is disabled.

6.17.31. SetUserCCParam

Interface definition:

void SetUserCCParam(COLOR_TRANSFORM_FACTOR stColorTransformFactor)

Function description:

Set stColorTransformFactor to color correction function (only supported for user-set mode) (The recommended range of color correction structure parameters is -4 ~ 4. If the parameter setting is less than -4, the correction effect is consistent with -4. If the parameter setting is greater than 4, the correction effect is consistent with 4.).

6.17.32. GetUserCCParam

Interface definition:

COLOR_TRANSFORM_FACTOR GetUserCCParam()

Function description:

Get current stColorTransformFactor which used to color correction function (only supported for user-set mode).

6.18. IGXImageFormatConvert

Image format conversion interface class, you can convert the image format through this object.

Interface List：

SetDstFormat	Set the desired conversion format
GetDstFormat	Get the desired conversion format
SetInterpolationType	Set image format conversion algorithm
GetInterpolationType	Get image format conversion algorithm
SetAlphaValue	Set the Alpha value of the image with Alpha channel
GetAlphaValue	Get Alpha channel value
GetBufferSizeForConversion	Method 1, get the image Buffer length of the desired format based on the input image pointer
GetBufferSizeForConversion	Method 2, get the image Buffer length of the desired format based on the input image data
Convert	Method 1, get the converted image Buffer according to the input image pointer

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Convert	Method 2, get the converted image Buffer based on the input image data
SetValidBits	Set the effective data bits to be retained when converting the input source image to the desired image format
GetValidBits	Obtain the effective data bits retained when converting the input source image to the desired image format
Set3DCalibParam	This function is used to load 3D calibration parameters and convert the altitude map into a point cloud map
Get3DCalibParam	Obtain 3D calibration parameters
SetYStep	Set the Y-direction step size for converting the height map to the point cloud
GetYStep	Obtain the Y-direction step size of the height conversion point cloud

6.18.1. Support conversion types

The table shows the currently supported conversion image formats, and there is a one-to-many relationship between the source image format and the destination image format (any of the source image formats can be converted to any of the corresponding destination image formats).

	Source image format	Destination image format
Bayer 8bit	GX_PIXEL_FORMAT_BAYER_GR8 GX_PIXEL_FORMAT_BAYER_RG8 GX_PIXEL_FORMAT_BAYER_GB8 GX_PIXEL_FORMAT_BAYER_BG8	GX_PIXEL_FORMAT_MONO8 GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8 GX_PIXEL_FORMAT_RGBA8 GX_PIXEL_FORMAT_BGRA8 GX_PIXEL_FORMAT_ARGB8 GX_PIXEL_FORMAT_ABGR8 GX_PIXEL_FORMAT_RGB8_PLANAR
Bayer 16bit	GX_PIXEL_FORMAT_BAYER_GR10 GX_PIXEL_FORMAT_BAYER_RG10 GX_PIXEL_FORMAT_BAYER_GB10 GX_PIXEL_FORMAT_BAYER_BG10 GX_PIXEL_FORMAT_BAYER_GR12 GX_PIXEL_FORMAT_BAYER_RG12 GX_PIXEL_FORMAT_BAYER_GB12 GX_PIXEL_FORMAT_BAYER_BG12 GX_PIXEL_FORMAT_BAYER_GR16 GX_PIXEL_FORMAT_BAYER_RG16 GX_PIXEL_FORMAT_BAYER_GB16 GX_PIXEL_FORMAT_BAYER_BG16	GX_PIXEL_FORMAT_MONO16 GX_PIXEL_FORMAT_RGB16 GX_PIXEL_FORMAT_BGR16 GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8 GX_PIXEL_FORMAT_RGB16_PLANAR
RGB 8bit	GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8	GX_PIXEL_FORMAT_YUV444_8 GX_PIXEL_FORMAT_YUV422_8 GX_PIXEL_FORMAT_YUV411_8 GX_PIXEL_FORMAT_YUV420_8_PLANAR GX_PIXEL_FORMAT_YCBCR444_8 GX_PIXEL_FORMAT_YCBCR422_8 GX_PIXEL_FORMAT_YCBCR411_8 GX_PIXEL_FORMAT_MONO8 GX_PIXEL_FORMAT_RGB8

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

RGB 16bit	GX_PIXEL_FORMAT_RGB16 GX_PIXEL_FORMAT_BGR16	GX_PIXEL_FORMAT_MONO16
Mono Packed	GX_PIXEL_FORMAT_MONO10_PACKED GX_PIXEL_FORMAT_MONO12_PACKED GX_PIXEL_FORMAT_MONO10_P GX_PIXEL_FORMAT_MONO12_P GX_PIXEL_FORMAT_MONO14_P	GX_PIXEL_FORMAT_MONO8 GX_PIXEL_FORMAT_MONO10 GX_PIXEL_FORMAT_MONO12 GX_PIXEL_FORMAT_MONO14 GX_PIXEL_FORMAT_MONO16 GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8
Bayer Packed	GX_PIXEL_FORMAT_BAYER_GR10_PACKED GX_PIXEL_FORMAT_BAYER_RG10_PACKED GX_PIXEL_FORMAT_BAYER_GB10_PACKED GX_PIXEL_FORMAT_BAYER_BG10_PACKED GX_PIXEL_FORMAT_BAYER_GR12_PACKED GX_PIXEL_FORMAT_BAYER_RG12_PACKED GX_PIXEL_FORMAT_BAYER_GB12_PACKED GX_PIXEL_FORMAT_BAYER_BG12_PACKED GX_PIXEL_FORMAT_BAYER_GR10_P GX_PIXEL_FORMAT_BAYER_RG10_P GX_PIXEL_FORMAT_BAYER_GB10_P GX_PIXEL_FORMAT_BAYER_BG10_P GX_PIXEL_FORMAT_BAYER_GR12_P GX_PIXEL_FORMAT_BAYER_RG12_P GX_PIXEL_FORMAT_BAYER_GB12_P GX_PIXEL_FORMAT_BAYER_BG12_P GX_PIXEL_FORMAT_BAYER_GR14_P GX_PIXEL_FORMAT_BAYER_RG14_P GX_PIXEL_FORMAT_BAYER_GB14_P GX_PIXEL_FORMAT_BAYER_BG14_P	GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8 GX_PIXEL_FORMAT_BAYER_GR8 GX_PIXEL_FORMAT_BAYER_RG8 GX_PIXEL_FORMAT_BAYER_GB8 GX_PIXEL_FORMAT_BAYER_BG8 GX_PIXEL_FORMAT_BAYER_GR10 GX_PIXEL_FORMAT_BAYER_RG10 GX_PIXEL_FORMAT_BAYER_GB10 GX_PIXEL_FORMAT_BAYER_BG10 GX_PIXEL_FORMAT_BAYER_GR12 GX_PIXEL_FORMAT_BAYER_RG12 GX_PIXEL_FORMAT_BAYER_GB12 GX_PIXEL_FORMAT_BAYER_BG12 GX_PIXEL_FORMAT_BAYER_GR14 GX_PIXEL_FORMAT_BAYER_RG14 GX_PIXEL_FORMAT_BAYER_GB14 GX_PIXEL_FORMAT_BAYER_BG14 GX_PIXEL_FORMAT_BAYER_GR16 GX_PIXEL_FORMAT_BAYER_RG16 GX_PIXEL_FORMAT_BAYER_GB16 GX_PIXEL_FORMAT_BAYER_BG16
COOR D3D_C16	GX_PIXEL_FORMAT_COORD3D_C16	GX_PIXEL_FORMAT_COORD3D_ABC32F GX_PIXEL_FORMAT_COORD3D_ABC32F_PLANAR

6.18.2. SetDstFormat

Interface definition:

void SetDstFormat(GX_PIXEL_FORMAT_ENTRY emDstFormat)

Function description:

Set the desired conversion format.

Parameter:

emDstFormat	[in]	The desired conversion format is detailed as GX_PIXEL_FORMAT_ENTRY and Support conversion types

6.18.3. GetDstFormat

Interface definition:

GX_PIXEL_FORMAT_ENTRY GetDstFormat()

Function description:

Get the desired conversion format.

6.18.4. SetInterpolationType

Interface definition:

void SetInterpolationType (GX_BAYER_CONVERT_TYPE_LIST emConvertType)

Function description:

Set the image format conversion algorithm.

Parameter:

emConvertType	[in]	Conversion algorithm, see details as GX_BAYER_CONVERT_TYPE_LIST Default Value: GX_RAW2RGB_NEIGHBOUR

6.18.5. GetInterpolationType

Interface definition:

GX_BAYER_CONVERT_TYPE_LIST GetInterpolationType ()

Function description:

Get the image format conversion algorithm.

6.18.6. SetAlphaValue

Interface definition:

void SetAlphaValue(ulong ui64AlphaValue)

Function description:

Set the Alpha value of the image with Alpha channel.

Parameter:

ui64AlphaValue	[in]	Alpha channel value, the value range is 0~255, the default value is 255.

6.18.7. GetAlphaValue

Interface definition:

ulong GetAlphaValue()

Function description:

Get the Alpha channel value.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.18.8. GetBufferSizeForConversion

Interface definition:

ulong GetBufferSizeForConversion(GxIAPINET.IBaseData pObjSrcImageData)

Function description:

Get the image Buffer length in the desired format based on the input image pointer.

Parameter:

pObjSrcImageData	[in]	The source image pointer, which can be accessed via the Acquisition callback delegate function and GetImage()

6.18.9. GetBufferSizeForConversion

Interface definition:

ulong GetBufferSizeForConversion(ulong ui64SrcWidth,
ulong ui64SrcHeight,
GX_PIXEL_FORMAT_ENTRY emSrcFormat)

Function description:

Get the image Buffer length in the desired format based on the input image data.

Parameter:

ui64SrcWidth	[in]	Source image width
ui64SrcHeight	[in]	Source image height
emSrcFormat	[in]	Source image format

6.18.10. Convert

Interface definition:

void Convert(GxIAPINET.IBaseData objSrcImageData,
System.IntPtr pDstBuffer,
ulong nDstBufferSize,
bool bFlip)

Function description:

Convert the input source image to the desired image format, convertible formats refer to Support conversion types.

Parameter:

objSrcImageData	[in]	The source image pointer, which can be accessed via the Acquisition callback delegate function and GetImage()
pDstBuffer	[in]	Destination image Buffer pointer, user requested memory
	[out]	Destination Image Buffer

nDstBufferSize	[in]	The length of the destination image Buffer, available via GetBufferSizeForConversion()
bFlip	[in]	Whether to flip (true-flip, false-no-flip)

6.18.11. Convert

Interface definition:

```
void Convert(IntPtr pSrcBuffer, ulong ui64SrcWidth,
             ulong ui64SrcHeight,
             GX_PIXEL_FORMAT_ENTRY emSrcFormat,
             IntPtr pDstBuffer,
             ulong nDstBufferSize, bool bFlip)
```

Function description:

Convert the input source image to the desired image format, convertible formats refer to Support conversion types.

Parameter:

pSrcBuffer	[in]	Source Image Buffer Pointer
ui64SrcWidth	[in]	Source image width
ui64SrcHeight	[in]	Source image height
emSrcFormat	[in]	Source image format
pDstBuffer:	[in]	Destination image Buffer pointer, user requested memory
	[out]	Destination image Buffer data
nDstBufferSize	[in]	The length of the destination image Buffer, available via GetBufferSizeForConversion()
Parameter bFlip	[in]	Whether to flip (true-flip, false-no-flip)

6.18.12. SetValidBits

Interface definition:

```
void SetValidBits(GX_VALID_BIT_LIST emValidBits)
```

Function description:

Set the reserved valid data bits when converting the input source image to the desired image format.

Parameter emValidBits:	[in]	See GX_VALID_BIT_LIST for valid data bits

6.18.13. GetValidBits

Interface definition:

```
GX_VALID_BIT_LIST GetValidBits()
```

Function description:

Obtain the reserved valid data bits when converting the input source image to the desired image format.

6.18.14. Set3DCalibParam

Interface definition:

void Set3DCalibParam(IntPtr pCalibParamBuffer, int nBufferSize)

Function description:

This function is used to load 3D calibration parameters and convert the height map into a point cloud map.

Parameter:

pCalibParamBuffer	[in]	3D calibration parameter memory
nBufferSize	[in]	3D calibration parameter memory length

6.18.15. Get3DCalibParam

Interface definition:

IntPtr Get3DCalibParam()

Function description:

Obtain 3D calibration parameters.

6.18.16. SetYStep

Interface definition:

void SetYStep(double dY)

Function description:

Set the Y-direction step size for converting the height map to a point cloud.

Parameter DY:	[in]	Step size in Y direction

6.18.17. GetYStep

Interface definition:

double GetYStep()

Function description:

Obtain the Y-direction step size of the height transition point cloud.

6.19. IGXImageProcess

Image processing function class, users can improve image quality through this object, including Static Defect Correction, Noise Reduction, Sharpening, Color Correction, Brightness, Contrast, Gamma adjustment and other functions.

Interface List：

StaticDefectCorrection	Static defect correction
ImageImprovment	Method1: Input image data for image quality improvement
ImageImprovment	Method2: Input image pointer for image quality improvement

CalculaLut	Camera lookup table calculation
ReadLutFile	Read LUT file

6.19.1. StaticDefectCorrection

Interface definition:

void StaticDefectCorrection(IntPtr pInputBuffer,
IntPtr pOutputBuffer,
GX_STATIC_DEFECT_CORRECTION stGxDefectCorrection,
IntPtr pDefectPosBuffer,
UInt32 nDefectPosBufferSize)

Function description:

Execute static defect correction command.

Parameter:

pInputBuffer	[in]	Source image Buffer pointer
pOutputBuffer	[in]	Destination image Buffer pointer
	[out]	Target image Buffer data
stGxDefectCorrection	[in]	Source image parameters See details in GX_STATIC_DEFECT_CORRECTION
pDefectPosBuffer	[in]	Defect pixel location data (Read data from defect pixel files, which are saved as dp files through StaticDefectCorrection plugin)
nDefectPosBufferSize	[in]	Defect pixel location data Buffer size

6.19.2. ImageImprovment

Interface definition:

void ImageImprovment(ref GX_IMAGE_INFO stImgInfo,
IntPtr pOutputBuf,
IImageProcessConfig^ objCfg)

Function description:

Improve image quality. Support input formats: RGB/BGR, Bayer, Mono.

Parameter:

stImgInfo	[in]	Source image information, see details in GX_IMAGE_INFO
pOutputBuf	[in]	Destination image Buffer pointer
	[out]	Destination image Buffer data
objCfg	[in]	Image processing function configuration

Supported Functions	Source Image Format		
	RGB/BGR	Bayer	Mono
DefectivePixelCorrect	×	√	√
Sharpen	×	√	√
Contrast	√	√	√
Gamma	√	√	√
Lightness	×	√	√

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Denoise	×	√	×
ColorCorrection	√	√	×
Saturation	×	√	×
ConvertType	×	√	×
Flip	√	√	√
Accelerate	×	√	√

6.19.3. ImageImprovment

Interface definition:

void ImageImprovment (ref GxIAPINET.IBaseData objFrameData,
IntPtr pOutputBuf, IImageProcessConfig objCfg)

Function description:

Improve image quality. Support input formats: RGB/BGR, Bayer, Mono.

Parameter:

objFrameData	[in]	Source image pointer, get from Callback function Delegate Function Type Definition or GetImage()
pOutputBuf	[in]	Destination image Buffer pointer
	[out]	Destination image Buffer data
objCfg	[in]	Image processing function configuration Supported functions see section 6.19.2.

6.19.4. CalculaLut

Interface definition:

void CalculaLut(Int nContrastParam, double dGamma,
Int nLightness, IntPtr pLut, ref Int pnLutLength)

Function description:

Camera lookup table calculation.

Parameter:

nContrastParam	[in]	Contrast, value range: -50 ~ 100, which can be obtained through GetContrastParam()
dGamma	[in]	Gamma, value range: 0.1 ~ 10, which can be obtained through GetGammaParam()
nLightness	[in]	Brightness, value range: -150 ~ 150, which can be obtained through GetLightnessParam()
pLut	[in]	LUT memory pointer, applied by users, which length is pnLutLength
	[out]	Camera LUT obtained through calculation, which can be obtained through GetRegisterFeature (“LUTValueAll”) ->SetBuffer(pLut, pnLutLength) to set LUT
pnLutLength	[in]	LUT length, which can be obtained through GetRegisterFeature (“LUTValueAll”)->GetLength()

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6.19.5. ReadLutFile

Interface definition:

void ReadLutFile(string strLutFilePath, IntPtr pLut, ref Int pnLutLength)

Function description:

Read .lut file saved by the lookup table plugin and parse the data into lookup table data that can be set to the corresponding camera based on the length of the camera lookup table.

Parameter:

strLutFilePath	[in]	LUT file path, LUTfile (xxx.lut) can be obtained through LookUpTable plugin, and the plugin path is: “Galaxy View->Menu->Plugin->LookUpTable Plugin”
pLut	[in]	LUT memory pointer, applied by users, which length is pnLutLength
	[out]	Camera LUT obtained by reading files Which can be obtained through GetRegisterFeature (“LUTValueAll”)->SetBuffer (pLut, pnLutLength) to set LUT
pnLutLength	[in]	LUT length, unit in bytes, Which can be obtained through GetRegisterFeature (“LUTValueAll”)-> GetLength()

6.20. IGXFlatFieldCorrection

Flat field correction function class. Users can obtain the FFC coefficients which can be used to obtain the image after FFC through this object. For specific usage, refer to the GxFlatFieldCorrection code sample.

Interface List：

SetFrameCount	Set the fusion frame count.
GetCoefficientsSize	Get the size of the FFC coefficients.
Calculate	Generate the FFC coefficients.
FlatFieldCorrection	Get the image after FFC using the FFC coefficients.

6.20.1. SetFrameCount

Interface definition:

void SetFrameCount (UInt16 nFFCFrameCount)

Function description:

Set the fusion frame count. This interface must be called before calling the Calculate interface.

Parameter:

nFFCFrameCount	[in]	FFC fusion frame count (range: 1, 2, 4, 8, 16) indicates the number of images fused when calculating the FFC coefficient. Larger values are advised for images with significant noise.

Note: Values outside the input range will raise an exception.

6.20.2. GetCoefficientsSize

Interface definition:

Int32 GetCoefficientsSize(ref GX_FLAT_FIELD_CORRECTION_PARAMETER pstFFCParam)

Function description:

Get the size of the FFC coefficients. This interface must be called before calling the Calculate interface. It used to allocate the FFC coefficient cache after obtaining the coefficient size.

Parameter:

pstFFCParam	[in]	FFC parameters, the construction method can be found in GX_FLAT_FIELD_CORRECTION_PARAMETER.

6.20.3. Calculate

Interface definition:

void Calculate(ref GX_FLAT_FIELD_CORRECTION_PARAMETER pstFFCParam, IntPtr pFFCCoefficients, ref Int32 pnFFCCoefficientsSize)

Function description:

Calculate to generate the FFC coefficients.

Parameter:

pstFFCParam	[in]	FFC parameters, the construction method can be found in GX_FLAT_FIELD_CORRECTION_PARAMETER.
pFFCCoefficients	[out]	The FFC coefficient cache.
pnFFCCoefficientsSize	[in]	The size of the FFC coefficient cache. It can be obtained by calling the GetCoefficientsSize Interface.

After obtaining the FFC coefficients, whether the camera supports FFC function can be determined firstly by calling the FFCValueAll function node. If it does, it can be directly set for the camera through this node. If it does not, the image after FFC for each frame can be obtained by calling the FlatFieldCorrection interface.

6.20.4. FlatFieldCorrection

Interface definition:

void FlatFieldCorrection(IntPtr pSrcBuffer, IntPtr pDstBuffer, GX_ACTUAL_BITS nActualBits, UInt32 ui64SrcWidth, UInt32 ui64SrcHeight, IntPtr pFFCCoefficients, ref Int32 pnFFCCoefficientsSize)

Function description:

Get the image after FFC using the FFC coefficients.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameter:

pSrcBuffe	[in]	The source image buffer pointer, it can be obtained by calling the GetBuffer interface after obtaining the image through the DQBuf interface.
pDstBuffer	[out]	The buffer pointer after FFC, it can be abtainedby calling the GetBuffer interface after obtaining the image through the DQBuf interface.
emActualBit	[in]	Source image effective bits, see GX_ACTUAL_BITS.
ui64SrcWidth	[in]	The width of the source image, it can be obtained by calling the GetWidth interface in the image object.
ui64SrcHeight	[in]	The height of the source image, it can be obtained by calling the GetHeight interface in the image object.
pFFCCoefficients	[in]	The FFC coefficients, it can be obtained by calling the Calculate interface, length pnFFCCoefficientsSize.
pnFFCCoefficientsSize	[in]	The length of the FFC coefficients, it can be obtained by calling the GetCoefficientsSize interface.

6.21. IGXDecompressor

Decompression function class. The compressed data can be decompressed into raw data via this object. Refer to the GxDecompression code sample for usage details.

Interface List：

Decompression	Decompress the compressed data into raw data.

6.21.1. Decompression

Interface definition:

```
void Decompression(IntPtr pCompressionBuffer,
    UInt64 nCompressionSize,
    IntPtr pDecompressionBuffer,
    ref UInt64 nDecompressionSize,
    GX_PIXEL_FORMAT_ENTRY emPixelFormat,
    UInt64 ui64ImgWidth,
    UInt64 ui64ImgHeight,
    Int32 i32CompressionMedth)
```

Function description:

Decompress the acquired compressed data into raw data (include the frame information).

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameter:

pCompressionBuffer	[in]	The buffer address of the compressed image.
nCompressionSize	[in]	The buffer size of the compressed image.
pDecompressionBuffer	[in|out]	The buffer address of the decompressed image.
nDecompressionSize	[in|out]	The buffer size of the decompressed image.
emPixelFormat	[in]	The pixel format.
ui64ImgWidth	[in]	The image width.
ui64ImgHeight	[in]	The image height.
i32CompressionMedth	[in]	The decompression method.

6.22. IGxVideoSaver

Video saving function class. The captured images can be saved to local video through this object.

6.22.1. AddFrame

Interface definition:

void AddFrame (IntPtr* pImageBuf)

Function description:

Add frame images.

Parameter:

pImageBuf	[in]	The address of the frame images buffer.

6.22.2. Close

Interface definition:

void Close()

Function description:

Stop recording the video.

6.23. CgalaxyException

Abnormal interface class.

Interface List：

GetErrorCode	Retrieve internal exception error codes from GxIAPI library

6.23.1. GetErrorCode

Interface definition:

int GetErrorCode()

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Function description:

Retrieve the internal exception error code of GxIAPI library.

6.24. IGXImageSpecialFunction (No longer maintained)

Image special interface class.

Interface List：

CalculaLut	Camera lookup table calculation
ReadLutFile	Read lookup table file

6.24.1. CalculaLut

Interface definition:

void CalculaLut(Int nContrastParam,
double dGamma,
Int nLightness,
IntPtr pLut, ref Int pnLutLength)

Function description:

Camera lookup table calculation.

Parameter:

nContrastParam:	[in]	Contrast, value range: -50 ~ 100, available via GetContrastParam()
dGamma	[in]	Gamma, value range: 0.1~10, available via GetGammaParam()
nLightness	[in]	Brightness, value range: -150~150, available via GetLightnessParam()
pLut	[in]	Pointer to the lookup table memory, which is requested by the user and the length is pnLutLength
	[out]	Camera lookup table obtained by calculation, setting up the camera lookup table via GetRegisterFeature(“LUTValueAll”).SetBuffer(pLut, pnLutLength)
pnLutLength	[in]	Lookup Table length, available via GetRegisterFeature(“LUTValueAll”).GetLength()

6.24.2. ReadLutFile

Interface definition:

void ReadLutFile(string strLutFilePath, IntPtr pLut, ref Int pnLutLength)

Function description:

Read the .lut file saved by the lookup table plugin and parse the data into lookup table format that can be set to the appropriate camera based on the camera lookup table length.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameter:

strLutFilePath	[in]	Lookup table file path, lookup table file (xxx.lut) can be obtained by lookup table plugin Plugin path: "Galaxy View->Menu bar->Plugins->LUT Create Tool"
pLut	[in]	Pointer to the lookup table memory, which is requested by the users and the length is pnLutLength
	[out]	Set the camera lookup table via GetRegisterFeature("LUTValueAll").SetBuffer(pLut, pnLutLength)
pnLutLength	[in]	Lookup table length in bytes Can be obtained by GetRegisterFeature("LUTValueAll").GetLength()

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7. Revision History

No.	Version	API Version	Changes	Date
1	V1.0.4	1.0.1509.x or above	1. Initial release	2015-09
2	V1.0.6	1.0.1610.x or above	1. Modify description of device event acquisition	2016-01-05
3	V1.0.8	1.0.1602.x or above	1. Add some informations of GetImage 2. Added description that remote device are only supported by gigabit networks 3. Modify the sample image	2016-02-01
4	V1.0.10	1.0.1602.x or above	1. Modifiy the description of GetImage	2016-02-04
5	V1.0.11	1.0.1603.x or above	1. Modify he precautions for debugging the gigabit network camera 2. Added the description of modifying the device heartbeat timeout by adding environment variables	2016-03-22
6	V1.0.13	1.0.1607.x or above	1. Add the interface description of GigEIPConfiguration and GigEForceIp	2016-07-20
7	V1.0.15	1.0.1610.x or above	1. Add the interface description of the acquisition buffer	2016-10-24
8	V1.0.16	1.0.1612.x or above	1. Added description of camera internal error event: internal error event ID and time stamp	2016-12-14
9	V1.0.20	1.0.1907.x or above	1. Modify section 2.10.4 Gamma default value to 1.0, contrast default value to 0, saturation default value to 64	2019-07-01
10	V1.0.21	1.0.1907.x or above	1. Add new device reset and reconnect port 2. Add some description information to the ImageProcess interface	2019-07-16
11	V1.0.22	1.0.1908.x or above	1. Added an interface for obtaining the optimal packet length of the device 2. Add the code for setting the optimal packet length of the device	2019-08-15
12	V1.0.23	1.0.1909.x or above	1. Add a data type: COLOR_TRANSFORM_FACTOR 2. Add 4 user-set color correction factor interfaces in IImageProcessConfig	2019-09-29
13	V1.2.0	1.0.1912.x or above	1. Add related parameters of StreamBufferHandlingMode	2019-12-25

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

No.	Version	API Version	Changes	Date
14	V1.2.1	1.0.2008.x or above	1. Add the description of how to get the sample code for feature read & write in section 2.9.5	2020-08-28
15	V1.2.2	1.0.2010.x or above	1. Add the description about set the number of acquisition buffer	2020-10-12
16	V1.2.3	1.0.2109.x or above	1. Add the interface description of ReadPort and WirtePort	2021-09-24
17	V1.2.4	1.0.2201.x or above	1. Add IGXImageFormatConvert interface class 2. Add IGXImageSpecialFunction interface class 3. Add GX_PIXEL_FORMAT_ENTRY enumeration type	2022-01-14
18	V1.2.5	1.0.2203.x or above	1. Add ReadLutFile interface	2022-03-23
19	V1.2.6	1.0.2209.x or above	1. Add Read and Set attribute value in batch 2. Modify 6.18.1, add support conversion types	2022-09-23
20	V1.3.0	2.0.2403.x or above	1. Add 6.3 IGXInterface 2. Add 6.2 IGXInterfaceInfo 3. Add 2.6 and 2.7 section, Interface usage instructions 4. Add 3.14 GX_TL_TYPE_LIST enumeration type 5. Modify 2.5 section, update new enumeration type 6. Modify 2.10.1 section, add feature controller obtaining methods 7. Add 6.1 section: IGXFactory GetAllInterfaceInfo, IGXFactory GetInterface, IGXFactory UpdateAllDeviceListEx interfaces 8. Add IGXDevice GetParentInterface 9. Add IGXStream GetPayloadSize 10. Add 6.6 section: IGXFeatureControl FeatureSave, IGXFeatureControl FeatureSaveW, IGXFeatureControl FeatureLoad, IGXFeatureControl FeatureLoadW interfaces	2024-03-19
21	V1.3.1	2.0.2403.x or above	1. Add GX_PIXEL_COLOR_FILTER, GX_ACTUAL_BITS enumeration types. Supplement GX_VALID_BIT_LIST enumeration type 2. Add GX_STATIC_DEFECT_CORRECTION, GX_IMAGE_INFO data structures 3. Add 6.1.17 section: Create image processing object description 4. Modify 6.18.1 section, supplement and correct supported format conversion types 5. Add 6.18.12 section:	2024-04-03

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

No.	Version	API Version	Changes	Date
			IGXImageFormatConvert::SetValidBits interface 6. Add 6.18.13 section: IGXImageFormatConvert::GetValidBits interface 7. Add 6.19 section: IGXImageProcess class	
22	V1.3.2	2.0.2404.x or above	1. Expand GX_PIXEL_FORMAT_ENTRY enumeration type 2. Add 6.18.14 section: IGXImageFormatConvert::Set3DCalibParam interface 3. Add 6.18.15 section: IGXImageFormatConvert::Get3DCalibParam interface 4. Add 6.18.16 section: IGXImageFormatConvert::SetYStep interface 5. Add 6.18.17 section: IGXImageFormatConvert::GetYStep interface	2024-04-08
23	V1.3.3	2.0.2405.x or above	1. Add GX_LOG_TYPE_LIST type 2. Add 6.1.4 section, CGXFactory::SetLogType Interface 3. Add 6.1.5 section, CGXFactory::GetLogType Interface 4. Modify the definitions of all Interfaces	2024-06-12
24	V1.3.4	2.0.2406.x or above	1. Add 6.14.7. DQBuf 2. Add 6.14.8. QBuf	2024-07-10
25	V1.3.5	2.0.2412.x or above	1. Add 3.20.GX_GIGE_ACTION_COMMAND_RESULT 2. Add 6.1.18. GigEIssueActionCommand Add 6.1.19. GigEIssueScheduledActionCommand	2024-12-10
26	V1.3.6	2.0.2412.x or above	1. Add 6.14.15.RegisterBuffer 2. Add 6.14.16.UnRegisterBuffer 3. Add 6.15.12.GetUserParam 4. Add section 6.24 5. Optimize section description information	2025-01-08
27	V1.3.7	2.0.2412.x or above	1. Add section 2.11.7 Frame Information 2. Add section 6.15.13 GetChunkDataFeatureControl()	2025-05-07
28	V1.3.8	2.0.2412.x or above	1. Add section 6.20 IGXFlatFieldCorrection 2. Add section 3.21 GX_FLAT_FIELD_CORRECTION_PARAMETER	2025-06-10
29	V1.3.9	2.0.2412.x or above	1. Update section 2.13.3 Reconnection	2025-06-15

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

No.	Version	API Version	Changes	Date
30	V1.4.0	2.0.2412.x or above	1. Add section 3.22 GX_ENUM_ITEM_VALUE 2. Update section 6.9 IEnumFeature, add section 6.9.4 SetEnumValue, 6.9.5 GetEnumValue and 6.9.6 GetEnumValue	2025-09-02
31	V1.4.1	2.0.2412.x or above	1. Add section 6.1.20 CreateDecompressor 2. Add section 6.21 IGXDecompressor	2025-11-13
32	V1.4.2	2.0.2412.x or above	1. Add section 2.15 Properties Setting UI 2. Add section 2.16 Saving Images and Videos 3. Update chapter 3, add section 3.23 GX_WINDOWS_ID, section 3.24 GX_WINDOWS_SHOW_MODE, section 3.25 GX_CFA_METHOD, section 3.26 GX_IMAGE_FORMAT_TYPE, section 3.27 GX_VIDEO_FORMAT_TYPE, section 3.28 GX_SAVE_IMAGE_INFO, section 3.29 GX_RECORD_PARAM 4. Add section 6.1.21 SaveImage, section 6.1.22 CreateVideoSaver, section 6.5.21 GXCreateWnd, section 6.5.22 GXDestroyWnd, section 6.5.23 GXSetShowPosition, section 6.5.25 GXShowWnd, section 6.5.24GXSetShowMode, section 6.5.26 GXSetWndTitle, section 6.22 IGxVideoSaver	2026-01-13
33	V1.4.3	2.0.2412.x or above	1. Modify section IFrameData, add GetOffsetX() and GetOffsetY()	2026-05-06

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch